

Welcome to the Analytics track

OHDSI-UK 2026 Training

2026-09-17

Welcome to the OHDSI-UK training

Aims of the analytics track

By the end of the training:

- Be able to conduct descriptive analyses using R with data in the OMOP CDM format
- Understand how to approach writing bespoke code for complex studies
- Know where to look for help

What to expect

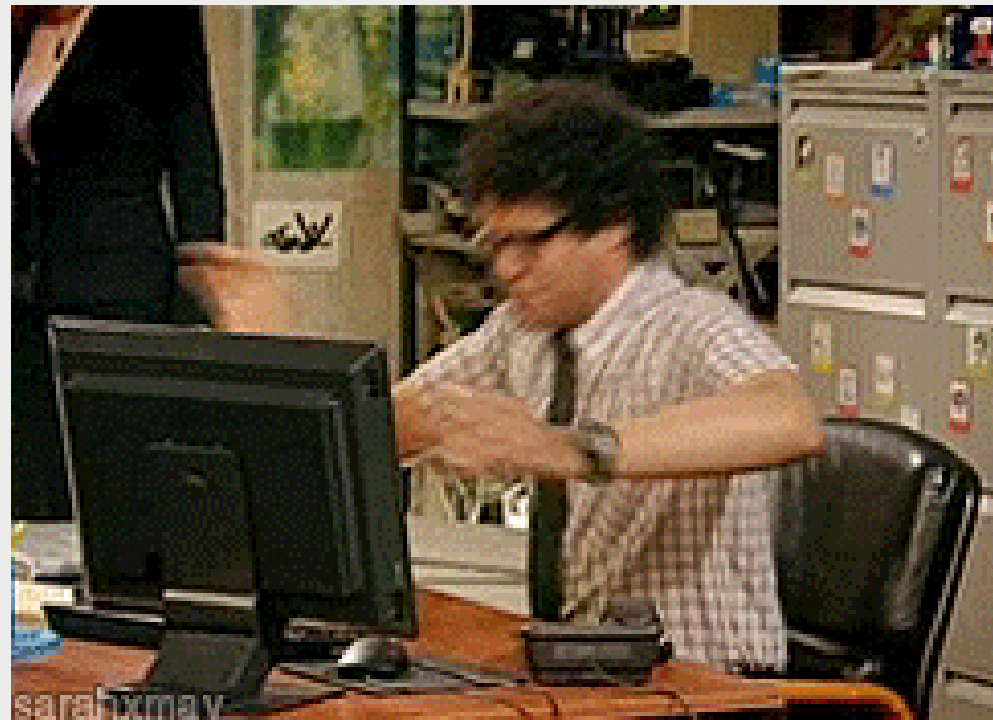


What to expect

- Quarto presentations - most presentations correspond to an R package that works with data in the OMOP CDM format.
- “Your Turn” exercises - during presentations where you can practice using the package (solutions will be on the next slide, but first try without looking at these!)
- General practical - time to apply what you have learned during the training.

Post-It notes

- Green - all good, no code currently running
- Yellow - code running
- Red - help me!



Set up

Download the needed data

- Contains the datasets you will need for today
- Open the *Analytics track* project with RStudio
- Run the **Setup.R** script in the *Your turn* folder
- Run the **createDatabases.R** script in the *Databases* folder
- Copy the databases to the *Back up* folder

Schedule

Time	Session
10:30 - 13:00	• Creating a CDM reference with CDMConnector • Characterise the OMOP CDM instance with OmopSketch • Writing custom queries using PatientProfiles and dbplyr
13:00 - 14:00	<i>Lunch break (Room D)</i>
14:00 - 16:30	• Creating cohorts using CohortConstructor • Characterise cohorts using CohortCharacteristics • Evaluating cohorts using PhenotypeR

Introductions

- What's your name?
- Where do you work? What is your role?
- Have you used R before?
- Have you worked with OMOP CDM data before?
- What do you want to get from this training?

Materials

All training materials are available from the **presentations** and **downloadable PDFs** on this website.

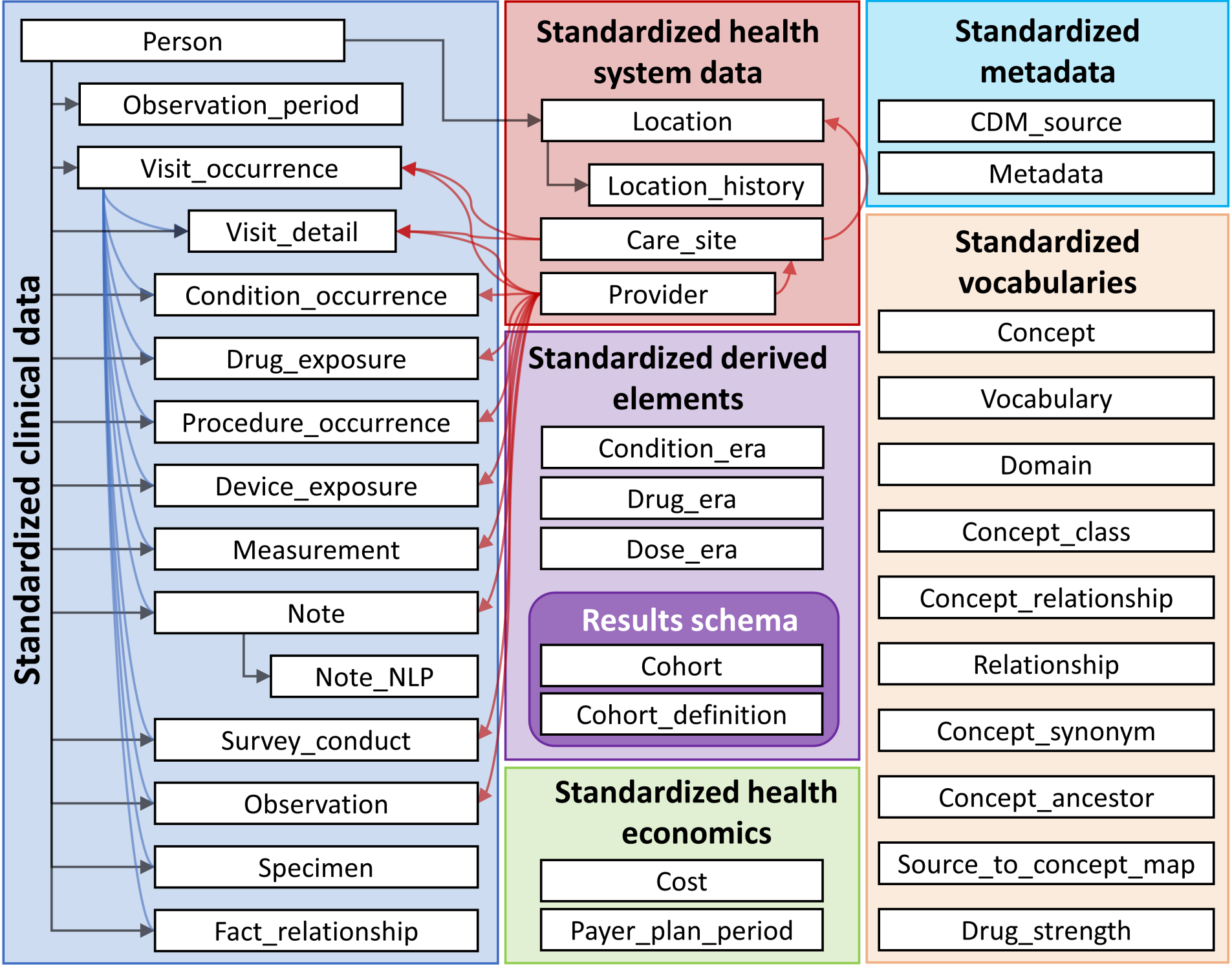
OMOP and CDMConnector

The OMOP Common Data Model

2026-09-17

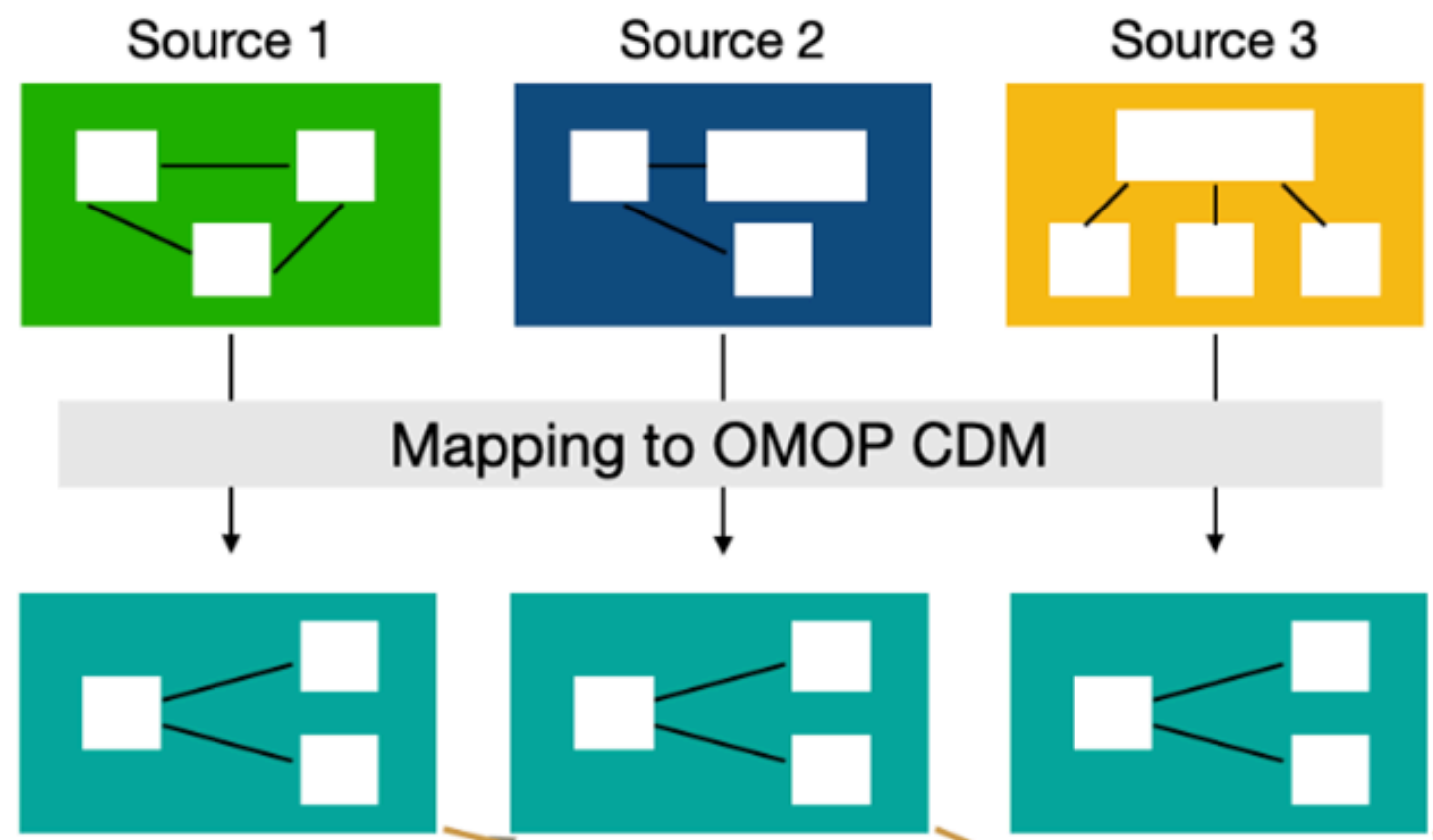
Introduction: The OMOP Common Data Model

Standardisation of the data format



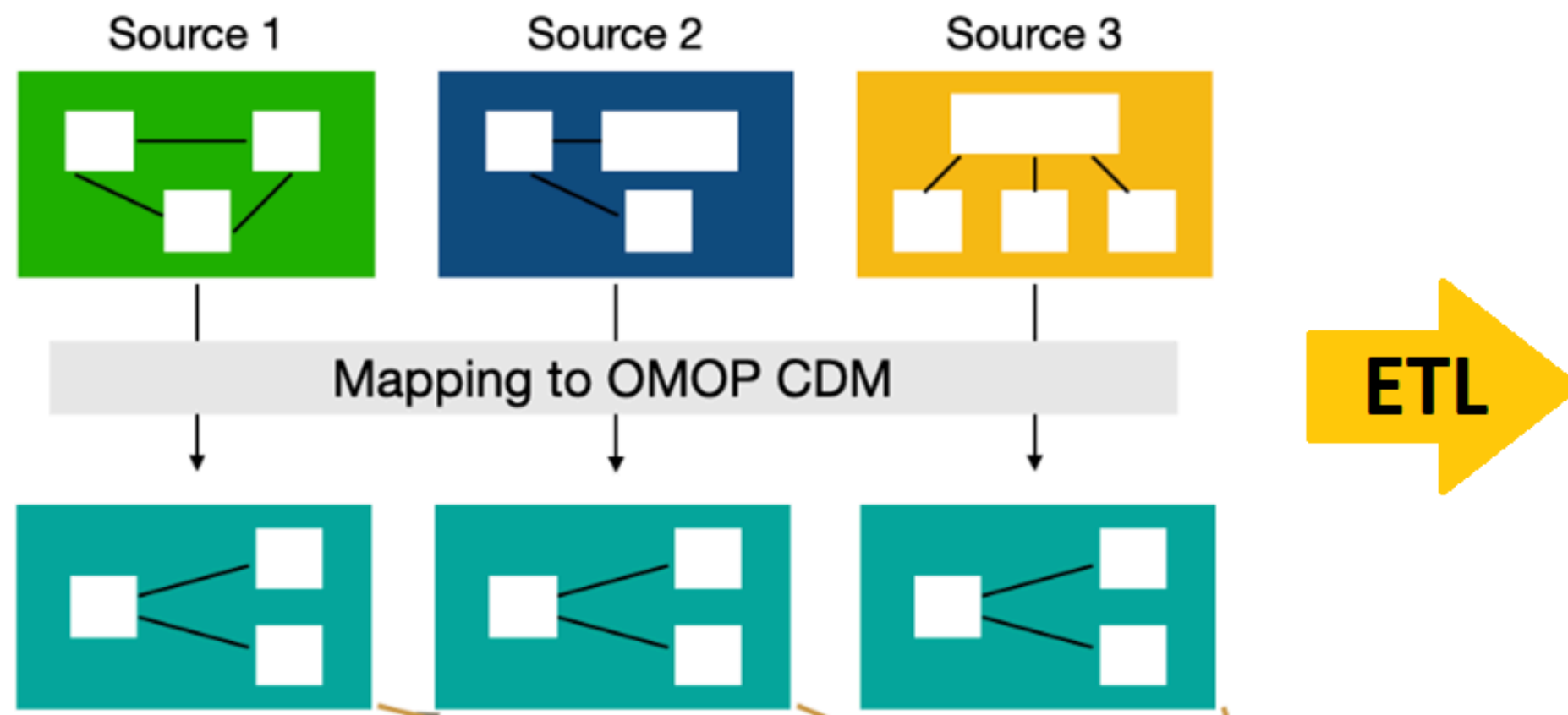
Tables and relation in the OMOP Common Data Model

Mapping a database to the OMOP CDM



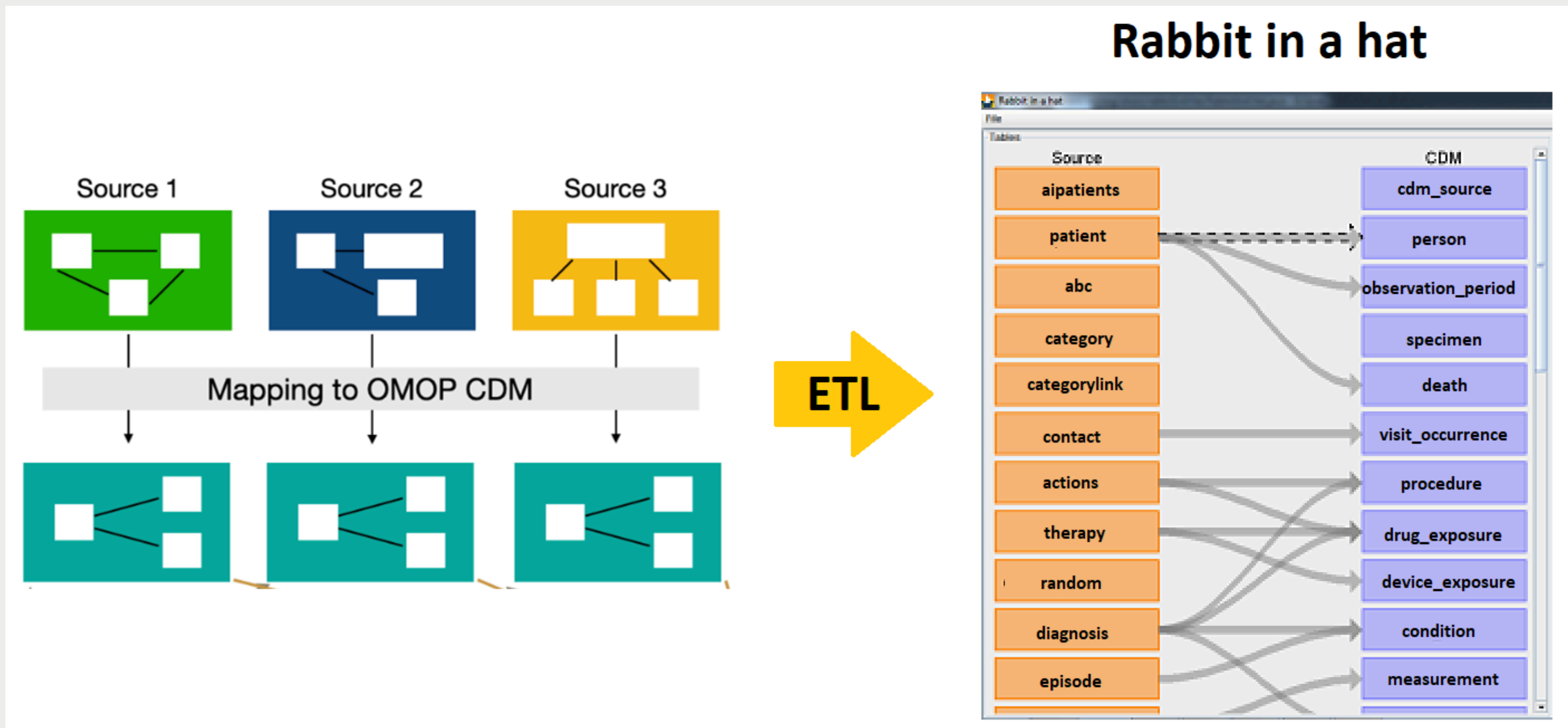
Mapping process

Mapping a database to the OMOP CDM



Mapping process

Mapping a database to the OMOP CDM



Mapping process

Standardisation of the vocabularies

From all the vocabularies OMOP CDM uses only a few as **Standard**: SNOMED for conditions, RxNorm for drugs, ...

The process to obtain an standard code from non standard one is called mapping. We can find the mapping in the concept_relationship table.

Each one of the records in clinical data tables (condition_occurrence, drug_exposure, measurement, observation, ...) will be coded by two codes:

- Source concept: particular to each database, it is the **original** code.
- Standard concept: equivalent code from the standard vocabulary.

Example of mapping

In concept relationship we can find different information such as:

```
> cdm$concept_relationship %>% select(relationship_id) %>% distinct() %>% pull()
[1] "AB-drug cjgt of" "Acc device used by" "Accepted use of" "Access of" "Active ing of" "Add monitor ind of" "Admin method of" "AMP of" "AMP restr ind of" "Answer of" "Answer of (PPI)" "Antineoplastic of"
[13] "ARP of" "Asso finding of" "Asso morph of" "Asso proc of" "Asso with finding" "ATC - RxNorm" "ATC - RxNorm pr lat" "ATC - RxNorm pr up" "ATC - RxNorm sec lat" "ATC - RxNorm sec up" "ATC - SNOMED eq" "ATC to NDFRT eq"
[25] "ATC to VA Class eq" "Available as box" "Basic dose form of" "Basis str subst of" "Biosimilar of" "Box of" "Brand name of" "CAP-Nebraska cat" "CAP-Nebraska eq" "CAP parent item of" "CAP protocol of" "CAP value of"
[37] "Category in Chapter" "Category of" "Causative agent of" "CD category of" "Chapter has Category" "Chapter to ICD0" "Characterizes" "Chem structure of" "Chem to Prep eq" "CI by" "CI chem class of" "CI MoA of"
[49] "CI physiol effect by" "CI to" "Clinical course of" "Combi prod ind of" "Comp material of" "Component of" "Conc denom unit of" "Conc denom val of" "Conc num unit of" "Conc num val of" "Concept alt_to from" "Concept alt_to to"
[61] "Concept poss_eq from" "Concept poss_eq to" "Concept replaces" "Concept same_as from" "Concept same_as to" "Concept was_a from" "Concept was_a to" "Consists of" "Constitutes" "Contained in" "Contained in panel"
[73] "Contained in version" "Contains" "Context of" "Count of act ing in" "Count of ing of" "CPT4 - LOINC eq" "CPT4 - SNOMED cat" "CPT4 - SNOMED eq" "Current treated by" "CVX - RxNorm" "Cytotox chemo RX of" "Cytotoxic chemo of"
[85] "Denom value of" "Denominator unit of" "Dev intended site of" "Device used by" "Diagnosed through" "Dir device of" "Dir morph of" "Dir proc site of" "Dir subst of" "Direct site of" "Disc indicator of" "Disp dose form of"
[97] "Disposition of" "DOI - RxNorm" "Dose form group of" "Dose form of" "Dose form unit of" "DRG - MDC cat" "DRG - MS-DRG eq" "Drug-drug inter for" "Drug class of drug" "Drug has drug class" "Due to of" "During"
[109] "End date of" "Endocrine tx of" "Energy used by" "Episodicity of" "ETC - RxNorm" "Excipient of" "FDA indication of" "FDA labeling of" "Filling of" "Finding asso with" "Finding context of" "Finding inform of"
[121] "Finding method of" "Finding site of" "Flavor of" "Focus of" "Followed by" "Follows" "Form continuity of" "Form of" "Free indicator of" "Has AB-drug cjgt" "Has AB-drug cjgt Rx" "Has accepted use"
[133] "Has access" "Has active ing" "Has add monitor ind" "Has admin method" "Has AMP" "Has AMP restr ind" "Has Answer" "Has answer (PPI)" "Has antineopl Rx" "Has antineoplastic" "Has ARP" "Has asso finding"
[145] "Has asso morph" "Has asso proc" "Has basic dose form" "Has basis str subst" "Has biosimilar" "Has brand name" "Has CAP parent item" "Has CAP protocol" "Has CAP value" "Has Category" "Has causative agent" "Has CD category"
[157] "Has chem structure" "Has CI" "Has CI chem class" "Has CI MoA" "Has CI physio effect" "Has clinical course" "Has combi prod ind" "Has comp material" "Has complication" "Has component" "Has conc denom unit" "Has conc denom val"
[169] "Has conc num unit" "Has conc num val" "Has context" "Has count of act ing" "Has count of ing" "Has cytotox chemo Rx" "Has cytotoxic chemo" "Has denomin value" "Has denominator unit" "Has dev intend site" "Has dir device" "Has dir morph"
[181] "Has dir proc site" "Has dir subst" "Has direct site" "Has disc indicator" "Has disp dose form" "Has disposition" "Has dose form" "Has dose form group" "Has dose form unit" "Has drug-drug inter" "Has due to" "Has end date"
[193] "Has endocrine tx" "Has endocrine tx Rx" "Has episodicity" "Has excipient" "Has FDA appr ind" "Has FDA indication" "Has FDA labeling" "Has filling" "Has finding context" "Has finding site" "Has flavor" "Has focus"
[205] "Has form" "Has form continuity" "Has free indicator" "Has Histology ICD0" "Has immunosuppr Rx" "Has immunosuppressor" "Has immunotherapy" "Has immunotherapy Rx" "Has indir device" "Has indir morph" "Has indir proc site" "Has inherent"
[217] "Has inherent loc" "Has intended site" "Has intent" "Has interpretation" "Has interprets" "Has kind" "Has laterality" "Has legal category" "Has licensed route" "Has life circumstan" "Has local therap Rx" "Has local therapy"
[229] "Has manifestation" "Has marketed form" "Has measurement" "Has metabolism" "Has metabolites" "Has method" "Has MoA" "Has modality" "Has modification" "Has non-avail ind" "Has numerator unit" "Has numerator value"
[241] "Has occurrence" "Has off-label ind" "Has ontological form" "Has parent item" "Has part of" "Has pathology" "Has pept-drug cjt Rx" "Has pept-drug cjt" "Has permiss range" "Has physio effect" "Has specimen" "Has specimen morph"
[253] "Has PPI parent code" "Has prec ingredient" "Has precise ing" "Has precondition" "Has precoord pair" "Has priority" "Has proc context" "Has proc device" "Has proc duration" "Has proc morph" "Has spec active ing" "Has supportive med"
[265] "Has prod character" "Has product comp" "Has property" "Has property type" "Has quantified form" "Has question source" "Has radiocjgt Rx" "Has radioconjugate" "Has radiotherapy" "Has radiotherapy Rx" "Has recipient cat" "Has relat context"
[277] "Has relative part" "Has release charact" "Has revision status" "Has role" "Has route" "Has route of admin" "Has scale type" "Has setting" "Has severity" "Has spec active ing" "Has specimen" "Has specimen morph"
[289] "Has specimen proc" "Has specimen source" "Has specimen subst" "Has specimen topo" "Has stage" "Has start date" "Has state of matter" "Has sub-specimen" "Has subject matter" "Has supplier" "Has support med Rx" "Has supportive med"
[301] "Has surface char" "Has surgical appr" "Has system" "Has targeted therapy" "Has targeted tx Rx" "Has technique" "Has temp finding" "Has temporal context" "Has therap class" "Has time aspect" "Has Topography ICD0" "Has trade family grp"
[313] "Has tradename" "Has transformation" "Has type" "Has type of service" "Has unit" "Has unit of admin" "Has unit of presen" "Has unit of prod use" "Has Value" "Has variant" "Has VMP" "Has VRP"
[325] "Histology of ICD0" "Historic treated by" "HOI - MedDRA" "HOI - SNOMED" "ICD0 br to OncoTree" "ICD0 to Chapter" "ICD0 to OncoTree eq" "ICD0 to Proc Schema" "ICD0 to Schema" "Immunosuppressor of" "Immunotherapy of" "Ind/CI - SNOMED"
[337] "Indir device of" "Indir morph of" "Indir proc site of" "Induced by" "Induces" "Is a" "Is off-label ind of" "Inherent location of" "Inheres in" "Inhibits effect" "Intended site of" "Intent of" "Interpretation of"
[349] "Is a" "Is characterized by" "Is CI of" "Is current in" "Is FDA-appr ind of" "Is historical in" "Is off-label ind of" "Is transcribed from" "Is translated from" "Kind of" "Legal category of" "Laterality of"
[361] "Licensed route of" "Life circumstan of" "Local therapy of" "LOINC - CPT4 eq" "Manifestation of" "Mapped from" "Maps to" "Maps to unit" "Maps to value" "Marketed form of" "May be inhibited by" "May be prevented by"
[373] "May be route of" "May be treated by" "May diagnose" "May have route" "May prevent" "Measurement of" "MedDRA - HOI" "MedDRA - SMQ" "MedDRA - SNOMED eq" "MedDRA - SNOMED eq" "Metabolism of"
[385] "Metabolite of" "Method of" "MoA of" "Modality of" "Modification of" "MS-DRG - DRG eq" "Multilex has ing" "Multilex ing of" "NDFRT - RxNorm eq" "NDFRT - RxNorm name" "NDFRT - SNOMED eq" "NDFRT dose form of"
[397] "NDFRT has dose form" "NDFRT has ing" "NDFRT ing of" "NDFRT to ATC eq" "NDFRT to VA Class eq" "Nebraska-CAP cat" "Nebraska-CAP eq" "Nebraska-CAP eq" "Non-avail ind of" "Numerator unit of" "Occurrence of" "Occurs after"
[409] "Occurs before" "OncoTree to ICD0 br" "OncoTree to ICD0 eq" "Ontological form of" "Panel contains" "Paral imprt ind" "Paral imprt of" "Parent item of" "Part of" "Pathology of" "Pept-drug cjt of" "Permiss range of"
[421] "Physiol effect by" "Physiol state of" "PK of" "Plays role" "PPI parent code of" "Prec ingredient of" "Precise ing of" "Precondition of" "Precoord pair of" "Prep to Chem eq" "Priority of" "Proc context of"
[433] "Proc device of" "Proc duration of" "Proc morph of" "Proc Schema to ICD0" "Proc site of" "Process output of" "Prod character of" "Product comp of" "Property of" "Property type of" "Quantified form of" "Question source of"
[445] "Radioconjugate of" "Radiotherapy of" "Recipient cat of" "Relat context of" "Relative part of" "Relative to" "Release charact of" "Revision status of" "Role of" "Role played by" "Route of"
[457] "Route of admin of" "Rx AB-drug cjt of" "Rx antineopl of" "Rx endocrine tx of" "Rx immunosuppr of" "Rx immunotherapy of" "Rx local therapy of" "Rx pept-drug cjt of" "Rx radiocjgt of" "Rx support med of" "Rx targeted tx of"
[469] "RxNorm - ATC" "RxNorm - ATC pr lat" "RxNorm - ATC sec lat" "RxNorm - ATC sec up" "RxNorm - CVX" "RxNorm - DOI" "RxNorm - ETC" "RxNorm - NDFRT eq" "RxNorm - NDFRT name" "RxNorm - SNOMED eq" "RxNorm - Source eq"
[481] "RxNorm - SPL" "RxNorm - VAProd eq" "RxNorm dose form of" "RxNorm has dose form" "RxNorm has ing" "RxNorm inverse is a" "RxNorm is a" "RxNorm - NDFRT eq" "RxNorm - NDFRT name" "RxNorm - SNOMED eq" "RxNorm - Source eq"
[493] "Setting of" "Severity of" "SMQ - MedDRA" "SNOMED - ATC eq" "SNOMED - CPT4 eq" "SNOMED - HOI" "SNOMED - ind/CI" "SNOMED - MedDRA eq" "SNOMED - NDFRT eq" "SNOMED - RxNorm eq" "SNOMED - VA Class eq" "SNOMED cat - CPT4"
[505] "Source - RxNorm eq" "Spec active ing of" "Specimen identity of" "Specimen morph of" "Specimen of" "Specimen proc of" "Specimen subst of" "Specimen topo of" "SPL - RxNorm" "Stage of" "Start date of" "State of matter of"
[517] "Sub-specimen of" "Subject matter of" "Subst used by" "Subsumes" "Supplier of" "Supportive med of" "Surg character of" "Surgical appr of" "System of" "Targeted therapy of" "Technique of" "Temp related to"
[529] "Temporal context of" "Therap class of" "Time aspect of" "Topography of ICD0" "Trade family grp of" "Tradename of" "Transcribes to" "Transformation of" "Translates to" "Type of" "Type of service of" "Unit mapped from"
[541] "Unit of" "Unit of admin of" "Unit of presen of" "Unit of prod use of" "Using acc device" "Using device" "Using energy" "Using finding inform" "Using finding method" "Using subst" "VA Class - SNOMED eq" "VA Class to ATC eq"
[553] "VA Class to NDFRT eq" "Value mapped from" "Value of" "Value to Schema" "VAProd - RxNorm eq" "Variable to Schema" "Variant of" "Version contains" "VMP has prescr stat" "VMP of" "VMP prescr stat of"
[565] "VRP of" "VRP prescr stat of"
```

Concept relationship

In particular, we have the **Maps to** and **Mapped from** relations that can help us to see the mapping between codes.

Example of mapping

```
> cdm$concept_relationship %>% filter(relationship_id == "Maps to") %>% filter(concept_id_1 == 44975210)
# Source:   SQL [1 x 6]
# Database: postgres
  concept_id_1 concept_id_2 relationship_id valid_start_date valid_end_date invalid_reason
      <int>      <int> <chr>           <date>           <date>           <chr>
1    44975210    1790982 Maps to      2009-06-01      2099-12-31      ""
```

Mapping process

Example of mapping

```
> cdm$concept_relationship %>% filter(relationship_id == "Maps to") %>% filter(concept_id_1 == 44975210)
# Source:   SQL [1 x 6]
# Database: postgres
  concept_id_1 concept_id_2 relationship_id valid_start_date valid_end_date invalid_reason
      <int>      <int> <chr>           <date>           <date>           <chr>
1    44975210    1790982 Maps to       2009-06-01       2099-12-31       ""
> cdm$concept %>% filter(concept_id %in% c(44975210, 1790982))
# Source:   SQL [2 x 10]
# Database: postgres
  concept_id concept_name      domain_id vocabulary_id concept_class_id standard_concept concept_code valid_start_date valid_end_date
      <int> <chr>                <chr>      <chr>        <chr>          <chr>          <chr>      <date>         <date>
1    1790982 chlorhexidine gluconate 1.2 MG/ML Mouthwash Drug      RxNorm      Clinical Drug      S           834127       2009-03-01      2099-12-31
2    44975210 chlorhexidine gluconate 1.2 MG/ML Mouthwash Drug      NDC         11-digit NDC      NA          00116200116 2009-06-01      2099-12-31
```

Mapping process

Example of mapping

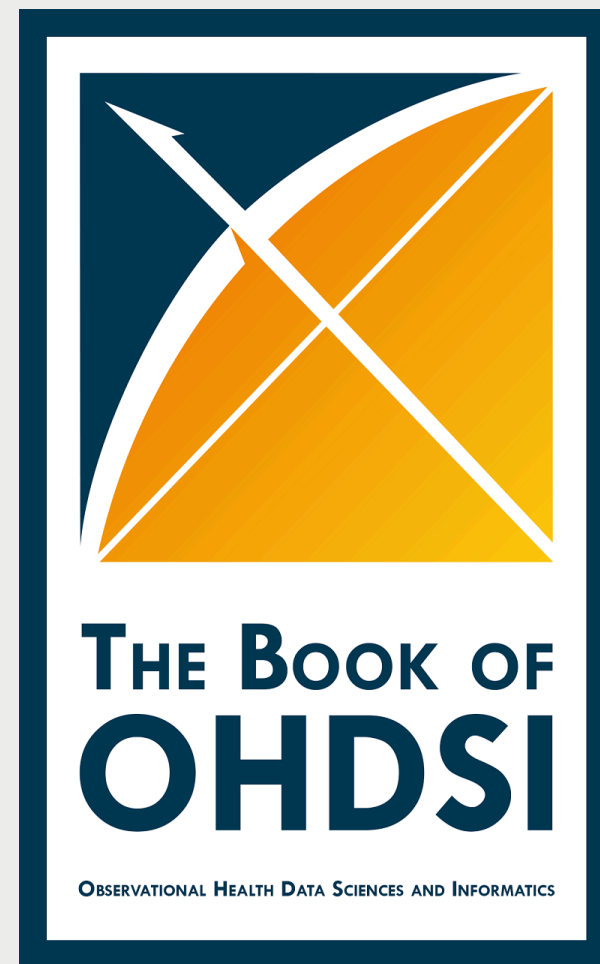
```
> cdm$concept_relationship %>% filter(relationship_id == "Maps to") %>% filter(concept_id_1 == 44975210)
# Source:   SQL [1 x 6]
# Database: postgres
  concept_id_1 concept_id_2 relationship_id valid_start_date valid_end_date invalid_reason
      <int>      <int> <chr>              <date>           <date>           <chr>
1    44975210    1790982 Maps to          2009-06-01       2099-12-31       ""
> cdm$concept %>% filter(concept_id %in% c(44975210, 1790982))
# Source:   SQL [2 x 10]
# Database: postgres
  concept_id concept_name          domain_id vocabulary_id concept_class_id standard_concept concept_code valid_start_date valid_end_date
      <int> <chr>                  <chr>      <chr>          <chr>          <chr>          <chr>      <date>           <date>
1    1790982 chlorhexidine gluconate 1.2 MG/ML Mouthwash Drug      RxNorm        Clinical Drug      S            834127       2009-03-01       2099-12-31
2    44975210 chlorhexidine gluconate 1.2 MG/ML Mouthwash Drug      NDC           11-digit NDC      NA           00116200116 2009-06-01       2099-12-31
> cdm$drug_exposure %>%
+   filter(drug_concept_id == 1790982) %>%
+   select(drug_exposure_id, drug_concept_id, drug_exposure_start_date, drug_exposure_end_date, drug_source_concept_id)
# Source:   SQL [?? x 5]
# Database: postgres
  drug_exposure_id drug_concept_id drug_exposure_start_date drug_exposure_end_date drug_source_concept_id
      <int64>      <int> <date>              <date>              <int>
1    123429592    1790982 2017-12-15          2017-12-30          44975210
2    778541702    1790982 2018-02-08          2018-02-23          44975210
3    4043828552    1790982 2018-01-05          2018-01-20          44975210
4    2594409161    1790982 2018-04-24          2018-05-25          44975210
5    1942091989    1790982 2017-01-24          2017-02-08          44975210
6    1857857471    1790982 2021-11-10          2021-11-25          44975210
7    2801754279    1790982 2021-11-10          2021-11-25          44975210
8    3908339319    1790982 2017-04-05          2017-04-18          44975210
9    2123508505    1790982 2017-04-12          2017-04-25          44975210
10   1310852067    1790982 2017-03-15          2017-03-28          44975210
# i more rows
# i Use `print(n = ...)` to see more rows
>
```

Mapping process

More details

For more details on how the vocabularies work you can check: **Vocabulary course in EHDEN academy**

All details about OMOP CDM and more can be found in: **the book of ohdsi.**



The book of ohdsi cover

Let's start coding

These are the packages that we will use in this presentation:

```
library(omock)
library(DBI)
library(CDMConnector)
library(duckdb)
library(here)
library(dplyr)
library(omopgenerics)
library(visOmopResults)
```

You can click on the specific functions to see **?help** and what package they come from:

```
cdmFromCon()
```

Create a mock reference

You can create a mock database using *omock* from one of the **28** available synthetic databases:

```
availableMockDatasets()
```

```
[1] "delphi-100k"           "delphi-100k_5.4"       "empty_cdm"
[4] "empty_cdm_5.3"        "empty_cdm_5.4"        "GiBleed"
[7] "synpuf-1k"            "synpuf-1k_5.3"        "synpuf-1k_5.4"
[10] "synthea-allergies-10k" "synthea-breast_cancer-10k" "synthea-contraceptives-10k"
[13] "synthea-covid19-10k"  "synthea-covid19-200k"  "synthea-dermatitis-10k"
[16] "synthea-heart-10k"    "synthea-hiv-10k"      "synthea-lung_cancer-10k"
[19] "synthea-medications-10k" "synthea-metabolic_syndrome-10k" "synthea-opioid_addiction-10k"
[22] "synthea-rheumatoid_arthritis-10k" "synthea-snf-10k"      "synthea-surgery-10k"
[25] "synthea-total_joint_replacement-10k" "synthea-veteran_prostate_cancer-10k" "synthea-veterans-10k"
[28] "synthea-weight_loss-10k"
```

```
cdm <- mockCdmFromDataset(datasetName = "GiBleed")
```

```
i Loading bundled GiBleed tables from package data.
```

```
i Adding drug_strength table.
```

```
i Creating local <cdm_reference> object.
```

```
cdm
```

— # OMOP CDM reference (local) of GiBleed —

- omop tables: care_site, cdm_source, concept, concept_ancestor, concept_class, concept_relationship, concept_synonym, condition_era, condition_occurrence, cost, death, device_exposure, domain, dose_era, drug_era, drug_exposure, drug_strength, fact_relationship, location, measurement, metadata, note, note_nlp, observation, observation_period, payer_plan_period, person, procedure_occurrence, provider, relationship, source_to_concept_map, specimen, visit_detail, visit_occurrence, vocabulary

- cohort tables: -
- achilles tables: -
- other tables: -

[← Back to Presentations](#)

<cdm_reference> object

```
class(cdm)
```

```
[1] "cdm_reference"
```

```
cdmName(cdm)
```

```
[1] "GiBleed"
```

```
cdmVersion(cdm)
```

```
[1] "5.3"
```

```
cdmSource(cdm)
```

```
This is a local cdm source
```

<cdm_table> object

```
cdm$person
```

```
# A tibble: 2,694 × 18
  person_id gender_concept_id year_of_birth month_of_birth day_of_birth birth_datetime race_concept_id
*   <int>         <int>         <int>         <int>         <int> <dtm>         <int>
1         6         8532         1963          12          31 1963-12-31 00:00:00      8516
2        123         8507         1950           4          12 1950-04-12 00:00:00      8527
3        129         8507         1974          10           7 1974-10-07 00:00:00      8527
4         16         8532         1971          10          13 1971-10-13 00:00:00      8527
5         65         8532         1967           3          31 1967-03-31 00:00:00      8516
6         74         8532         1972           1           5 1972-01-05 00:00:00      8527
7         42         8532         1909          11           2 1909-11-02 00:00:00      8527
8        187         8507         1945           7          23 1945-07-23 00:00:00      8527
9         18         8532         1965          11          17 1965-11-17 00:00:00      8527
10        111         8532         1975           5           2 1975-05-02 00:00:00      8527
# i 2,684 more rows
```

```
class(cdm$person)
```

```
[1] "omop_table" "cdm_table"  "tbl_df"     "tbl"        "data.frame"
```

Connecting to a database from R (the DBI package)

In general the *OMOP* datasets that we will use won't be locally, and they will on a database.

Database connections from R can be made using the **DBI** package.

Connect to postgres:

```
library(RPostgres)
db <- dbConnect(
  Postgres(),
  dbname = "...",
  host = "...",
  user = "...",
  password = "..."
)
```

Connecting to a database from R (the DBI package)

Connect to Sql server:

```
library(odbc)
db <- dbConnect(
  odbc(),
  Driver      = "ODBC Driver 18 for SQL Server",
  Server      = "...",
  Database    = "...",
  UID         = "...",
  PWD         = "...",
  TrustServerCertificate = "yes",
  Port        = "..."
)
```

In this [CDMConnector article](#) you can see how to connect to the different supported DBMS.

Databases organisation

Databases are organised by **schemas** (blueprint or plan that defines how the data will be organised and structured within the database).

In general, OMOP databases have two schemas:

- **cdm schema**: it contains all the tables of the cdm. Usually we only will have reading permission for this schema.
- **write schema**: it is a place where we can store tables (like cohorts). We need writing permissions to this schema.

Connect to duckdb database

duckdb is a package that allows us to work with local databases.

You should see that in the *Database* folder there are three datasets:

- **GiBleed**
- **Delphi**
- **Synpuf**

Let's create our first cdm reference

```
# connect to the database
dbdir <- here("Databases", "GiBleed.duckdb")
con <- dbConnect(drv = duckdb(dbdir = dbdir))
```

duckdb keeps downloaded extensions and secrets in a temporary directory:

i /tmp/RtmpTB9E3p/duckdb

This is removed when the R session ends.

- Extensions are re-downloaded each session.
- Secrets are lost.

i Run `duckdb(shared_home = TRUE)` (or create `~/.duckdb`) to keep them (suitable for most users).

i Run `duckdb(shared_home = FALSE)` to accept the temporary directory (and silence this message).

i See `?duckdb_storage` for details and alternatives.

```
cdm <- cdmFromCon(con = con, cdmSchema = "main", writeSchema = "results")
```

Note: method with signature `'DBIConnection#Id'` chosen for function `'dbExistsTable'`,
target signature `'duckdb_connection#Id'`.

`"duckdb_connection#ANY"` would also be valid

```
cdm
```

— # OMOP CDM reference (duckdb) of Synthea —

- omop tables: `care_site`, `cdm_source`, `concept`, `concept_ancestor`, `concept_class`, `concept_relationship`, `concept_synonym`, `condition_era`, `condition_occurrence`, `cost`, `death`, `device_exposure`, `domain`, `dose_era`, `drug_era`, `drug_exposure`, `drug_strength`, `fact_relationship`, `location`, `measurement`, `metadata`, `note`, `note_nlp`, `observation`, `observation_period`, `payer_plan_period`, `person`, `procedure_occurrence`, `provider`, `relationship`, `source_to_concept_map`, `specimen`, `visit_detail`, `visit_occurrence`, `vocabulary`

- cohort tables: -

- achilles tables: -

- other tables: -

Access to tables of the cdm reference

```
cdm$person
```

```
# A query:  ?? x 18
# Database: DuckDB 1.5.5 [unknown@Linux 6.17.0-1022-azure:R 4.6.1//home/runner/work/OHDSI-UK-2026-Training/OHDSI-UK-2026-Training/Databases/GiBleed.duckdb]
```

	person_id	gender_concept_id	year_of_birth	month_of_birth	day_of_birth	birth_datetime	race_concept_id
	<int>	<int>	<int>	<int>	<int>	<dtm>	<int>
1	6	8532	1963	12	31	1963-12-31 00:00:00	8516
2	123	8507	1950	4	12	1950-04-12 00:00:00	8527
3	129	8507	1974	10	7	1974-10-07 00:00:00	8527
4	16	8532	1971	10	13	1971-10-13 00:00:00	8527
5	65	8532	1967	3	31	1967-03-31 00:00:00	8516
6	74	8532	1972	1	5	1972-01-05 00:00:00	8527
7	42	8532	1909	11	2	1909-11-02 00:00:00	8527
8	187	8507	1945	7	23	1945-07-23 00:00:00	8527
9	18	8532	1965	11	17	1965-11-17 00:00:00	8527
...	...	...	...	...	...	...	...

Read tables in GiBleed

Once we read a table we can operate with it and for example count the number of rows of person table.

```
cdm$person |>  
  count()
```

```
# A query:  ?? x 1
```

```
# Database: DuckDB 1.5.5 [unknown@Linux 6.17.0-1022-azure:R 4.6.1//home/runner/work/OHDSI-UK-2026-Training/OHDSI-UK-2026-Training/Databases/GiBleed.duckdb]
```

```
      n  
<dbl>  
1 2694
```

Operation with tidyverse

If you are familiarised with **tidyverse** you can use any of the usual **dplyr** commands in you database tables.

```
cdm$drug_exposure |>
  group_by(drug_concept_id) |>
  summarise(number_persons = n_distinct(person_id))
```

```
# A query: ?? x 2
```

```
# Database: DuckDB 1.5.5 [unknown@Linux 6.17.0-1022-azure:R 4.6.1//home/runner/work/OHDSI-UK-2026-Training/OHDSI-UK-2026-Training/Databases/GiBleed.duckdb]
```

	drug_concept_id	number_persons
	<int>	<dbl>
1	19133873	1272
2	40213306	1560
3	19133905	856
4	19073188	240
5	920300	630
6	19128065	605
7	46275444	35
8	19019979	300
9	19073183	157

How does this work?

```
cdm$drug_exposure |>
  group_by(drug_concept_id) |>
  summarise(number_persons = n_distinct(person_id)) |>
  show_query()
```

<SQL>

```
SELECT drug_concept_id, COUNT(DISTINCT row(person_id)) AS number_persons
FROM drug_exposure
GROUP BY drug_concept_id
```

Database name

When we have a cdm object we can check which is the name of that database using:

```
cdmName(cdm)
```

```
[1] "Synthea"
```

In some cases we want to give a database a name that we want, this can be done at the connection stage:

```
cdm <- cdmFromCon(  
  con = con, cdmSchema = "main", writeSchema = "results", cdmName = "GiBleed"  
)
```

```
cdmName(cdm)
```

```
[1] "GiBleed"
```

Create a new table

```
cdm$my_saved_table <- cdm$condition_occurrence |>
  filter(condition_concept_id == 4112343) |>
  select(person_id, condition_start_date) |>
  compute(name = "my_saved_table")
```

```
cdm$my_custom_name <- cdm$person |>
  compute(name = "not_my_custom_name")
```

Error in `[<-`:

✖ You can't assign a table named not_my_custom_name to my_custom_name.

i You can change the name using compute:

```
cdm[['my_custom_name']] <- yourObject |>
  dplyr::compute(name = 'my_custom_name')
```

i You can also change the name using the `name` argument in your function: `name = 'my\_custom\_name'`.

Use a prefix

It is **VERY IMPORTANT** that when we create the cdm object we use a prefix:

```
cdm <- cdmFromCon(  
  con = con,  
  cdmSchema = "main",  
  writeSchema = "results",  
  writePrefix = "my_prefix_"  
)  
cdm
```

— # OMOP CDM reference (duckdb) of Synthea —

- omop tables: care_site, cdm_source, concept, concept_ancestor, concept_class, concept_relationship, concept_synonym, condition_era, condition_occurrence, cost, death, device_exposure, domain, dose_era, drug_era, drug_exposure, drug_strength, fact_relationship, location, measurement, metadata, note, note_nlp, observation, observation_period, payer_plan_period, person, procedure_occurrence, provider, relationship, source_to_concept_map, specimen, visit_detail, visit_occurrence, vocabulary
- cohort tables: -
- achilles tables: -
- other tables: -

omock

👉 Packages website

👉 CRAN link

👉 Manual

✉️ mike.du@ndorms.ox.ac.uk



omopgenerics

👉 Packages website

👉 CRAN link

👉 Manual

✉️ marti.catalasabate@ndorms.ox.ac.uk

visOmopResults

👉 Packages website

👉 CRAN link

👉 Manual



nuria.mercadebesora@ndorms.ox.ac.uk



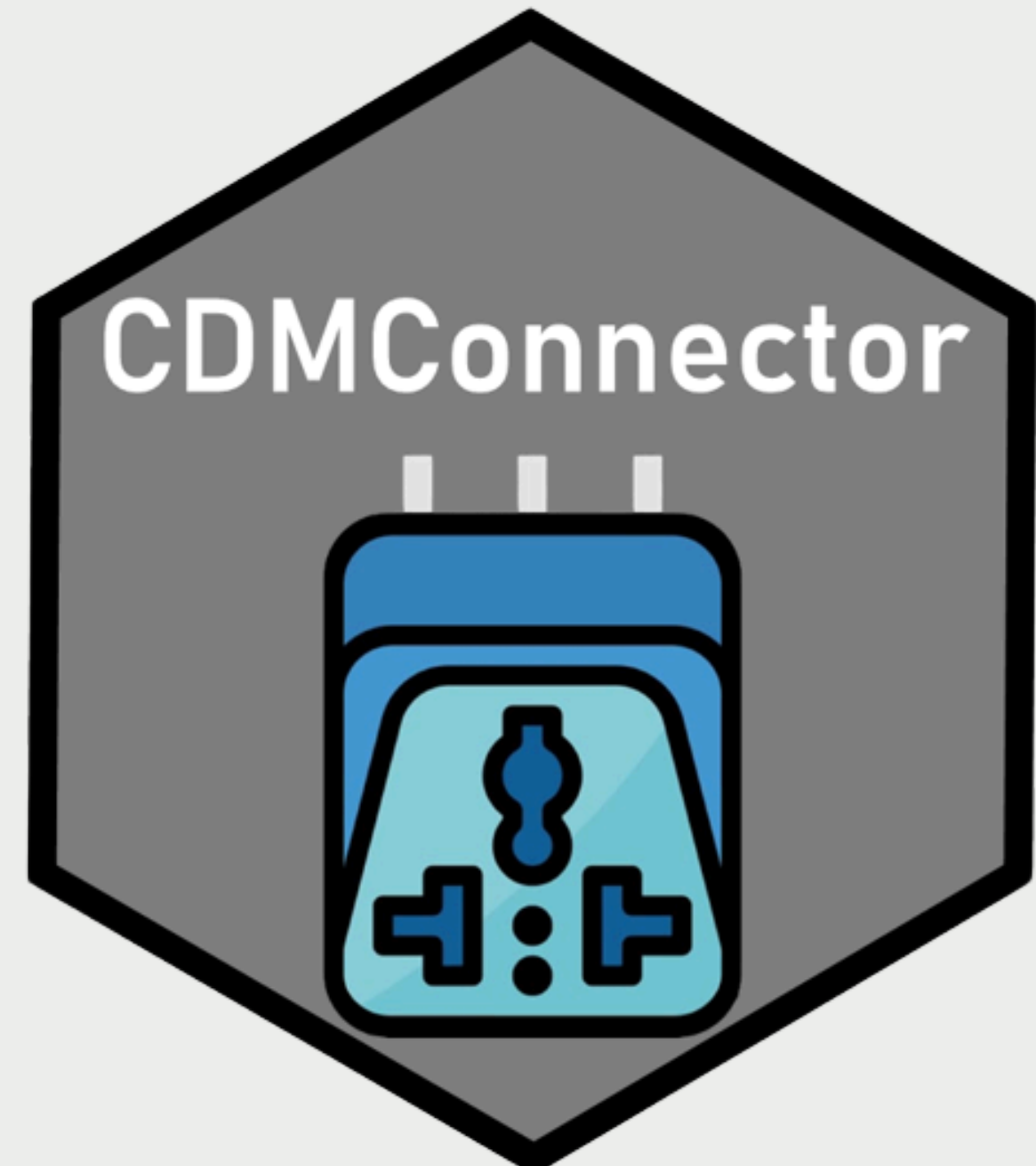
CDMConnector

👉 Packages website

👉 CRAN link

👉 Manual

✉️ marti.catalasabate@ndorms.ox.ac.uk



Thank you!

[← Back to Presentations](#)



OmapSketch

Characterise databases

2026-09-17

Overview and motivation

- OmopSketch is designed to **summarise key information** of an OMOP-mapped database.
- Two main use cases:
 - **General characterisation** — understand what's in your database
 - **Feasibility assessment** — is this database suitable for my study?
- Available on [CRAN](#) and [GitHub](#).



Setup

Load packages and create the CDM object

- Load the required packages

```
library(OmopSketch)
library(CDMConnector)
library(duckdb)
library(dplyr)
library(here)
library(omock)
```

- Create a CDM object from the **GiBleed** synthetic dataset

```
con <- dbConnect(drv = duckdb(dbdir = here("Databases", "GiBleed.duckdb")))
```

```
[1] 0
```

```
cdm <- cdmFromCon(
  con = con,
  cdmSchema = "main",
  writeSchema = "results",
  writePrefix = "os_"
)
```

- The **cdm** object is your gateway to all OmopSketch functions – every function takes **cdm** as its first argument.

1. Snapshot

What is a snapshot?

`summariseOmapSnapshot()` captures **metadata about the CDM at a point in time** — useful for reproducibility and maintaining clear records of changes.

Information provided:

- **Vocabulary version** — from the `vocabulary` table
- **Table sizes** — record counts for `person` and `observation_period`
- **Observation period span** — earliest and latest dates in the data
- **Source type** — origin or type of the data source

summariseO mopSnapshot ()

- Returns a **summarised result** object.

```
snapshot <- summariseO mopSnapshot(cdm = cdm)
```

```
glimpse(snapshot)
```

```
Rows: 16
Columns: 13
$ result_id      <int> 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1
$ cdm_name       <chr> "Synthea", "Synthea", "Synthea", "Synthea", "Synthea", "Synthea", "Synthea", "Synthea", "Synthea", "Synt...
$ group_name     <chr> "overall", "overall", "overall", "overall", "overall", "overall", "overall", "overall", "overall", "over...
$ group_level    <chr> "overall", "overall", "overall", "overall", "overall", "overall", "overall", "overall", "overall", "over...
$ strata_name    <chr> "overall", "overall", "overall", "overall", "overall", "overall", "overall", "overall", "overall", "over...
$ strata_level   <chr> "overall", "overall", "overall", "overall", "overall", "overall", "overall", "overall", "overall", "over...
$ variable_name  <chr> "general", "general", "general", "cdm", "cdm", "cdm", "cdm", "cdm", "cdm", "cdm", "observation_perio...
$ variable_level <chr> NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA
$ estimate_name  <chr> "snapshot_date", "person_count", "vocabulary_version", "source_name", "version", "holder_name...
$ estimate_type  <chr> "date", "integer", "character", "character", "character", "character", "character", "character", "characte...
$ estimate_value <chr> "2026-09-17", "2694", "v5.0 18-JAN-19", "Synthea synthetic health database", "5.3", "OHDSI Co...
$ additional_name <chr> "overall", "overall", "overall", "overall", "overall", "overall", "overall", "overall", "overall", "over...
```

tableOmopSnapshot ()

- Format the result into a **gt** table

```
tableOmopSnapshot(result = snapshot, type = "gt")
```

Snapshot of the cdm Synthea

Database name	
Synthea	
Estimate	
General	
Snapshot date	2026-09-17
Person count	2,694
Vocabulary version	v5.0 18-JAN-19
Cdm	
Source name	Synthea synthetic health database
Version	5.3
Holder name	OHDSI Community
Release date	2019-05-25
Description	SyntheaTM is a Synthetic Patient Population Simulator. The goal is to output synthetic, realistic (but not real), patient data and associated health records in a variety of formats.
Documentation reference	https://synthetichealth.github.io/synthea/
Observation period	
N	2,694

2. Observation period

What does the observation period tell us?

`summariseObservationPeriod()` summarises the **observation_period** table.

Information provided:

- **Number of records per person** — how many observation periods per individual
- **Duration in days** — length of each observation period
- **Days to next observation period** — gaps between consecutive periods
- **Data quality** (optional) — flags inconsistencies
- **Missingness** (optional) — counts of missing values per column

summariseObservationPeriod()

```
obs_result <- summariseObservationPeriod(cdm, missingData = TRUE, quality = TRUE)
glimpse(obs_result)
```

Rows: 3,126

Columns: 13

```
$ result_id      <int> 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, ...
$ cdm_name       <chr> "Synthea", "Synthea", "Synthea", "Synthea", "Synthea", "Synthea", "Synthea", "Synthea", "Synthea", "Synt...
$ group_name     <chr> "observation_period_ordinal", "observation_period_ordinal", "observation_period_ordinal", "ob...
$ group_level    <chr> "all", "all", "all", "all", "all", "all", "all", "all", "all", "all", "all", "all", "all", "all", "a...
$ strata_name    <chr> "overall", "overall", "overall", "overall", "overall", "overall", "overall", "overall", "overall", "over...
$ strata_level   <chr> "overall", "overall", "overall", "overall", "overall", "overall", "overall", "overall", "overall", "over...
$ variable_name  <chr> "Records per person", "Records per person", "Records per person", "Records per person", "Reco...
$ variable_level <chr> NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, N...
$ estimate_name  <chr> "mean", "sd", "min", "q05", "q25", "median", "q75", "q95", "max", "mean", "sd", "min", "q05",...
$ estimate_type  <chr> "numeric", "numeric", "integer", "integer", "integer", "integer", "integer", "integer", "inte...
$ estimate_value <chr> "1", "0", "1", "1", "1", "1", "1", "1", "1", "1", "14402.0014972862", "8725.3408283113", "1", "143...
$ additional_name <chr> "overall", "overall", "overall", "overall", "overall", "overall", "overall", "overall", "overall", "over...
```

tableObservationPeriod()

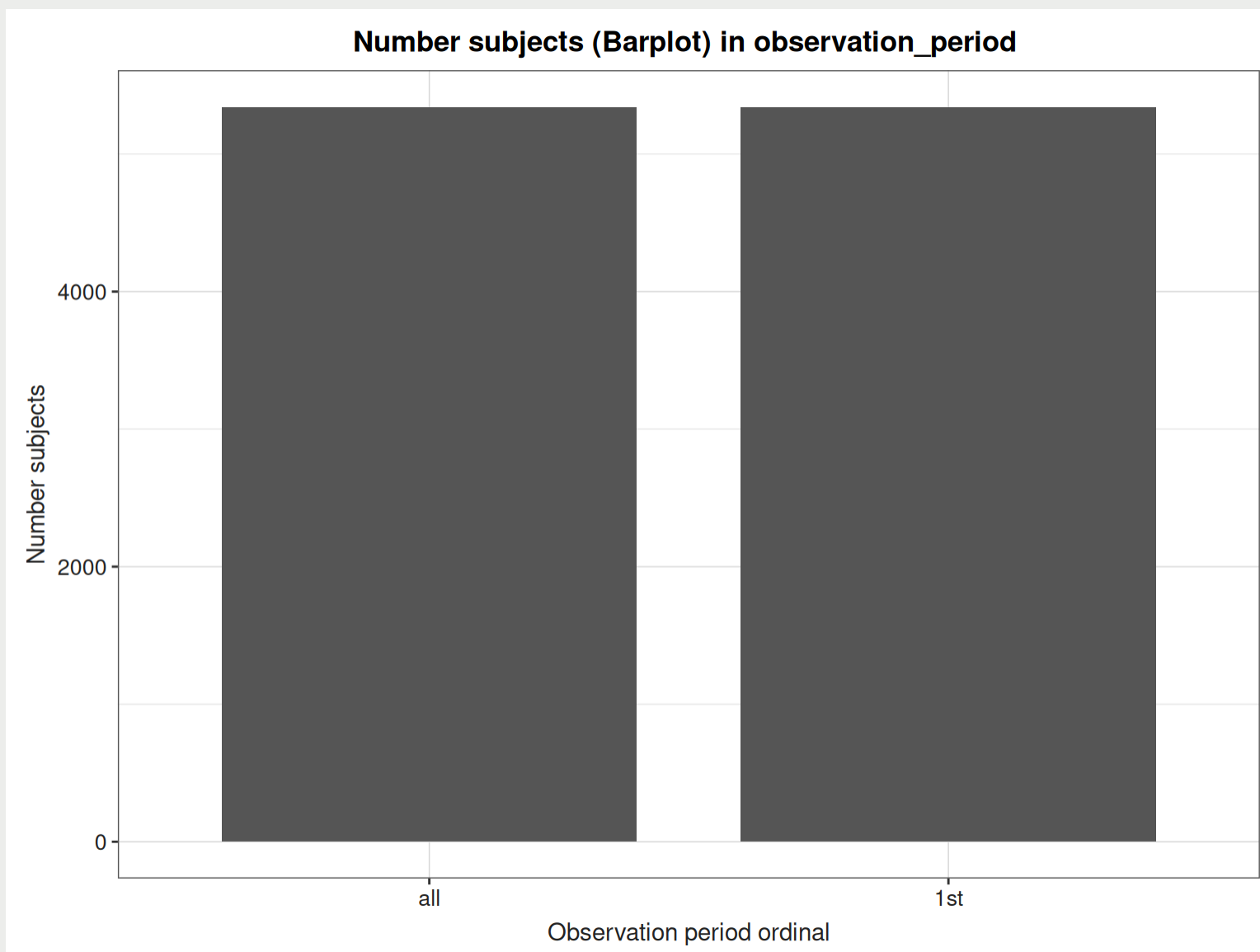
```
tableObservationPeriod(result = obs_result, type = "gt")
```

Summary of observation_period table

Observation period ordinal	Variable name	Variable level	Is required	Estimate name	Data source
					Synthea
all	Number records	–	overall	N	5,343
	Number subjects	–	overall	N	5,343
	Subjects not in person table	–	overall	N (%)	2,649 (49.58%)
	Records per person	–	overall	Mean (SD)	1.00 (0.00)
				Median [Q25 - Q75]	1 [1 - 1]
				Range [min to max]	[1 to 1]
	Duration in days	–	overall	Mean (SD)	14,402.00 (8,725.34)
				Median [Q25 - Q75]	13,868.00 [7,102.50 - 20,955.50]
				Range [min to max]	[1.00 to 40,349.00]
	Type concept id	Period covering healthcare encounters	overall	N (%)	5,343 (100.00%)
	Start date before birth date	–	overall	N (%)	0 (0.00%)
	End date before start date	–	overall	N (%)	0 (0.00%)
	Column name	Observation period end date	TRUE	N missing data (%)	0 (0.00%)

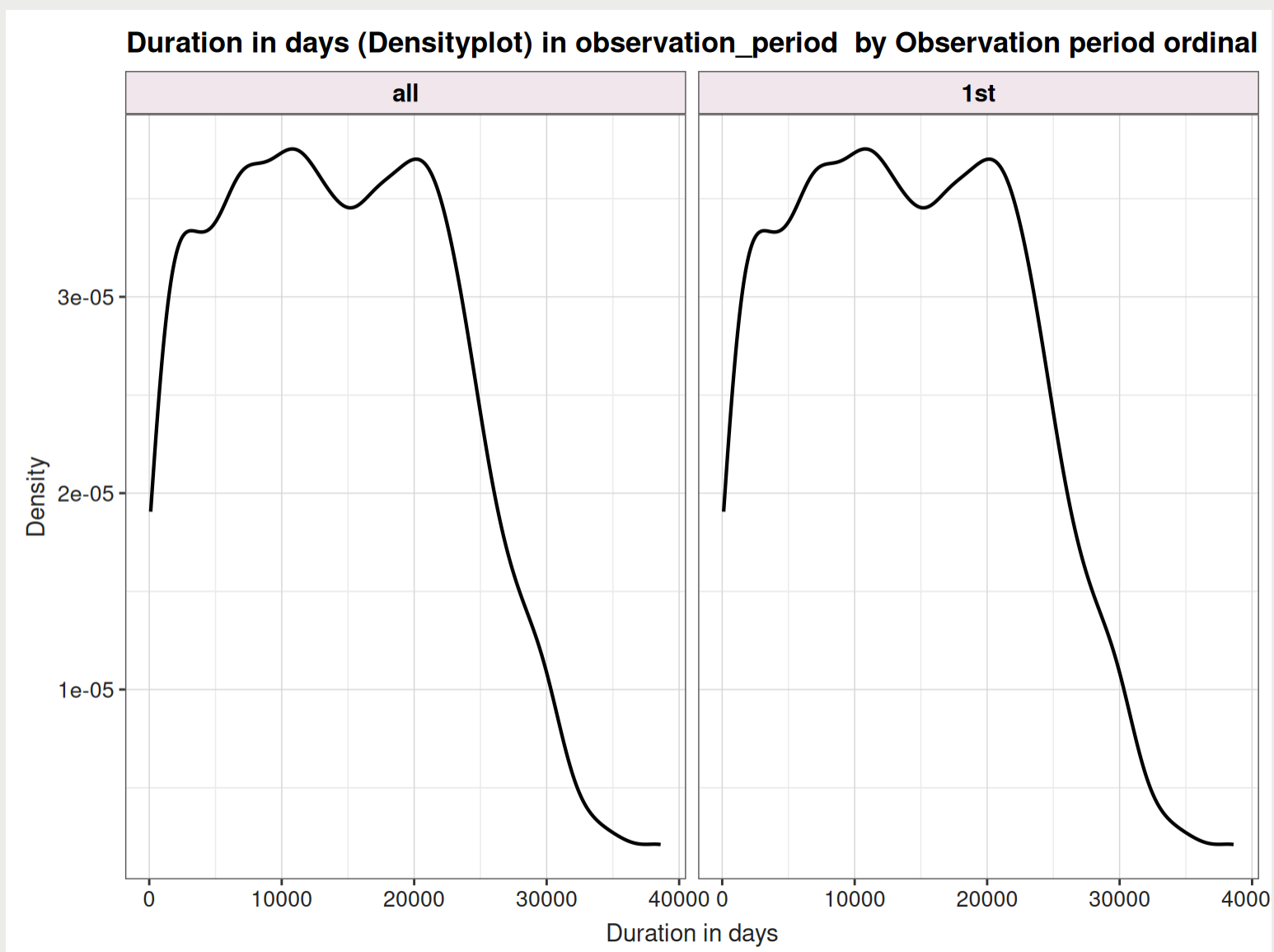
plotObservationPeriod() — Number of subjects

```
plotObservationPeriod(  
  result = obs_result,  
  plotType = "barplot",  
  variableName = "Number subjects"  
)
```



plotObservationPeriod() — Duration in days

```
plotObservationPeriod(  
  result = obs_result,  
  plotType = "densityplot",  
  variableName = "Duration in days",  
  facet = "observation_period_ordinal"  
)
```



3. Clinical tables

What does clinical table characterisation do?

`summariseClinicalRecords()` provides a **comprehensive audit** of any clinical table in the CDM.

Information provided:

summariseClinicalRecords()

```
clinical_result <- summariseClinicalRecords(  
  cdm          = cdm,  
  omopTableName = "condition_occurrence",  
  conceptSummary = TRUE,  
  missingData   = TRUE,  
  quality       = TRUE  
)
```

- You can pass multiple tables at once: `omopTableName = c("condition_occurrence", "drug_exposure")`

tableClinicalRecords()

```
tableClinicalRecords(result = clinical_result, type = "gt")
```

Summary of condition_occurrence table

Variable name	Variable level	Estimate name	Database name
			Synthea
condition_occurrence			
Number records	–	N	65,332
Number subjects	–	N (%)	2,694 (100.00%)
Subjects not in person table	–	N (%)	0 (0.00%)
Records per person	–	Mean (SD)	24.25 (7.41)
		Median [Q25 - Q75]	23 [19 - 29]
		Range [min to max]	[5 to 65]
Duration of records	–	Mean (SD)	53.17 (423.43)
		Median [Q25 - Q75]	14.00 [7.00 - 21.00]
		Range [min to max]	[0.00 to 18,979.00]
In observation	No	N (%)	450 (0.69%)
	Yes	N (%)	64,882 (99.31%)
Domain	Condition	N (%)	65,332 (100.00%)
Standard vocabulary	Snomed	N (%)	65,332 (100.00%)



Your turn: Clinical table audit

Task: Characterise **both** `drug_exposure` and `procedure_occurrence` in one call. Include missing data and quality checks.

Then answer:

1. Which table has more records per person on average?
2. What % of drug records are within the observation period?

►  [Click to see solution](#)

4. Trends over time

What do trends tell us?

- `summariseTrend()` reveals **how database content changes over time** — essential for spotting data gaps, coverage periods, and temporal changes.

event vs episode

`summariseTrend()` has two separate arguments for tables — and the choice changes **how records are counted across time intervals**.

event — tables where occurrences happen at a **single point in time**

- Each record is assigned to the interval containing its **start date only**
- e.g. a diagnosis on 3 March 2015 → counts in 2015, nowhere else
- Typical tables: `condition_occurrence`, `drug_exposure`, `procedure_occurrence`, `measurement`, `observation`, `visit_occurrence`, `death`

episode — tables that describe **periods spanning over time**

- Each record contributes to **every interval between its start and end date**
- e.g. an observation period from Jan 2015 to Dec 2016 → counts in both 2015 and 2016
- Typical tables: `observation_period`, `drug_era`, `condition_era`

Outputs available

output argument	What it counts
"record"	Number of records (default)
"person"	Number of distinct persons
"person-days"	Number of person-days (episode only)
"sex"	Number of females
"age"	Median age at start of period

summariseTrend()

```
trend_result <- summariseTrend(
  cdm      = cdm,
  event    = c("condition_occurrence", "drug_exposure"),
  interval = "years", # "years" | "quarters" | "months" | "overall"
  output   = "record", # "record" | "person" | "person-days" | "sex" | "age"
  dateRange = as.Date(c("1950-01-01", "2000-12-31"))
)

glimpse(trend_result)
```

Rows: 208

Columns: 13

```
$ result_id      <int> 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, ...
$ cdm_name       <chr> "Synthea", "Synthea", "Synthea", "Synthea", "Synthea", "Synthea", "Synthea", "Synthea", "Synthea", "Synt...
$ group_name     <chr> "omop_table", "omop_table", "omop_table", "omop_table", "omop_table", "omop_table", "omop_table", "omop_tab...
$ group_level    <chr> "condition_occurrence", "condition_occurrence", "drug_exposure", "drug_exposure", "condition_...
$ strata_name    <chr> "overall", "overall", "overall", "overall", "overall", "overall", "overall", "overall", "overall", "over...
$ strata_level   <chr> "overall", "overall", "overall", "overall", "overall", "overall", "overall", "overall", "overall", "over...
$ variable_name  <chr> "Number of records", "Number of records", "Number of records", "Number of records", "Number o...
$ variable_level <chr> NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, N...
$ estimate_name  <chr> "count", "percentage", "count", "percentage", "count", "percentage", "count", "percentage", "...
$ estimate_type  <chr> "integer", "percentage", "integer", "percentage", "integer", "percentage", "integer", "percen...
$ estimate_value <chr> "332", "0.77", "267", "0.61", "334", "0.78", "248", "0.57", "425", "0.99", "327", "0.75", "40...
$ additional_name <chr> "time_interval", "time_interval", "time_interval", "time_interval", "time_interval", "time_in...
```

summariseTrend() — check event/episode type

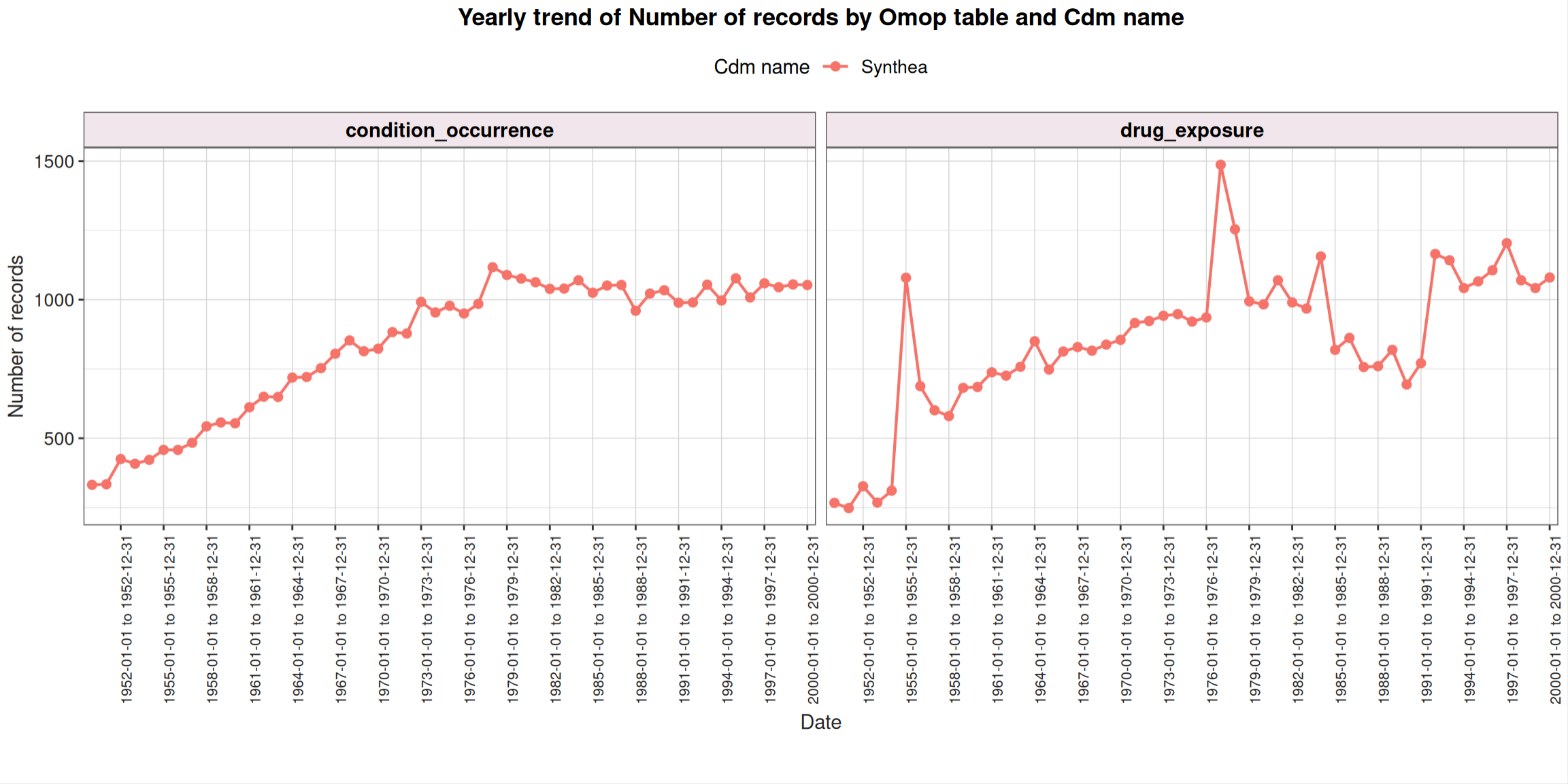
You can check how each table was treated in the result settings. The **type** column will show **"event"** or **"episode"** per table.

```
settings(trend_result) |> glimpse()
```

```
Rows: 1
Columns: 12
$ result_id      <int> 1
$ result_type    <chr> "summarise_trend"
$ package_name   <chr> "OmopSketch"
$ package_version <chr> "1.1.1"
$ group          <chr> "omop_table"
$ strata         <chr> ""
$ additional     <chr> "time_interval"
$ min_cell_count <chr> "0"
$ interval       <chr> "years"
$ study_period_end <chr> "2000-12-31"
$ study_period_start <chr> "1950-01-01"
$ type          <chr> "event"
```

plotTrend()

```
plotTrend(result = trend_result, facet = "omop_table", colour = "cdm_name")
```



tableTrend()

```
tableTrend(result = trend_result, type = "reactable")
```

Search							Database name
Type; OMOP table	Variable name	Time interval	Estimate name	Interval	Study period end	Study period start	Synthea
▼event; condition _occurre							



Your turn: Sex-stratified trends

Task: Create a plot showing the **yearly** trend in records for **drug_exposure** and **condition_occurrence**, stratified by **sex**, for the date range **1990–2010**.

Hints:

- `output = "record"` gives you record counts
- `sex = TRUE` gives you sex-stratified summaries
- `dateRange = as.Date(c("1990-01-01", "2010-12-31"))`
- Use `facet = "omop_table"` and `colour = "sex"` in the plot

▶  [Click to see solution](#)

5. Concept counts

summariseConceptIdCounts()

- Counts how often each concept ID appears — useful for **feasibility** (do we have enough patients with condition X?).

```
concept_result <- summariseConceptIdCounts(
  cdm          = cdm,
  omopTableName = "procedure_occurrence",
  countBy      = c("record", "person")
)
glimpse(concept_result)
```

Rows: 102

Columns: 13

```
$ result_id      <int> 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, ...
$ cdm_name       <chr> "Synthea", "Synthea", "Synthea", "Synthea", "Synthea", "Synthea", "Synthea", "Synthea", "Synthea", "Synt...
$ group_name     <chr> "omop_table", "omop_table", "omop_table", "omop_table", "omop_table", "omop_table", "omop_table", "omop_tab...
$ group_level    <chr> "procedure_occurrence", "procedure_occurrence", "procedure_occurrence", "procedure_occurrence...
$ strata_name    <chr> "overall", "overall", "overall", "overall", "overall", "overall", "overall", "overall", "overall", "over...
$ strata_level   <chr> "overall", "overall", "overall", "overall", "overall", "overall", "overall", "overall", "overall", "over...
$ variable_name  <chr> "Suture open wound", "Suture open wound", "Direct current cardioversion", "Direct current car...
$ variable_level <chr> "4125906", "4125906", "4078793", "4078793", "4163872", "4163872", "4037675", "4037675", "4181...
$ estimate_name  <chr> "count_records", "count_subjects", "count_records", "count_subjects", "count_records", "count...
$ estimate_type  <chr> "integer", "integer", "integer", "integer", "integer", "integer", "integer", "integer", "integer", "inte...
$ estimate_value <chr> "2487", "1606", "1342", "134", "652", "574", "102", "102", "133", "79", "138", "138", "908", ...
$ additional_name <chr> "source_concept_id &&& source_concept_name &&& standard_vocabulary &&& source_vocabulary", "s...
```

tableConceptIdCounts()

- Filter using the `display` argument:

<code>display</code>	What is shown
<code>"overall"</code> (default)	All records
<code>"standard"</code>	Standard concepts only
<code>"source"</code>	Source codes only
<code>"missing standard"</code>	Source codes with no standard mapping
<code>"missing source"</code>	Standard concepts with no source code

tableConceptIdCounts()

```
tableConceptIdCounts(result = concept_result, display = "standard", type = "reactable")
```

Search						
Database	OMOP table	Standard concept name	Standard concept id	Standard vocabulary	N records	N subjects
Synthea	procedure_occurrence	Allergy screening test	4191853	Snomed	704	320
Synthea	procedure_occurrence	Appendectomy	4198190	Snomed	157	157
Synthea	procedure_occurrence	Asthma screening	4062501	Snomed	101	101

ce						
Synthea	procedure _occurrence	Bilateral tubal ligation	4117038	Snomed	12	12
Synthea	procedure _occurrence	Bone density scan	4195803	Snomed	655	491
Synthea	procedure _occurrence	Bone immobiliz ation	4170947	Snomed	1789	1278
Synthea	procedure _occurrence	Brief general examinati on	4037675	Snomed	102	102
Synthea	procedure _occurrence	Catheter ablation of	4057420	Snomed	438	236

← Back to Presentations

		tissue of heart				
Synthea	procedure _occurrence	Cholecystectomy	4242997	Snomed	3	3
Synthea	procedure _occurrence	Cognitive and behavioral therapy	4043071	Snomed	2820	146
1–10 of 51 rows		Show	10	Previous 1 2 3 4 5 6 Next		

tableTopConceptCounts()

```
tableTopConceptCounts(result = concept_result, countBy = "record", top = 10, type = "gt")
```

Top 10 concepts in procedure_occurrence table ranked by record count

Top	Data source
	Synthea
procedure_occurrence	
1	Standard: Subcutaneous immunotherapy (4107731 - SNOMED) Source: Subcutaneous immunotherapy (4107731 - SNOMED) 17520
2	Standard: Cognitive and behavioral therapy (4043071 - SNOMED) Source: Cognitive and behavioral therapy (4043071 - SNOMED) 2820
3	Standard: Suture open wound (4125906 - SNOMED) Source: Suture open wound (4125906 - SNOMED) 2487
4	Standard: Bone immobilization (4170947 - SNOMED) Source: Bone immobilization (4170947 - SNOMED) 1789
5	Standard: Sputum examination (4151422 - SNOMED) Source: Sputum examination (4151422 - SNOMED) 1659
6	Standard: Direct current cardioversion (4078793 - SNOMED) Source: Direct current cardioversion (4078793 - SNOMED) 1342
7	Standard: Pulmonary rehabilitation (4035793 - SNOMED) Source: Pulmonary rehabilitation (4035793 - SNOMED) 908
8	Standard: Intramuscular injection (4295880 - SNOMED) Source: Intramuscular injection (4295880 - SNOMED) 908
9	Standard: Allergy screening test (4191853 - SNOMED) Source: Allergy screening test (4191853 - SNOMED)

Top	Data source
	Synthea
	704
10	Standard: Bone density scan (4195803 - SNOMED) Source: Bone density scan (4195803 - SNOMED) 655

[← Back to Presentations](#)

6. Full database characterisation

databaseCharacteristics()

- `databaseCharacteristics()` runs **all summaries in one call**.

```
full_result <- databaseCharacteristics(  
  cdm                = cdm,  
  omopTableName      = c("drug_exposure", "condition_occurrence"),  
  sex                = TRUE,  
  ageGroup           = list(c(0, 50), c(51, Inf)),  
  interval           = "years",  
  dateRange          = as.Date(c("2010-01-01", "2020-12-31")),  
  conceptIdCounts    = FALSE  
)
```

shinyCharacteristics()

- Then launch a **Shiny app** to explore everything interactively:

```
shinyCharacteristics(result = full_result, directory = here())
```

Live Shiny demo

 Database characterisation

 Background

 Snapshot

 Population Characteristics

 Person Table Summary

 Observation Period

 Clinical Tables

 Concept Counts

OmopSketch Database Characterisation

Shiny App Overview

This Shiny App presents the results of a database characterisation performed using the **OmopSketch** R package. It provides summaries and visualisations to support exploration of OMOP CDM-compliant datasets.

The app includes the following components:

- **Snapshot:** Metadata extracted from the `cdm_source` table, using the output of `summariseOmopSnapshot()`.
- **Population Characteristics:** Summary of the demographics of the population in observation, generated using **CohortConstructor** and **CohortCharacteristics**.
- **Person:** Summary of person table, from `summarisePerson()`.
- **Observation Period:** Distribution and length of observation periods, based on `summariseObservationPeriod()`.
- **Trends:** Temporal trends of individuals and records in observation, including changes in median age, proportion of females, and number of person-days, generated from `summariseTrend()`.
- **Clinical Records:** Summary of clinical tables focused on vocabulary usage and quality checks, from `summariseClinicalRecords()`.
- **Concept Counts** (*optional*): Counts of `concept_id` s across tables, generated by `summariseConceptIdCounts()`.

Shiny generation

[↗ Open full app](#)

Resources

👉 Package website

👉 CRAN

👉 PDF manual

✉ cecilia.campanile@ndorms.ox.ac.uk

Thank you!

[← Back to Presentations](#)



PatientProfiles

Adding patient-level characteristics to OMOP CDM tables

2026-09-17

What you will be able to do

- Add patient-level characteristics to cohort and CDM tables.
- Check whether records are inside observation and quantify lookback/follow-up time.
- Create features from intersections with cohorts, concepts, and OMOP CDM tables.

Today's workflow

1. Connect to a synthetic OMOP CDM database.
2. Create two small cohorts for live examples.
3. Add demographics and observation-period features.
4. Build intersection features.



Settings

PatientProfiles starts with a table that has a person identifier and, usually, an index date.

It returns the same table with extra columns.



Tip

For cohort tables, the default index date is **cohort_start_date**. For CDM clinical tables, set the relevant date column explicitly.

Setup

Load packages and create the CDM object

- Load the required packages

```
library(PatientProfiles)
library(CDMConnector)
library(CohortConstructor)
library(duckdb)
library(dplyr)
library(here)
library(omock)
library(DBI)
```

- Create a CDM object from the **GiBleed** synthetic dataset

```
con <- dbConnect(drv = duckdb(dbdir = here("Databases", "GiBleed.duckdb")))
```

```
cdm <- cdmFromCon(
  con = con,
  cdmSchema = "main",
  writeSchema = "results",
  writePrefix = "pp_"
)
```

Create a cohort

Using **CohortConstructor** we will create:

- **my_cohort**: people with arthritis-related records.
- **medications**: people with heparin records.

```
cdm$my_cohort <- conceptCohort(  
  cdm = cdm,  
  conceptSet = list(arthritis = c(80180, 80809)),  
  name = "my_cohort"  
)  
  
cdm$medications <- conceptCohort(  
  cdm = cdm,  
  conceptSet = list(heparin = 1367571),  
  name = "medications"  
)
```

Add demographics

addSex()

Use `addSex()` when the table contains `person_id` or `subject_id` and you want the person's sex as a new column:

```
cdm$my_cohort |>
  addSex(
    sexName = "sex", # name of the new column (default = "sex")
    missingSexValue = "None" # label for missing gender_concept_id (default = "None")
  )
```

A query: ?? x 5

Database: DuckDB 1.5.5 [unknown@Linux 6.17.0-1022-azure:R 4.6.1//home/runner/work/OHDSI-UK-2026-Training/OHDSI-UK-2026-Training/Databases/GiBleed.duckdb]

	cohort_definition_id	subject_id	cohort_start_date	cohort_end_date	sex
	<int>	<int>	<date>	<date>	<chr>
1	1	6	2005-07-13	2005-07-13	Female
2	1	123	1986-03-24	1986-03-24	Male
3	1	129	2014-03-21	2014-03-21	Male
4	1	16	2013-03-21	2013-03-21	Female
5	1	65	2009-06-03	2009-06-03	Female
6	1	74	2015-08-12	2015-08-12	Female
7	1	42	1950-05-16	1950-05-16	Female
8	1	187	1984-11-29	1984-11-29	Male
9	1	18	2009-03-21	2009-03-21	Female

addAge()

Use `addAge()` to calculate age at an index date. The table must contain `person_id` or `subject_id`:

```
cdm$my_cohort |>
  addAge(
    indexDate = "cohort_start_date", # date to compute age (default = "cohort_start_date")
    ageName = "age", # name of the age column (default = "age")
    ageMissingMonth = 1, # Month for individuals with missing month (default = 1)
    ageMissingDay = 1, # Day for individuals with missing day (default = 1)
    ageImposeMonth = FALSE, # Whether to impose default month to all individuals (default = FALSE)
    ageImposeDay = FALSE, # Whether to impose default day to all individuals (default = FALSE)
    ageUnit = "years" # 'years', 'months' or 'days'.
  )
```

A query: ?? x 5

Database: DuckDB 1.5.5 [unknown@Linux 6.17.0-1022-azure:R 4.6.1//home/runner/work/OHDSI-UK-2026-Training/OHDSI-UK-2026-Training/Databases/GiBleed.duckdb]

	cohort_definition_id	subject_id	cohort_start_date	cohort_end_date	age
	<int>	<int>	<date>	<date>	<dbl>
1	1	318	1994-12-10	1994-12-10	41
2	1	850	1990-01-23	1990-01-23	42
3	1	1322	1958-09-01	1958-09-01	35
4	1	1418	2014-08-08	2014-08-08	41
5	1	2056	2016-02-26	2016-02-26	40
6	1	2180	2017-08-06	2017-08-06	38
7	1	2498	2007-11-18	2007-11-18	36
8	1	3566	1988-06-23	1988-06-23	36
9	1	3583	2002-11-12	2002-11-12	34

addAge()

Add age groups

```
cdm$drug_exposure |>
  addAge(
    indexDate = "drug_exposure_start_date",
    ageGroup = list(c(0, 39), c(40, Inf))
  ) |>
  select("person_id", "drug_exposure_start_date", "age", "age_group")
```

```
# A query: ?? x 4
# Database: DuckDB 1.5.5 [unknown@Linux 6.17.0-1022-azure:R 4.6.1//home/runner/work/OHDSI-UK-2026-Training/OHDSI-UK-2026-Training/Databases/GiBleed.duckdb]
  person_id drug_exposure_start_date   age age_group
    <int>   <date>                <dbl> <chr>
1      573 1960-04-09                0 0 to 39
2     1332 2010-10-06               67 40 or above
3      576 2017-10-25               51 40 or above
4     4550 2000-11-13               38 0 to 39
5     3895 1982-03-30               58 40 or above
6     3199 1975-08-19                2 0 to 39
7      476 2008-05-31              40 40 or above
8     1154 1971-05-16                0 0 to 39
9      186 1976-05-31                4 0 to 39
--      --
```

addAge()

You can create several labelled age-group columns in one call:

```
cdm$drug_exposure |>
  addAge(
    indexDate = "drug_exposure_start_date",
    ageGroup = list(
      "age_group" = list("<40" = c(0, 39), ">=40" = c(40, Inf)),
      "category" = list("kids" = c(0, 17), "adults" = c(18, Inf))
    )
  ) |>
  select("person_id", "drug_exposure_start_date", "age", "age_group", "category")
```

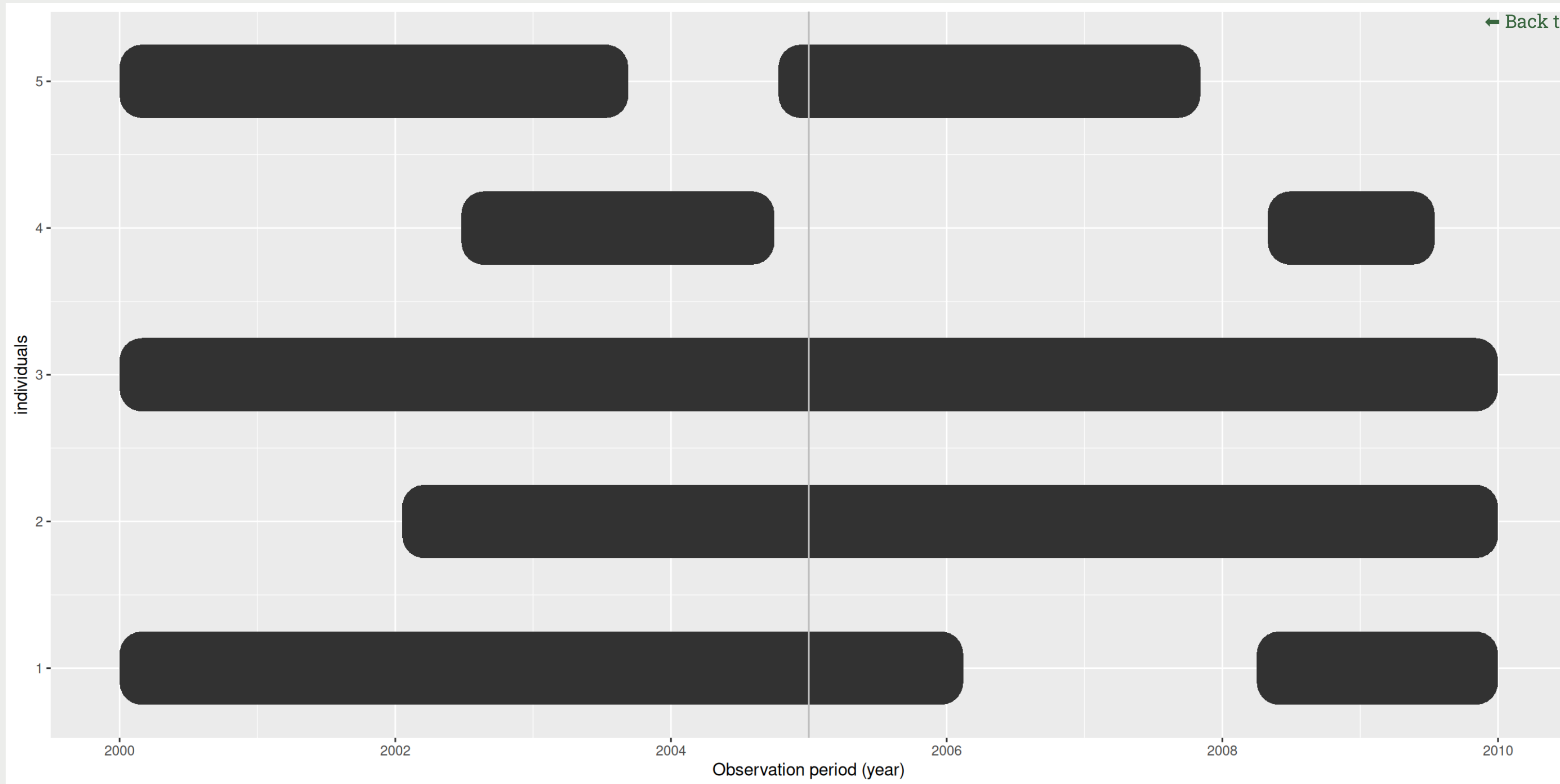
A query: ?? x 5

Database: DuckDB 1.5.5 [unknown@Linux 6.17.0-1022-azure:R 4.6.1//home/runner/work/OHDSI-UK-2026-Training/OHDSI-UK-2026-Training/Databases/GiBleed.duckdb]

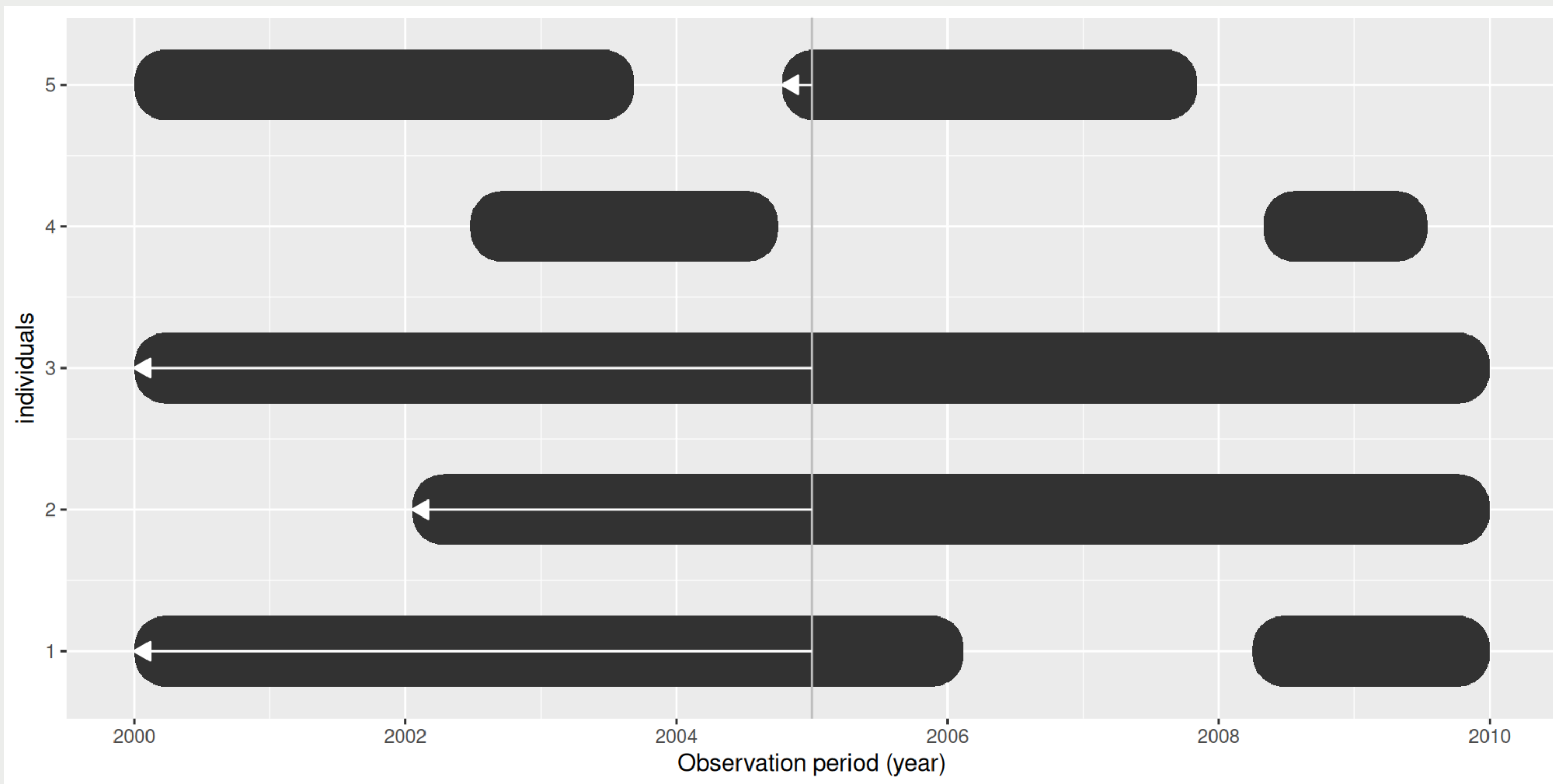
	person_id	drug_exposure_start_date	age	age_group	category
	<int>	<date>	<dbl>	<chr>	<chr>
1	573	1960-04-09	0	<40	kids
2	1332	2010-10-06	67	>=40	adults
3	576	2017-10-25	51	>=40	adults
4	4550	2000-11-13	38	<40	adults
5	3895	1982-03-30	58	>=40	adults
6	3199	1975-08-19	2	<40	kids
7	476	2008-05-31	40	>=40	adults
8	1154	1971-05-16	0	<40	kids
9	186	1976-05-31	4	<40	kids
...	...	...	...	...	...

addPriorObservation()

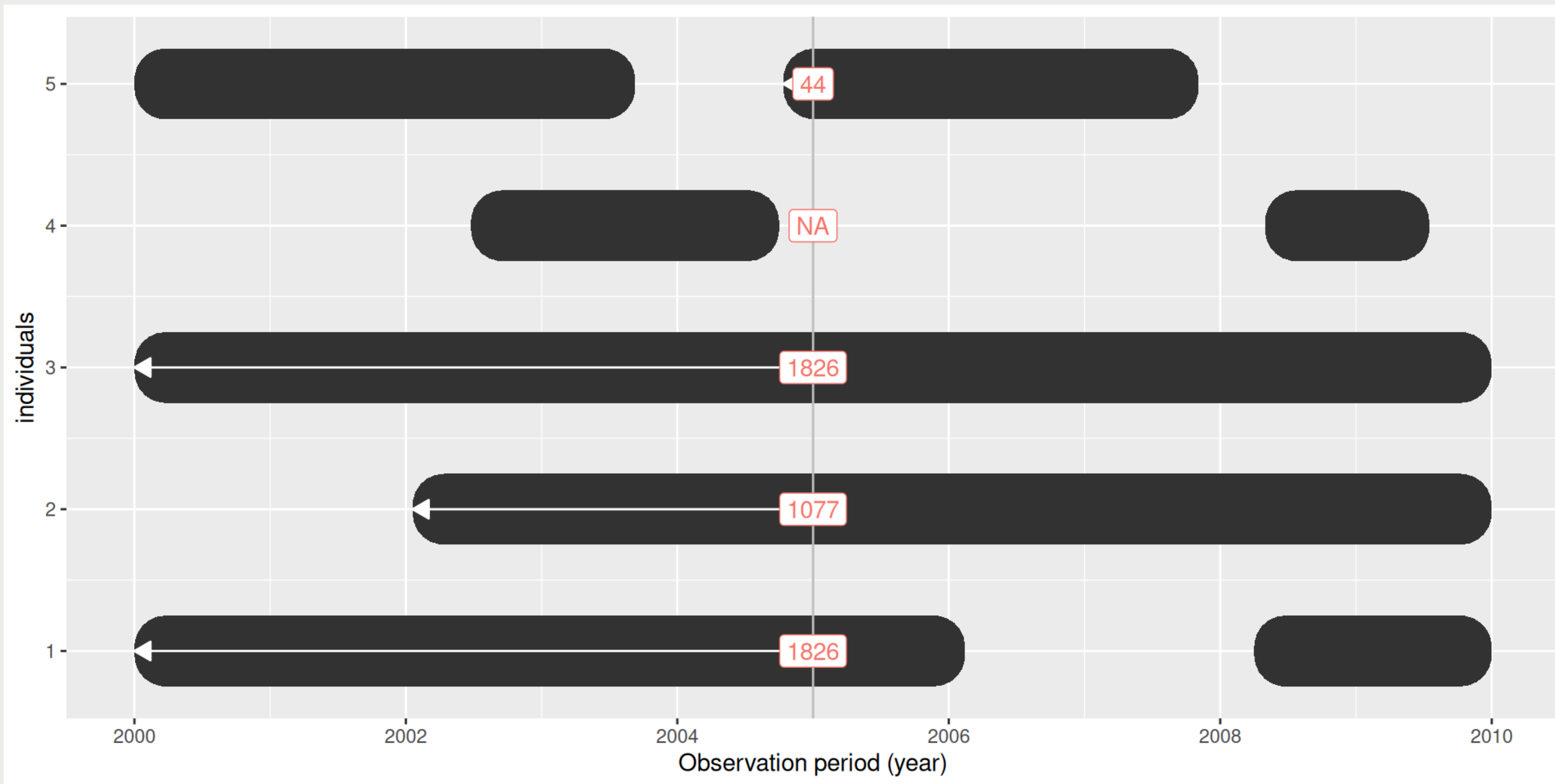
How much lookback time did each person have before the index date?



addPriorObservation()



addPriorObservation()



addPriorObservation()

The default output is the number of days from observation start to index date:

```
cdm$my_cohort |>  
  addPriorObservation()
```

```
# A query:  ?? x 5
```

```
# Database: DuckDB 1.5.5 [unknown@Linux 6.17.0-1022-azure:R 4.6.1//home/runner/work/OHDSI-UK-2026-Training/OHDSI-UK-2026-Training/Databases/GiBleed.duckdb]
```

	cohort_definition_id	subject_id	cohort_start_date	cohort_end_date	prior_observation
	<int>	<int>	<date>	<date>	<dbl>
1	1	318	1994-12-10	1994-12-10	15194
2	1	850	1990-01-23	1990-01-23	15680
3	1	1322	1958-09-01	1958-09-01	12920
4	1	1418	2014-08-08	2014-08-08	15320
5	1	2056	2016-02-26	2016-02-26	14624
6	1	2180	2017-08-06	2017-08-06	14190
7	1	2498	2007-11-18	2007-11-18	13280
8	1	3566	1988-06-23	1988-06-23	13485
9	1	3583	2002-11-12	2002-11-12	12425

addPriorObservation()

You can return the observation-period start date instead of a duration:

```
cdm$condition_occurrence |>
  addPriorObservation(
    indexDate = "condition_start_date",
    priorObservationName = "start_observation", # name of the column
    priorObservationType = "date" # default = "days"
  ) |>
  select("person_id", "condition_occurrence_id", "condition_concept_id", "condition_start_date", "start_observation")
```

A query: ?? x 5

Database: DuckDB 1.5.5 [unknown@Linux 6.17.0-1022-azure:R 4.6.1//home/runner/work/OHDSI-UK-2026-Training/OHDSI-UK-2026-Training/Databases/GiBleed.duckdb]

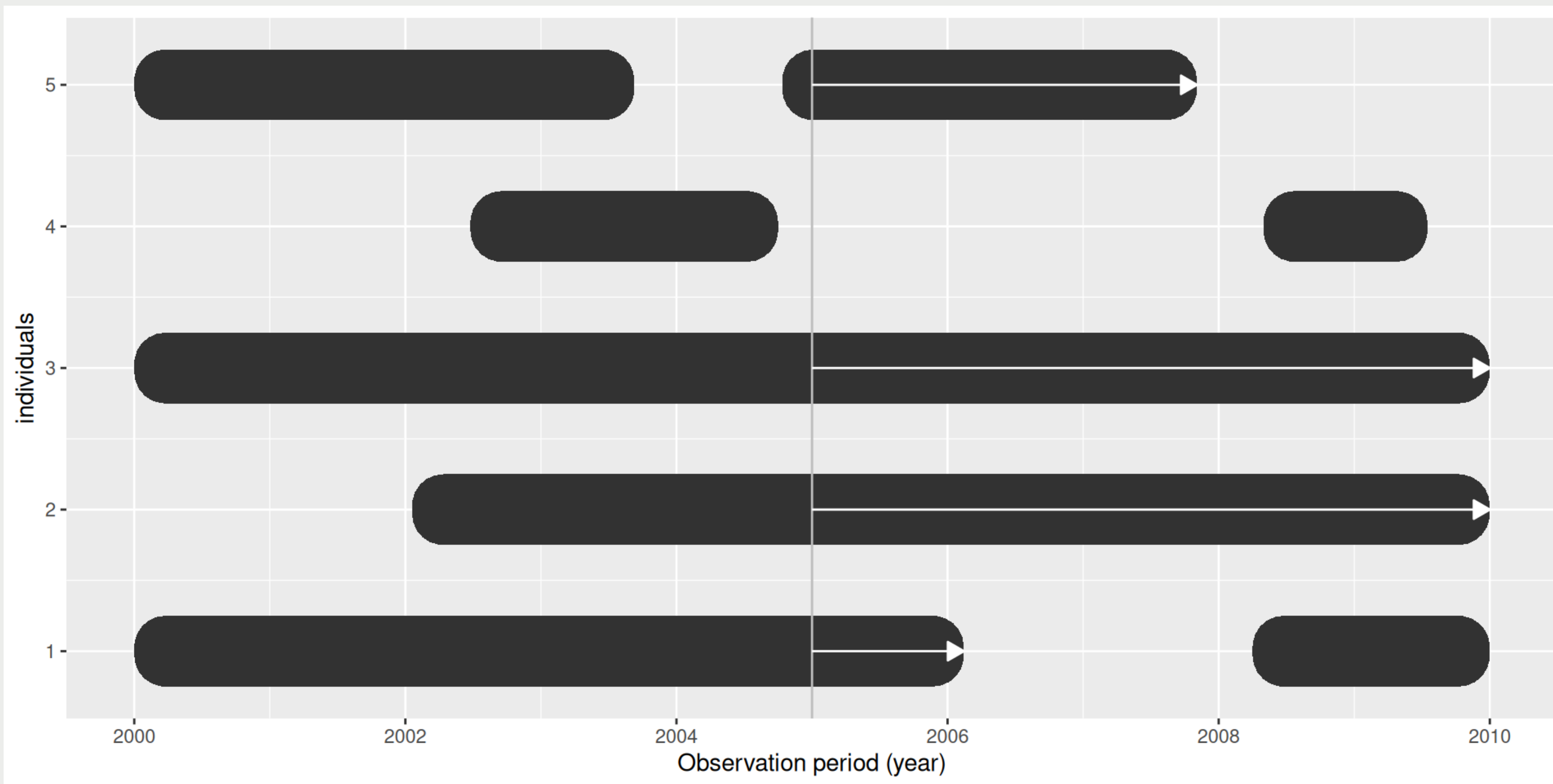
	person_id	condition_occurrence_id	condition_concept_id	condition_start_date	start_observation
	<int>	<int>	<int>	<date>	<date>
1	263	4483	4112343	2015-10-02	1954-06-20
2	273	4657	192671	2011-10-10	1975-09-14
3	283	4815	28060	1984-02-15	1980-09-01
4	293	4981	378001	2005-11-07	1964-10-13
5	304	5153	257012	1974-07-30	1956-05-07
6	312	5313	4134304	1991-05-14	1972-01-24
7	326	5513	28060	1979-09-23	1970-06-01
8	334	5655	40481087	1999-07-12	1935-06-06
9	341	5811	40481087	1990-09-14	1960-07-11

addFutureObservation()

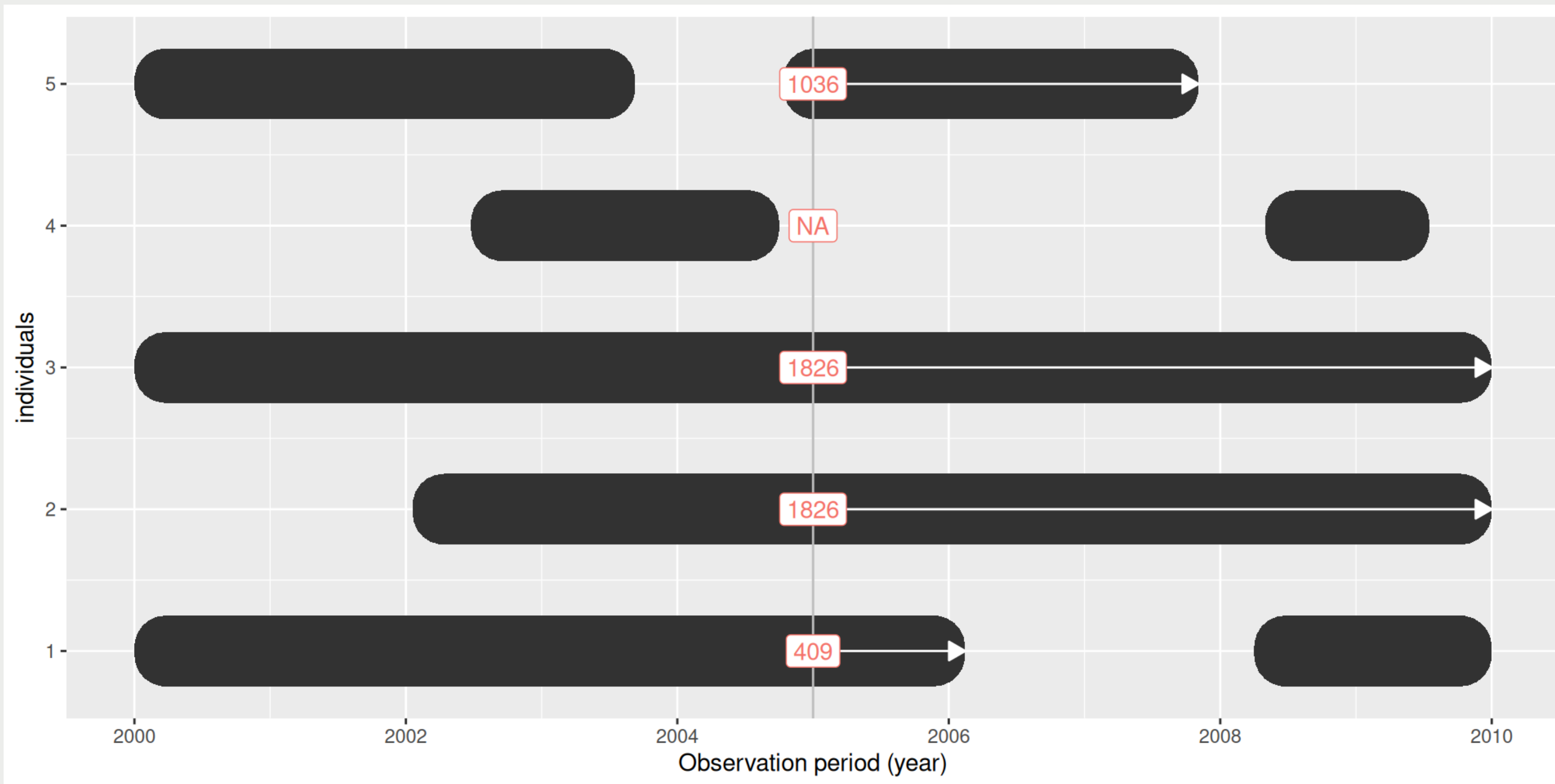
How much follow-up time did each person have after the index date? . . .



addFutureObservation()



addFutureObservation()



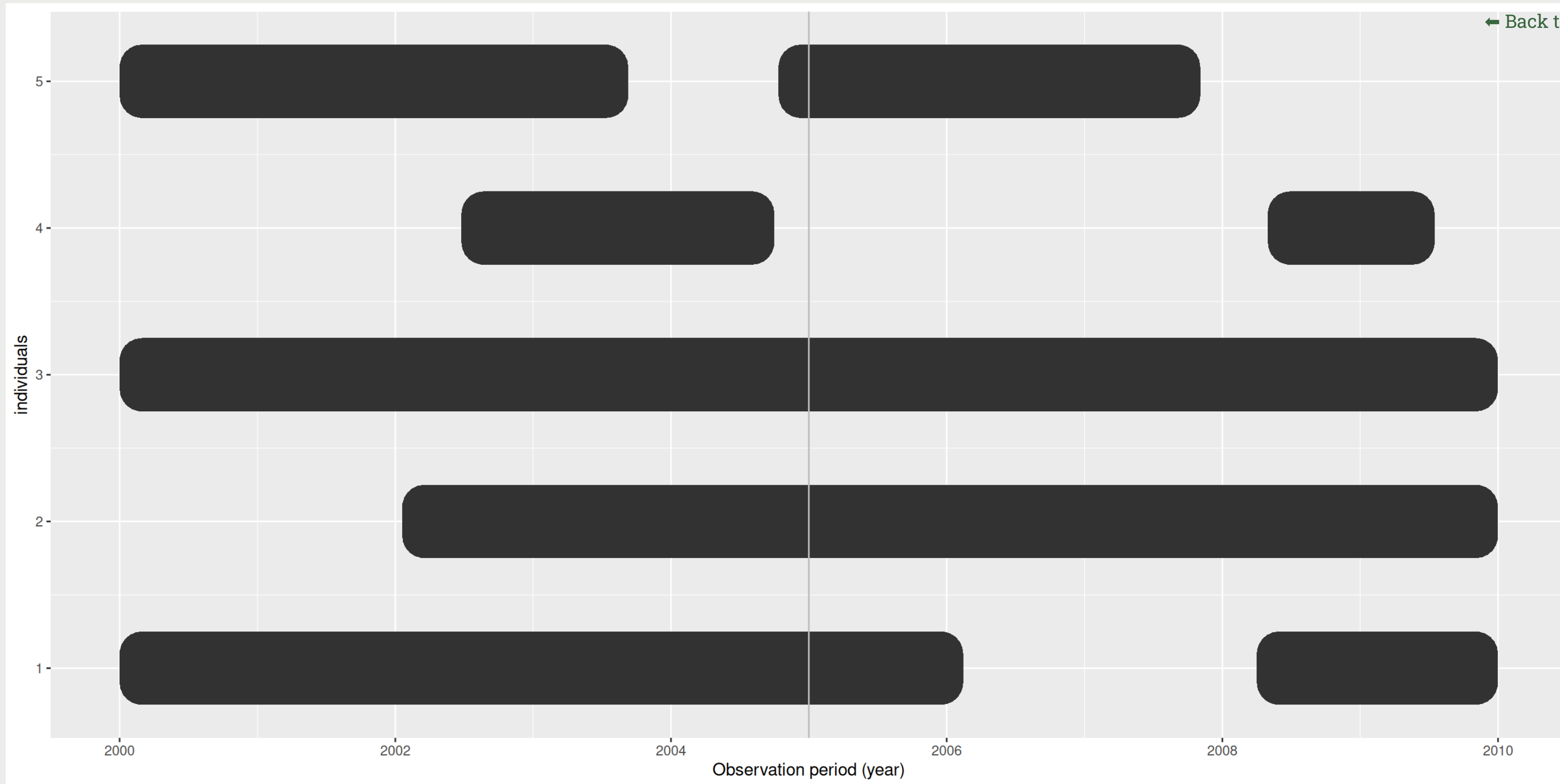
addFutureObservation()

```
cdm$my_cohort |>  
  addFutureObservation()
```

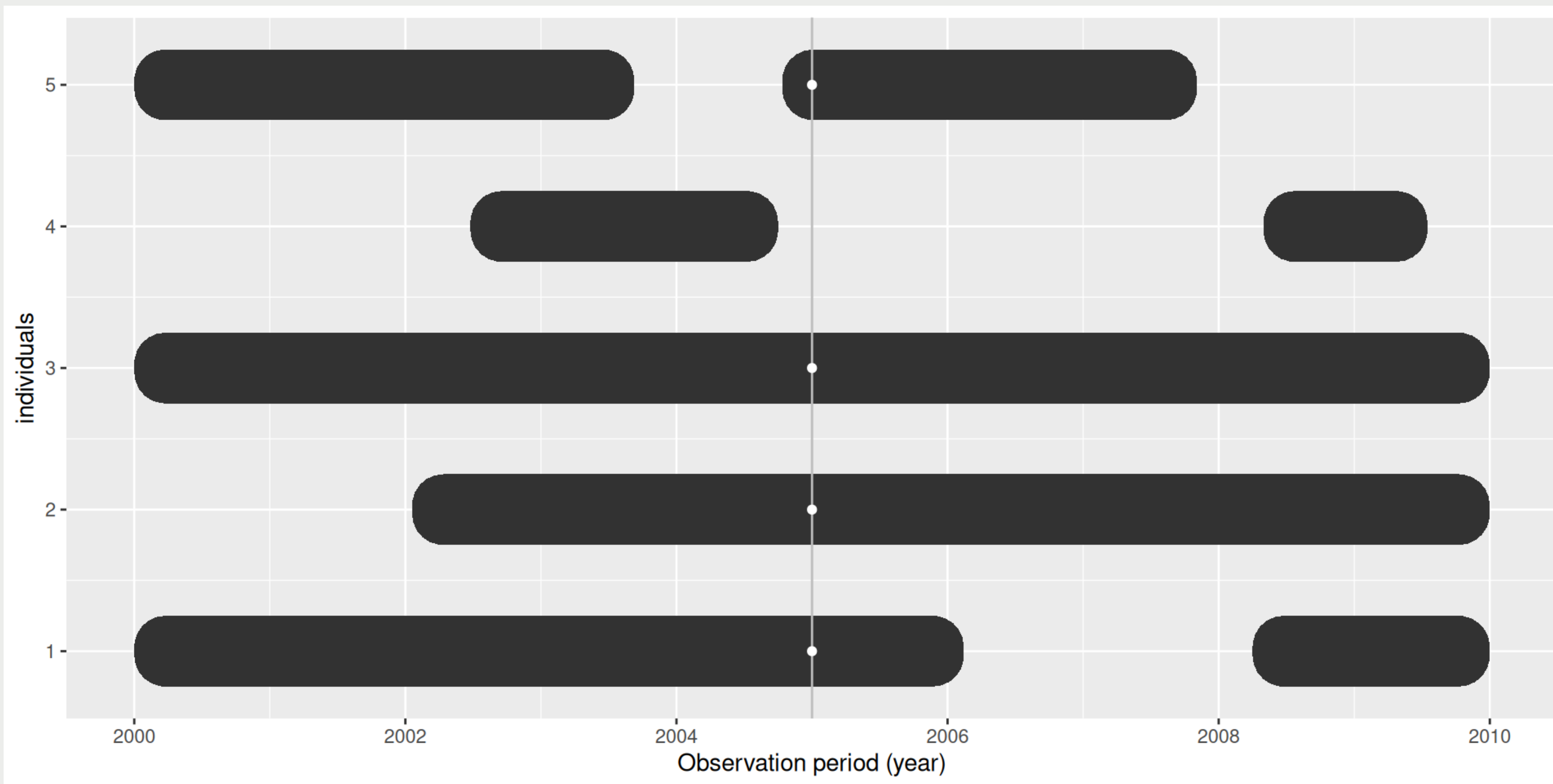
```
# A query:  ?? x 5  
# Database: DuckDB 1.5.5 [unknown@Linux 6.17.0-1022-azure:R 4.6.1//home/runner/work/OHDSI-UK-2026-Training/OHDSI-UK-  
2026-Training/Databases/GiBleed.duckdb]  
  cohort_definition_id subject_id cohort_start_date cohort_end_date future_observation  
                <int>      <int> <date>                <date>                <dbl>  
1                1        318 1994-12-10            1994-12-10                8697  
2                1        850 1990-01-23            1990-01-23               10710  
3                1       1322 1958-09-01            1958-09-01               21912  
4                1       1418 2014-08-08            2014-08-08                1613  
5                1       2056 2016-02-26            2016-02-26                1092  
6                1       2180 2017-08-06            2017-08-06                 426  
7                1       2498 2007-11-18            2007-11-18                3987  
8                1       3566 1988-06-23            1988-06-23               11036  
9                1       3583 2002-11-12            2002-11-12                6013
```

addInObservation()

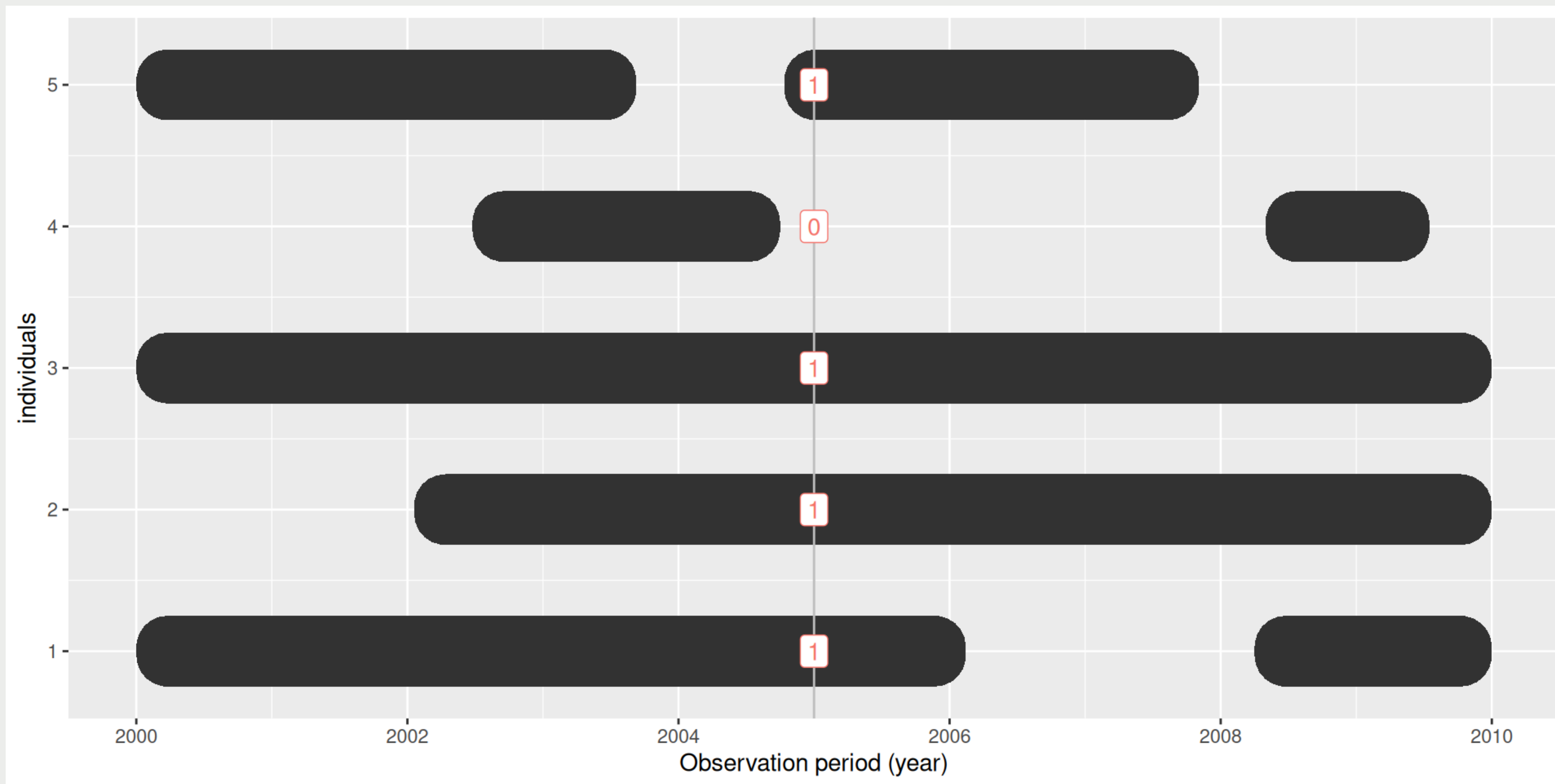
Is each index date inside an observation period?



addInObservation()



addInObservation()



addInObservation()

```
cdm$condition_occurrence |>
  addInObservation(indexDate = "condition_start_date") |>
  filter(in_observation == 1) |>
  select("condition_concept_id", "person_id", "condition_start_date", "in_observation")
```

```
# A query:  ?? x 4
# Database: DuckDB 1.5.5 [unknown@Linux 6.17.0-1022-azure:R 4.6.1//home/runner/work/OHDSI-UK-2026-Training/OHDSI-UK-2026-Training/Databases/GiBleed.duckdb]
  condition_concept_id person_id condition_start_date in_observation
            <int>      <int> <date>                <dbl>
1             4112343       263 2015-10-02              1
2              192671       273 2011-10-10              1
3               28060       283 1984-02-15              1
4              378001       293 2005-11-07              1
5              257012       304 1974-07-30              1
6             4134304       312 1991-05-14              1
7               28060       326 1979-09-23              1
8            40481087       334 1999-07-12              1
9            40481087       341 1990-09-14              1
--
```

addDateOfBirth()

Use `addDateOfBirth()` to add subjects' date of birth.

```
cdm$my_cohort |>
  addDateOfBirth()
```

```
# A query:  ?? x 5
# Database: DuckDB 1.5.5 [unknown@Linux 6.17.0-1022-azure:R 4.6.1//home/runner/work/OHDSI-UK-2026-Training/OHDSI-UK-2026-Training/Databases/GiBleed.duckdb]
  cohort_definition_id subject_id cohort_start_date cohort_end_date date_of_birth
              <int>      <int> <date>              <date>              <date>
1                    1         318 1994-12-10          1994-12-10          1953-05-04
2                    1         850 1990-01-23          1990-01-23          1947-02-18
3                    1        1322 1958-09-01          1958-09-01          1923-04-18
4                    1        1418 2014-08-08          2014-08-08          1972-08-28
5                    1        2056 2016-02-26          2016-02-26          1976-02-11
6                    1        2180 2017-08-06          2017-08-06          1978-09-30
7                    1        2498 2007-11-18          2007-11-18          1971-07-10
8                    1        3566 1988-06-23          1988-06-23          1951-07-23
9                    1        3583 2002-11-12          2002-11-12          1968-11-05
--                    -        - - - - - - - - - - - - - - - - - - - - - - -
```

addBirthday()

Use `addBirthday()` to add specific birthdays of individuals, accounting for leap years and correcting the invalid dates to the next valid date

```
cdm$my_cohort |>
  addDateOfBirth() |>
  addBirthday(
    birthday = 5, # default is 0
    birthdayName = "5th_birthday"
  )
```

A query: ?? x 6

Database: DuckDB 1.5.5 [unknown@Linux 6.17.0-1022-azure:R 4.6.1//home/runner/work/OHDSI-UK-2026-Training/OHDSI-UK-2026-Training/Databases/GiBleed.duckdb]

	cohort_definition_id	subject_id	cohort_start_date	cohort_end_date	date_of_birth	`5th_birthday`
	<int>	<int>	<date>	<date>	<date>	<date>
1	1	318	1994-12-10	1994-12-10	1953-05-04	1958-05-04
2	1	850	1990-01-23	1990-01-23	1947-02-18	1952-02-18
3	1	1322	1958-09-01	1958-09-01	1923-04-18	1928-04-18
4	1	1418	2014-08-08	2014-08-08	1972-08-28	1977-08-28
5	1	2056	2016-02-26	2016-02-26	1976-02-11	1981-02-11
6	1	2180	2017-08-06	2017-08-06	1978-09-30	1983-09-30
7	1	2498	2007-11-18	2007-11-18	1971-07-10	1976-07-10
8	1	3566	1988-06-23	1988-06-23	1951-07-23	1956-07-23
9	1	3583	2002-11-12	2002-11-12	1968-11-05	1973-11-05

addDemographics()

`addDemographics()` is a convenience wrapper for common patient-level features.

```
cdm$my_cohort |>
  addDemographics()
```

A query: ?? x 8

```
# Database: DuckDB 1.5.5 [unknown@Linux 6.17.0-1022-azure:R 4.6.1//home/runner/work/OHDSI-UK-2026-Training/OHDSI-UK-2026-Training/Databases/GiBleed.duckdb]
```

	cohort_definition_id	subject_id	cohort_start_date	cohort_end_date	age	sex	prior_observation
	<int>	<int>	<date>	<date>	<dbl>	<chr>	<dbl>
1	1	6	2005-07-13	2005-07-13	41	Female	15170
2	1	123	1986-03-24	1986-03-24	35	Male	13130
3	1	129	2014-03-21	2014-03-21	39	Male	14410
4	1	16	2013-03-21	2013-03-21	41	Female	15134
5	1	65	2009-06-03	2009-06-03	42	Female	15405
6	1	74	2015-08-12	2015-08-12	43	Female	15925
7	1	42	1950-05-16	1950-05-16	40	Female	14804
8	1	187	1984-11-29	1984-11-29	39	Male	14374
9	1	18	2009-03-21	2009-03-21	43	Female	15830

addDemographics()

```
cdm$my_cohort |>
  addDemographics(
    age = TRUE,
    ageGroup = list("kids" = c(0, 17), "adults" = c(18, Inf)),
    sex = FALSE,
    priorObservation = TRUE,
    priorObservationName = "observation_start",
    priorObservationType = "date",
    futureObservation = TRUE,
    futureObservationName = "observation_end",
    futureObservationType = "date"
  )
```

A query: ?? x 8

Database: DuckDB 1.5.5 [unknown@Linux 6.17.0-1022-azure:R 4.6.1//home/runner/work/OHDSI-UK-2026-Training/OHDSI-UK-2026-Training/Databases/GiBleed.duckdb]

	cohort_definition_id	subject_id	cohort_start_date	cohort_end_date	age	age_group	observation_start
	<int>	<int>	<date>	<date>	<dbl>	<chr>	<date>
1	1	318	1994-12-10	1994-12-10	41	adults	1953-05-05
2	1	850	1990-01-23	1990-01-23	42	adults	1947-02-18
3	1	1322	1958-09-01	1958-09-01	35	adults	1923-04-18
4	1	1418	2014-08-08	2014-08-08	41	adults	1972-08-28
5	1	2056	2016-02-26	2016-02-26	40	adults	1976-02-12
6	1	2180	2017-08-06	2017-08-06	38	adults	1978-09-30
7	1	2498	2007-11-18	2007-11-18	36	adults	1971-07-10
8	1	3566	1988-06-23	1988-06-23	36	adults	1951-07-23
9	1	3583	2002-11-12	2002-11-12	34	adults	1968-11-05

Your Turn

Starting from `cdm$my_cohort`, create a table with:

- sex
- age groups `<18`, `18-64`, and `65+`
- prior observation in days
- future observation in days

Your Turn

```
cdm$my_cohort |>
  addDemographics(
    age = FALSE,
    ageGroup = list(
      "<18" = c(0, 17),
      "18-64" = c(18, 64),
      "65+" = c(65, Inf)
    ),
    priorObservation = TRUE,
    priorObservationType = "days",
    futureObservation = TRUE,
    futureObservationType = "days"
  )
```

```
# A query: ?? x 8
# Database: DuckDB 1.5.5 [unknown@Linux 6.17.0-1022-azure:R 4.6.1//home/runner/work/OHDSI-UK-2026-Training/OHDSI-UK-2026-Training/Databases/GiBleed.duckdb]
```

	cohort_definition_id	subject_id	cohort_start_date	cohort_end_date	age_group	sex	prior_observation
	<int>	<int>	<date>	<date>	<chr>	<chr>	<dbl>
1	1	6	2005-07-13	2005-07-13	18-64	Female	15170
2	1	123	1986-03-24	1986-03-24	18-64	Male	13130
3	1	129	2014-03-21	2014-03-21	18-64	Male	14410
4	1	16	2013-03-21	2013-03-21	18-64	Female	15134
5	1	65	2009-06-03	2009-06-03	18-64	Female	15405
6	1	74	2015-08-12	2015-08-12	18-64	Female	15925
7	1	42	1950-05-16	1950-05-16	18-64	Female	14804
8	1	187	1984-11-29	1984-11-29	18-64	Male	14374
9	1	18	2009-03-21	2009-03-21	18-64	Female	15830
...	...	...	...	...	...	...	...

Choosing a characteristic helper

Need	Helper
Sex	<code>addSex()</code>
Age or age groups	<code>addAge()</code>
Lookback time	<code>addPriorObservation()</code>
Follow-up time	<code>addFutureObservation()</code>
Date inside observation	<code>addInObservation()</code>
Common demographics together	<code>addDemographics()</code>

Add intersections

The intersection functions

- Did the person have an event?
- How many events did they have?
- When was the first or last event?
- How many days until or since the event?

15 functions

- addCohortIntersectFlag
- addCohortIntersectCount
- addCohortIntersectDays
- addCohortIntersectDate
- addCohortIntersectField
- addConceptIntersectFlag
- addConceptIntersectCount
- addConceptIntersectDays
- addConceptIntersectDate
- addConceptIntersectField
- addTableIntersectFlag
- addTableIntersectCount
- addTableIntersectDays
- addTableIntersectDate
- addTableIntersectField

How to choose

Source of target records	Flag
Cohort table	<code>addCohortIntersectFlag()</code>
Concept set	<code>addConceptIntersectFlag()</code>
OMOP table	<code>addTableIntersectFlag()</code>
Source of target records	Count
Cohort table	<code>addCohortIntersectCount()</code>
Concept set	<code>addConceptIntersectCount()</code>
OMOP table	<code>addTableIntersectCount()</code>

How to choose

Source of target records	Date
Cohort table	<code>addCohortIntersectDate()</code>
Concept set	<code>addConceptIntersectDate()</code>
OMOP table	<code>addTableIntersectDate()</code>
Source of target records	Days
Cohort table	<code>addCohortIntersectDays()</code>
Concept set	<code>addConceptIntersectDays()</code>
OMOP table	<code>addTableIntersectDays()</code>

How to choose

Source of target records	Field
Cohort table	<code>addCohortIntersectField()</code>
Concept set	<code>addConceptIntersectField()</code>
OMOP table	<code>addTableIntersectField()</code>

addCohortIntersectFlag

Did the person enter another cohort in the next 180 days or 181-365 days?

```
cdm$my_cohort |>
  addCohortIntersectFlag(
    targetCohortTable = "medications",
    window = list(c(1, 180), c(181, 365))
  ) |> glimpse()
```

Rows: ??

Columns: 6

```
$ cohort_definition_id <int> 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1...
$ subject_id          <int> 318, 850, 1322, 1418, 2056, 2180, 2498, 3566, 3583, 4276, 4474, 3, 370, 974, 23...
$ cohort_start_date   <date> 1994-12-10, 1990-01-23, 1958-09-01, 2014-08-08, 2016-02-26, 2017-08-06, 2007-1...
$ cohort_end_date     <date> 1994-12-10, 1990-01-23, 1958-09-01, 2014-08-08, 2016-02-26, 2017-08-06, 2007-1...
$ heparin_1_to_180    <dbl> 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0...
$ heparin_181_to_365  <dbl> 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0...
```

addTableIntersectCount

How many drug exposure records did the person have in the year after cohort entry?

```
cdm$my_cohort |>
  addTableIntersectCount(
    tableName = "drug_exposure",
    window = c(0, 365),
    nameStyle = "number_prescriptions",
    name = "my_cohort2" #Name of the new table, if NULL a temporary table is returned.
  )
```

A query: ?? x 5

Database: DuckDB 1.5.5 [unknown@Linux 6.17.0-1022-azure:R 4.6.1//home/runner/work/OHDSI-UK-2026-Training/OHDSI-UK-2026-Training/Databases/GiBleed.duckdb]

	cohort_definition_id	subject_id	cohort_start_date	cohort_end_date	number_prescriptions
	<int>	<int>	<date>	<date>	<dbl>
1	1	318	1994-12-10	1994-12-10	2
2	1	850	1990-01-23	1990-01-23	1
3	1	1322	1958-09-01	1958-09-01	1
4	1	1418	2014-08-08	2014-08-08	3
5	1	2056	2016-02-26	2016-02-26	1
6	1	2180	2017-08-06	2017-08-06	1
7	1	2498	2007-11-18	2007-11-18	1
8	1	3566	1988-06-23	1988-06-23	1
9	1	3583	2002-11-12	2002-11-12	2

addCohortIntersectDate

When did the person first enter the medication cohort after cohort entry?

```
cdm$my_cohort |>
  addCohortIntersectDate(
    targetCohortTable = "medications",
    targetCohortId = "heparin",
    window = c(0, Inf),
    order = "first", # date to use if there are multiple records for an individual during the window of interest
    nameStyle = "first_{cohort_name}"
  )
```

A query: ?? x 5

Database: DuckDB 1.5.5 [unknown@Linux 6.17.0-1022-azure:R 4.6.1//home/runner/work/OHDSI-UK-2026-Training/OHDSI-UK-2026-Training/Databases/GiBleed.duckdb]

	cohort_definition_id	subject_id	cohort_start_date	cohort_end_date	first_heparin
	<int>	<int>	<date>	<date>	<date>
1	1	318	1994-12-10	1994-12-10	NA
2	1	850	1990-01-23	1990-01-23	NA
3	1	1322	1958-09-01	1958-09-01	NA
4	1	1418	2014-08-08	2014-08-08	NA
5	1	2056	2016-02-26	2016-02-26	NA
6	1	2180	2017-08-06	2017-08-06	NA
7	1	2498	2007-11-18	2007-11-18	NA
8	1	3566	1988-06-23	1988-06-23	NA
9	1	3583	2002-11-12	2002-11-12	NA
...	...	...	...	...	...

addConceptIntersectDays

How many days until the next record for a concept set?

```
cdm$my_cohort |>
  addConceptIntersectDays(
    conceptSet = list(tramadol = 1103314),
    window = c(0, 365),
    inObservation = FALSE, # If TRUE only records inside an observation period will be considered.
    nameStyle = "next_{concept_name}"
  ) |>
  glimpse()
```

Rows: ??

Columns: 5

```
$ cohort_definition_id <int> 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1...
$ subject_id          <int> 318, 850, 1322, 1418, 2056, 2180, 2498, 3566, 3583, 4276, 4474, 3, 370, 974, 23...
$ cohort_start_date   <date> 1994-12-10, 1990-01-23, 1958-09-01, 2014-08-08, 2016-02-26, 2017-08-06, 2007-1...
$ cohort_end_date     <date> 1994-12-10, 1990-01-23, 1958-09-01, 2014-08-08, 2016-02-26, 2017-08-06, 2007-1...
$ next_tramadol       <dbl> NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA,...
```

addConceptIntersectField

What is the last value of body weight recorded before index day?

```
cdm$my_cohort |>
  addConceptIntersectField(
    conceptSet = list(body_weight = 3012888),
    field = "value_as_number",
    window = c(-Inf, 0),
    order = "last",
    nameStyle = "body_weight_value"
  ) |>
  glimpse()
```

Rows: ??

Columns: 5

```
$ cohort_definition_id <int> 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1...
$ subject_id          <int> 318, 850, 1322, 1418, 2056, 2180, 2498, 3566, 3583, 4276, 4474, 3, 370, 974, 23...
$ cohort_start_date   <date> 1994-12-10, 1990-01-23, 1958-09-01, 2014-08-08, 2016-02-26, 2017-08-06, 2007-1...
$ cohort_end_date     <date> 1994-12-10, 1990-01-23, 1958-09-01, 2014-08-08, 2016-02-26, 2017-08-06, 2007-1...
$ body_weight_value   <chr> NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA,...
```

Your Turn

- Did the person have an heparin cohort entry in the year before index?
- How many condition occurrence records did they have in the year before index?

Your Turn

```
cdm$my_cohort |>
  addCohortIntersectFlag(
    targetCohortTable = "medications",
    targetCohortId = "heparin",
    window = list("prior_year" = c(-365, -1)),
    nameStyle = "prior_{cohort_name}"
  )

cdm$my_cohort |>
  addTableIntersectCount(
    tableName = "condition_occurrence",
    window = c(-365, -1),
    nameStyle = "prior_conditions"
  )
```

A query: ?? x 6

Database: DuckDB 1.5.5 [unknown@Linux 6.17.0-1022-azure:R 4.6.1//home/runner/work/OHDSI-UK-2026-Training/OHDSI-UK-2026-Training/Databases/GiBleed.duckdb]

	cohort_definition_id	subject_id	cohort_start_date	cohort_end_date	prior_heparin	prior_conditions
	<int>	<int>	<date>	<date>	<dbl>	<dbl>
1	1	850	1990-01-23	1990-01-23	0	1
2	1	1418	2014-08-08	2014-08-08	0	2
3	1	4474	1998-12-15	1998-12-15	0	1
4	1	3	1957-12-08	1957-12-08	0	2
5	1	370	2000-01-08	2000-01-08	0	1
6	1	2389	2008-08-09	2008-08-09	0	2
7	1	4811	1999-10-25	1999-10-25	0	1
8	1	419	1954-08-08	1954-08-08	0	2
9	1	3114	1983-02-21	1983-02-21	0	1

More intersection functions in PatientProfiles

- `addDeathDate`
- `addDeathFlag`
- `addDeathDays`

Resources

 [Package website](#)

 [CRAN](#)

 [PDF manual](#)

 cecilia.campanile@ndorms.ox.ac.uk

CohortConstructor

Build and Curate Study Cohorts in R Using the OMOP Common Data Model

2026-09-17

Introduction

- CohortConstructor package is designed to support cohort building pipelines in R, using data mapped to the OMOP Common Data Model.
- The code is publicly available in OHDSI's GitHub repository **CohortConstructor**.
- CohortConstructor v0.6.3 is available in **CRAN**.
- Vignettes with further information can be found in the package **website**.

Set up

OMOP Cohorts in R

OMOP Cohorts in R

- The `<cohort_table>` class is defined in the R package **omopgenerics**.
- This is the class that **CohortConstructor** uses, as well as other OMOP analytical packages.
- As defined in **omopgenerics**, a `<cohort_table>` must have at least the following 4 columns (without any missing values in them):
 - **cohort_definition_id**: Unique identifier for each cohort in the table.
 - **subject_id**: Unique patient identifier.
 - **cohort_start_date**: Date when the person enters the cohort.
 - **cohort_end_date**: Date when the person exits the cohort.

OMOP Cohorts in R

```
cdm$cohort
```

```
# A query:  ?? x 4
# Database: DuckDB 1.5.5 [unknown@Linux 6.17.0-1022-azure:R 4.6.1//tmp/Rtmpu0okBt/file202c7a4874c0.duckdb]
  cohort_definition_id subject_id cohort_start_date cohort_end_date
                <int>      <int> <date>              <date>
1                   2        2343 1996-11-09          1996-11-23
2                   2        2507 1995-02-14          1995-03-07
3                   2        2849 1966-06-17          1966-07-08
4                   2        2883 1966-07-15          1966-07-29
5                   2        2906 1986-04-15          1986-04-22
6                   2        3191 2008-01-03          2008-01-17
7                   2        3789 1945-09-16          1945-09-23
8                   2        3930 1927-02-28          1927-03-14
9                   1         648 1949-11-02          1949-11-10
10                  1       2792 2011-02-26          2011-03-11
... 
```

OMOP Cohorts in R

Additionally, the `<cohort_table>` object has the following **attributes**:

- **Settings**: Relate each cohort definition ID with a cohort name and other variables that define the cohort.

```
settings(cdm$cohort)
```

```
# A tibble: 2 × 4
  cohort_definition_id cohort_name      cdm_version vocabulary_version
      <int> <chr>                <chr>         <chr>
1           1 viral_pharyngitis 5.3          v5.0 18-JAN-19
2           2 viral_sinusitis  5.3          v5.0 18-JAN-19
```

OMOP Cohorts in R

- **Attrition:** Store information on each inclusion criteria applied and how many records and subjects were kept after.

```
attrition(cdm$cohort)
```

```
# A tibble: 8 × 7
  cohort_definition_id number_records number_subjects reason_id reason
excluded_subjects
<int>
1 1 10217 2606 1 Initial qualifying events 0
0
2 1 10217 2606 2 Record in observation 0
0
3 1 10217 2606 3 Not missing record date 0
0
4 1 10217 2606 4 Merge overlapping records 0
0
5 2 17268 2686 1 Initial qualifying events 0
-
```

OMOP Cohorts in R

- **Cohort count:** Number of records and subjects for each cohort.

```
cohortCount(cdm$cohort)
```

```
# A tibble: 2 × 3
```

	cohort_definition_id	number_records	number_subjects
	<int>	<int>	<int>
1	1	10217	2606
2	2	17268	2686

OMOP Cohorts in R

- **Cohort codelist:** Codelists used to define entry events and inclusion criteria for each cohort.

```
attr(cdm$cohort, "cohort_codelist")
```

```
# A query: ?? x 4
# Database: DuckDB 1.5.5 [unknown@Linux 6.17.0-1022-azure:R 4.6.1//tmp/Rtmpu0okBt/file202c7a4874c0.duckdb]
  cohort_definition_id codelist_name      concept_id codelist_type
      <int> <chr>              <int> <chr>
1             1 viral_pharyngitis    4112343 index event
2             2 viral_sinusitis      40481087 index event
```

CohortConstructor function Sets

Base cohorts *Cohort*

construction based on clinical concepts or demographics.

Requirements and Filtering

Demographic restrictions, event presence/absence conditions, and filtering specific records.

Update cohort entry and exit

Adjusting entry and exit dates to align with study periods, observation windows, or key events.

Transformation and

Combination *Merging, stratifying, collapsing, matching, or intersecting cohorts. (Not covered today)*

Base cohorts

Functions to build base cohorts

- `demographicsCohort()`
- `conceptCohort()`
- `measurementCohort()`
- `deathCohort()`

Create the cdm_reference object

```
cdm <- omock::mockCdmFromDataset(datasetName = "GiBleed",  
                                  source = "duckdb")
```

Demographics based - Example

- Two cohorts, females and males, both aged 18 to 60 years old, with at least 365 days of previous observation in the database.

```
cdm$age_cohort <- demographicsCohort(  
  cdm = cdm,  
  ageRange = c(18, 60),  
  sex = c("Female", "Male"),  
  minPriorObservation = 365,  
  name = "age_cohort"  
)  
  
settings(x = cdm$age_cohort)
```

A tibble: 2 × 5

	cohort_definition_id	cohort_name	age_range	sex	min_prior_observation
	<int>	<chr>	<chr>	<chr>	<dbl>
1	1	demographics_1	18_60	Female	365
2	2	demographics_2	18_60	Male	365

Demographics based - Example

```
cohortCount(cohort = cdm$age_cohort)
```

```
# A tibble: 2 × 3
```

	cohort_definition_id	number_records	number_subjects
	<int>	<int>	<int>
1	1	1373	1373
2	2	1321	1321

```
attrition(x = cdm$age_cohort)
```

```
# A tibble: 12 × 7
```

	cohort_definition_id	number_records	number_subjects	reason_id	reason
excluded_records	excluded_subjects				
	<int>	<int>	<int>	<int>	<chr>
<int>	<int>				
1	1	2694	2694	1	Initial qualifying events
0	0				
2	1	2694	2694	2	Non-missing sex
0	0				
3	1	1373	1373	3	Sex requirement: Female
1321	1321				
4	1	1373	1373	4	Non-missing year of birth
0	0				
5	1	1373	1373	5	Age requirement: 18 to 60

Demographics based - Example

To better visualise the **attrition**, we can use the package **CohortCharacteristics** to create a **formatted table**:

```
cdm$age_cohort |>
  summariseCohortAttrition() |>
  tableCohortAttrition()
```

Reason	Variable name			
	number_records	number_subjects	excluded_records	excluded_subjects
GiBleed; demographics_1				
Initial qualifying events	2,694	2,694	0	0
Non-missing sex	2,694	2,694	0	0
Sex requirement: Female	1,373	1,373	1,321	1,321
Non-missing year of birth	1,373	1,373	0	0
Age requirement: 18 to 60	1,373	1,373	0	0
Prior observation requirement: 365 days	1,373	1,373	0	0
GiBleed; demographics_2				
Initial qualifying events	2,694	2,694	0	0
Non-missing sex	2,694	2,694	0	0
Sex requirement: Male	1,321	1,321	1,373	1,373
Non-missing year of birth	1,321	1,321	0	0
Age requirement: 18 to 60	1,321	1,321	0	0
Prior observation requirement: 365 days	1,321	1,321	0	0

Concept based - Example

Let's create a cohort of medications that contains two drugs: diclofenac, and acetaminophen.

1. Create codelist of the drug of interest

```
drugCodes <- list(  
  "diclofenac" = c(1124300L),  
  "acetaminophen" = c(1125315L, 1127078L, 1127433L, 19133768L, 40162522L, 40229134L, 40231925L)  
)
```

Concept based - Example

2. Create concept based cohorts

```
cdm$medications <- conceptCohort(  
  cdm = cdm,  
  conceptSet = drugCodes,  
  name = "medications"  
)
```

```
settings(x = cdm$medications)
```

```
# A tibble: 2 × 4
```

	cohort_definition_id	cohort_name	cdm_version	vocabulary_version
	<int>	<chr>	<chr>	<chr>
1	1	acetaminophen	5.3	v5.0 18-JAN-19
2	2	diclofenac	5.3	v5.0 18-JAN-19

Concept based - Example

- Attrition

Reason	Variable name			
	number_records	number_subjects	excluded_records	excluded_subjects
acetaminophen				
Initial qualifying events	14,205	2,679	0	0
Record in observation	14,205	2,679	0	0
Not missing record date	14,205	2,679	0	0
Merge overlapping records	13,908	2,679	297	0
diclofenac				
Initial qualifying events	850	850	0	0
Record in observation	830	830	20	20
Not missing record date	830	830	0	0
Merge overlapping records	830	830	0	0

Concept based - Example

- Cohort codelist as an attribute

```
attr(cdm$medications, "cohort_codelist")
```

```
# A query: ?? x 4
# Database: DuckDB 1.5.5 [unknown@Linux 6.17.0-1022-azure:R 4.6.1//tmp/Rtmpu0okBt/file202c7ee02d69.duckdb]
  cohort_definition_id codelist_name concept_id codelist_type
      <int> <chr>           <int> <chr>
1             1 acetaminophen   1125315 index event
2             1 acetaminophen   1127078 index event
3             1 acetaminophen   1127433 index event
4             1 acetaminophen  19133768 index event
5             1 acetaminophen   40162522 index event
6             1 acetaminophen   40229134 index event
7             1 acetaminophen   40231925 index event
8             2 diclofenac       1124300 index event
```



Your turn!



Exercise 1 - Base cohorts

Create a base cohort table named **base_conditions_cohort** that has two cohorts: Rheumatoid Arthritis (RA) and Pneumonia. Summarise cohort counts and attrition. Use the codelist below.

```
codes <- list(  
  "ra" = c(80809L),  
  "pneumonia" = c(255848L)  
)
```



Click to see solution

Requirements and Filtering

Functions to apply requirements and filter

- On demographics
 - `requireDemographics()`
 - `requireAge()`
 - `requireSex()`
 - `requirePriorObservation()`
 - `requireFutureObservation()`
- On cohort entries
 - `requireIsFirstEntry()`
 - `requireIsLastEntry()`
 - `requireIsEntry()`
- Require presence or absence based on other cohorts, concepts, and tables
 - `requireCohortIntersect()`
 - `requireConceptIntersect()`
 - `requireTableIntersect()`
- Other
 - `requireInDateRange()`
 - `requireMincohortCount()`
 - `requireDuration()`

Requirement functions - Example

- We can apply different inclusion criteria using CohortConstructor's functions in a pipe-line fashion. For instance, in what follows we require
 - only first record per person
 - subjects 18 years old or more at cohort start date
 - only females
 - at least 30 days of prior observation at cohort start date

```
cdm$medications_requirement <- cdm$medications |>
  requireIsFirstEntry(name = "medications_requirement") |>
  requireDemographics(
    ageRange = list(c(18, 150)),
    sex = "Female",
    minPriorObservation = 30,
    name = "medications_requirement"
  )
```

Requirement functions - Example

Attrition Acetaminophen

Reason	Variable name			
	number_records	number_subjects	excluded_records	excluded_subjects
acetaminophen				
Initial qualifying events	14,205	2,679	0	0
Record in observation	14,205	2,679	0	0
Not missing record date	14,205	2,679	0	0
Merge overlapping records	13,908	2,679	297	0
Restricted to first entry	2,679	2,679	11,229	0
Age requirement: 18 to 150	309	309	2,370	2,370
Sex requirement: Female	176	176	133	133
Prior observation requirement: 30 days	176	176	0	0
Future observation requirement: 0 days	176	176	0	0

Requirement functions - Example

- Now, we only want to keep those records for which prescription happened on the same day of a healthcare visit.

```
cdm$medications_requirement <- cdm$medications_requirement |>
  requireTableIntersect(
    tableName = "visit_occurrence",
    indexDate = "cohort_start_date",
    window = c(0, 0),
    intersections = c(1, Inf)
  )
```

Requirement functions - Example

Attrition Acetaminophen

Reason	Variable name			
	number_records	number_subjects	excluded_records	excluded_subjects
acetaminophen				
Initial qualifying events	14,205	2,679	0	0
Record in observation	14,205	2,679	0	0
Not missing record date	14,205	2,679	0	0
Merge overlapping records	13,908	2,679	297	0
Restricted to first entry	2,679	2,679	11,229	0
Age requirement: 18 to 150	309	309	2,370	2,370
Sex requirement: Female	176	176	133	133
Prior observation requirement: 30 days	176	176	0	0
Future observation requirement: 0 days	176	176	0	0
In table visit_occurrence between 0 & 0 days relative to cohort_start_date between 1 and Inf	1	1	175	175



Your turn!



Exercise 2 - Requirement and filtering

Create a new cohort named “study_cohort” by applying the following criteria to the base conditions cohort:

- For all cohorts, require that records start after January 1, 1990.
- For the Rheumatoid Arthritis cohort, exclude patients with a history of osteoarthritis prior to or on the day of diagnosis.

```
osteoarthritisCodes <- list("osteoarthritis" = c(80180L))
```

▶  Click to see solution

Update cohort entry and exit

Functions to update cohort start and end dates

- Cohort exit
 - `exitAtObservationEnd()`
 - `exitAtDeath()`
 - `exitAtFirstDate()`
 - `exitAtLastDate()`
- Cohort entry
 - `entryAtFirstDate()`
 - `entryAtLastDate()`
- Trim start and end dates
 - `trimDemographics()`
 - `trimToDateRange()`
 - `trimDuration()`
- Pad start and end dates
 - `padCohortDate()`
 - `padCohortEnd()`
 - `padCohortStart()`

Update cohort entry and exit - Example

- We can trim start and end dates to match demographic requirements.
- For instance, cohort dates can be trimmed so the subject contributes time while:
 - Aged 20 to 40 years old
 - Prior observation of at least 365 days

```
cdm$medications_trimmed <- cdm$medications |>
  trimDemographics(
    ageRange = list(c(20, 40)),
    minPriorObservation = 365,
    name = "medications_trimmed"
  )
```

Update cohort entry and exit - Example

Reason	Variable name			
	number_records	number_subjects	excluded_records	excluded_subjects
acetaminophen				
Initial qualifying events	14,205	2,679	0	0
Record in observation	14,205	2,679	0	0
Not missing record date	14,205	2,679	0	0
Merge overlapping records	13,908	2,679	297	0
Non-missing year of birth	13,908	2,679	0	0
Age requirement: 20 to 40	4,049	2,124	9,859	555
Prior observation requirement: 365 days	4,049	2,124	0	0



Your turn!



Exercise 3 - Update cohort entry and exit

1) Since Rheumatoid Arthritis is a chronic condition, define its cohort exit so patients remain in the cohort from first diagnostic date to the end of the patient's observation.

👉 Use the *cohortId* argument to choose which cohort to modify.

▶ 💡 Click to see solution

CohortConstructor

- 👉 Packages website
- 👉 CRAN link
- 👉 Manual



CohortCharacteristics

An R package to characterise cohorts

2026-09-17

CohortCharacteristics



Website

CohortCharacteristics is on CRAN:

```
install.packages("CohortCharacteristics")
```

The package documentation and vignettes are available on the website:

<https://darwin-eu.github.io/CohortCharacteristics/>

Set up

```
options("width"=115)
library(duckdb)
library(CDMConnector)
library(PatientProfiles)
library(CohortConstructor)
library(dplyr)
library(here)
library(CohortCharacteristics)

cdm <- omock::mockCdmFromDataset(datasetName = "GiBleed",
                                source = "duckdb")
```

Let's get started

Let's create some cohorts that we will use during the presentation:

```
codes <- c(list(anemia = 439777,  
               rheumatoid_arthritis = 80809,  
               osteoarthritis = 80180,  
               pneumonia = 255848))  
  
cdm[["my_cohort"]] <- conceptCohort(cdm = cdm,  
                                   conceptSet = codes,  
                                   name = "my_cohort")  
  
cdm[["my_cohort"]] |>  
  glimpse()
```

Rows: ??

Columns: 4

```
$ cohort_definition_id <int> 1, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, ...  
$ subject_id          <int> 3699, 359, 377, 793, 1950, 2483, 2509, 3582, 3745, 4239, 5317, 250, 1138, 1211, 1776...  
$ cohort_start_date   <date> 1999-07-08, 2014-10-28, 2003-02-13, 2010-05-14, 1978-12-17, 1984-08-11, 2006-05-20,...  
$ cohort_end_date     <date> 1999-07-08, 2014-10-28, 2003-02-13, 2010-05-14, 1978-12-17, 1984-08-11, 2006-05-20,...
```

Workflow

We have three types of functions:

- **summarise**: these functions produce a standardised summary of a cohort. This output is called `<summarised_result>`.
- **table**: these functions produce tables from the `<summarised_result>` object generated by the *summarise* functions.
- **plot**: these functions produce plots from the `<summarised_result>` object generated by the *summarise* functions. (We won't show these today)

Contents

- summariseCohortCount
- summariseCohortAttrition
- summariseCharacteristics
- summariseLargeScaleCharacteristics
- summariseCohortTiming
- summariseCohortOverlap

summariseCohortCount

Use `summariseCohortCount()` to get counts for your cohort. Pipe your cohort table into the function as shown below.

```
results <- cdm$my_cohort |>
  summariseCohortCount()
```

It returns cohort counts in a `summarised_result` object.

```
glimpse(results)
```

```
Rows: 8
Columns: 13
$ result_id      <int> 1, 1, 1, 1, 1, 1, 1, 1
$ cdm_name       <chr> "GiBleed", "GiBleed", "GiBleed", "GiBleed", "GiBleed", "GiBleed", "GiBleed", "GiBleed"
$ group_name     <chr> "cohort_name", "cohort_name", "cohort_name", "cohort_name", "cohort_name", "cohort_name", "cohort_name"...
$ group_level    <chr> "anemia", "anemia", "osteoarthritis", "osteoarthritis", "pneumonia", "pneumonia", "rheum..."
$ strata_name    <chr> "overall", "overall", "overall", "overall", "overall", "overall", "overall", "overall"
$ strata_level   <chr> "overall", "overall", "overall", "overall", "overall", "overall", "overall", "overall"
$ variable_name  <chr> "Number records", "Number subjects", "Number records", "Number subjects", "Number record..."
$ variable_level <chr> NA, NA, NA, NA, NA, NA, NA, NA
$ estimate_name  <chr> "count", "count", "count", "count", "count", "count", "count", "count"
$ estimate_type  <chr> "integer", "integer", "integer", "integer", "integer", "integer", "integer", "integer"
$ estimate_value <chr> "100", "100", "2630", "2630", "52", "52", "18", "18"
$ additional_name <chr> "overall", "overall", "overall", "overall", "overall", "overall", "overall", "overall"
```

summariseCohortCount

Note

Previously we saw the function `cohortCount()` to see the counts of a `<cohort_table>`.

```
cohortCount(cdm$my_cohort)
```

```
# A tibble: 4 × 3
  cohort_definition_id number_records number_subjects
      <int>           <int>           <int>
1             1           100             100
2             2          2630            2630
3             3            52             52
4             4            18             18
```

The function `summariseCohortCount()` produces a similar result but in a standardised output (`<summarised_result>`).

tableCohortCount

To get a tidy table from the `<summarised_result>` object.

```
tableCohortCount(result = results)
```

CDM name	Variable name	Estimate name	Cohort name			
			anemia	osteoarthritis	pneumonia	rheumatoid_arthritis
GiBleed	Number records	N	100	2,630	52	18
	Number subjects	N	100	2,630	52	18

summariseCohortAttrition

To summarise cohort attrition, pipe your cohort table into the function as shown below.

```
results <- cdm$my_cohort |>
  summariseCohortAttrition()
```

It returns cohort attrition in a **summarised_result** object.

```
glimpse(results)
```

```

Rows: 64
Columns: 13
$ result_id      <int> 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2...
$ cdm_name       <chr> "GiBleed", "GiBleed", "GiBleed", "GiBleed", "GiBleed", "GiBleed", "GiBleed", "GiBleed", "GiBleed", ...
$ group_name     <chr> "cohort_name", "cohort_name", "cohort_name", "cohort_name", "cohort_name", "cohort_name", "cohort_name"...
$ group_level    <chr> "anemia", "anemia", "anemia", "anemia", "anemia", "anemia", "anemia", "anemia", "anemia", "anemia"...
$ strata_name    <chr> "reason", "reason", "reason", "reason", "reason", "reason", "reason", "reason", "reason", "reason"...
$ strata_level   <chr> "Initial qualifying events", "Initial qualifying events", "Initial qualifying events", "...
$ variable_name  <chr> "number_records", "number_subjects", "excluded_records", "excluded_subjects", "number_re...
$ variable_level <chr> NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, ...
$ estimate_name  <chr> "count", "count", "count", "count", "count", "count", "count", "count", "count", "count", "count"...
$ estimate_type  <chr> "integer", "integer", "integer", "integer", "integer", "integer", "integer", "integer", "integer", ...
$ estimate_value <chr> "102", "102", "0", "0", "100", "100", "2", "2", "100", "100", "0", "0", "100", "100", "0...
$ additional_name <chr> "reason_id", "reason_id", "reason_id", "reason_id", "reason_id", "reason_id", "reason_id", "reason_id"...

```

summariseCohortAttrition



Note

Yesterday we saw the function `attrition()` to see the attrition of a `<cohort_table>`.

```
attrition(cdm$my_cohort)
```

```
# A tibble: 16 × 7
```

	cohort_definition_id	number_records	number_subjects	reason_id	reason	excluded_records	excluded_subjects
	<int>	<int>	<int>	<int>	<chr>	<int>	<int>
1	1	102	102	1	Initial qualif...	0	0
2	1	100	100	2	Record in obse...	2	2
3	1	100	100	3	Not missing re...	0	0
4	1	100	100	4	Merge overlapp...	0	0
5	2	2694	2694	1	Initial qualif...	0	0
6	2	2630	2630	2	Record in obse...	64	64
7	2	2630	2630	3	Not missing re...	0	0
8	2	2630	2630	4	Merge overlapp...	0	0
9	3	52	52	1	Initial qualif...	0	0
10	3	52	52	2	Record in obse...	0	0
11	3	52	52	3	Not missing re...	0	0
12	3	52	52	4	Merge overlapp...	0	0
13	4	18	18	1	Initial qualif...	0	0
14	4	18	18	2	Record in obse...	0	0
15	4	18	18	3	Not missing re...	0	0
16	4	18	18	4	Merge overlapp...	0	0

The function `summariseCohortAttrition()` produces a similar result but in a standardised output (`<summarised_result>`).

tableCohortAttrition

To get a tidy table from the `<summarised_result>` object.

```
tableCohortAttrition(result = results)
```

Reason	Variable name			
	number_records	number_subjects	excluded_records	excluded_subjects
GiBleed; anemia				
Initial qualifying events	102	102	0	0
Record in observation	100	100	2	2
Not missing record date	100	100	0	0
Merge overlapping records	100	100	0	0
GiBleed; osteoarthritis				
Initial qualifying events	2,694	2,694	0	0
Record in observation	2,630	2,630	64	64
Not missing record date	2,630	2,630	0	0
Merge overlapping records	2,630	2,630	0	0
GiBleed; pneumonia				
Initial qualifying events	52	52	0	0
Record in observation	52	52	0	0
Not missing record date	52	52	0	0
Merge overlapping records	52	52	0	0
GiBleed; rheumatoid_arthritis				
Initial qualifying events	18	18	0	0
Record in observation	18	18	0	0

summariseCharacteristics

The main functionality of the package is to extract cohort characteristics with `summariseCharacteristics()`.

```
results <- cdm$my_cohort |>
  summariseCharacteristics()
```

It returns cohort characteristics in a `<summarised_result>` object.

```
glimpse(results)
```

Rows: 204

Columns: 13

```
$ result_id      <int> 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1...
$ cdm_name       <chr> "GiBleed", "GiBleed", "GiBleed", "GiBleed", "GiBleed", "GiBleed", "GiBleed", "GiBleed", "GiBleed", ...
$ group_name     <chr> "cohort_name", "cohort_name", "cohort_name", "cohort_name", "cohort_name", "cohort_name", "cohort_name"...
$ group_level    <chr> "anemia", "anemia", "anemia", "anemia", "anemia", "anemia", "anemia", "anemia", "anemia", "anemia"...
$ strata_name    <chr> "overall", "overall", "overall", "overall", "overall", "overall", "overall", "overall", "overall", ...
$ strata_level   <chr> "overall", "overall", "overall", "overall", "overall", "overall", "overall", "overall", "overall", ...
$ variable_name  <chr> "Number records", "Number subjects", "Cohort start date", "Cohort start date", "Cohort s...
$ variable_level <chr> NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, "Female", "F...
$ estimate_name  <chr> "count", "count", "min", "q25", "median", "q75", "max", "min", "q25", "median", "q75", "...
$ estimate_type  <chr> "integer", "integer", "date", "date", "date", "date", "date", "date", "date", "date", "date", "d...
$ estimate_value <chr> "100", "100", "1912-04-30", "1974-01-30", "1990-12-04", "2005-08-09", "2018-11-08", "191...
$ additional_name <chr> "overall", "overall", "overall", "overall", "overall", "overall", "overall", "overall", "overall", ...
```

tableCharacteristics

To get a tidy table from the `<summarised_result>` object.

```
tableCharacteristics(result = results, header = c("cdm_name", "cohort_name"))
```

			CDM name			
			GiBleed			
			Cohort name			
Variable name	Variable level	Estimate name	anemia	osteoarthritis	pneumonia	rheumatoid_arthritis
Number records	–	N	100	2,630	52	18
Number subjects	–	N	100	2,630	52	18
Cohort start date	–	Median [Q25 - Q75]	1990-12-04 [1974-01-30 - 2005-08-09]	1999-11-16 [1988-03-03 - 2008-08-07]	2007-07-06 [1998-09-17 - 2013-04-07]	2000-08-10 [1992-01-18 - 2008-01-01]
		Range	1912-04-30 to 2018-11-08	1941-12-05 to 2019-04-14	1980-03-03 to 2019-02-18	1971-07-29 to 2017-06-06
Cohort end date	–	Median [Q25 - Q75]	1990-12-04 [1974-01-30 - 2005-08-09]	1999-11-16 [1988-03-03 - 2008-08-07]	2007-07-06 [1998-09-17 - 2013-04-07]	2004-11-17 [1993-03-05 - 2011-05-22]
		Range	1912-04-30 to 2018-11-08	1941-12-05 to 2019-04-14	1980-03-03 to 2019-02-18	1971-07-29 to 2017-06-06
Age	–	Median [Q25 - Q75]	32.00 [17.75 - 50.25]	38.00 [36.00 - 41.00]	82.00 [78.00 - 86.25]	41.00 [34.25 - 55.00]
		Mean (SD)	33.49 (22.14)	38.15 (3.27)	80.33 (9.48)	45.22 (15.11)
		Range	0.00 to 81.00	31.00 to 46.00	53.00 to 96.00	24.00 to 77.00
Sex	Female	N (%)	78 (78.00%)	1,341 (50.99%)	36 (69.23%)	8 (44.44%)
	Male	N (%)	22 (22.00%)	1,289 (49.01%)	16 (30.77%)	10 (55.56%)
Prior observation	–	Median [Q25 - Q75]	12,045.00 [6,843.00 - 18,706.00]	14,097.00 [13,161.25 - 15,039.00]	30,079.00 [28,538.25 - 31,726.50]	15,330.00 [12,866.25 - 20,440.00]
		Mean (SD)	12,570.42 (8,107.27)	14,105.97 (1,193.14)	29,498.13 (3,467.33)	16,870.94 (5,513.54)
		Range	0.00 to 29,930.00	11,340.00 to 17,044.00	19,485.00 to 35,210.00	9,125.00 to 28,469.00
Future observation	–	Median [Q25 - Q75]	9,641.50 [4,596.00 - 14,747.00]	6,790.00 [3,568.00 - 10,012.00]	12,000.00 [10,000.00 - 15,000.00]	6,065.50 [4,058.00 - 9,716.00]

Commonly used arguments for summariseCharacteristics

- **strata**: creates stratified results.
- **cohortId**: filters by cohort ID.
- **ageGroup**: defines the age groups to summarise.
- **tableIntersect**: count/flag/date/days
- **cohortIntersect**: count/flag/date/days
- **conceptIntersect**: count/flag/date/days
- **link to documentation**

ageGroup example

```
result <- summariseCharacteristics(cdm$my_cohort, ageGroup = list(c(0,10), c(11,18), c(19,150)))  
tableCharacteristics(result = result, header = "cohort_name")
```

ageGroup example

CDM name	Variable name	Variable level	Estimate name	Cohort name			
				anemia	osteoarthritis	pneumonia	rheumatoid_arthritis
GiBleed	Number records	—	N	100	2,630	52	18
	Number subjects	—	N	100	2,630	52	18
	Cohort start date	—	Median [Q25 - Q75]	1990-12-04 [1974-01-30 - 2005-08-09]	1999-11-16 [1988-03-03 - 2008-08-07]	2007-07-06 [1998-09-17 - 2013-04-07]	2000-08-10 [1992-01-18 - 2008-01-01]
			Range	1912-04-30 to 2018-11-08	1941-12-05 to 2019-04-14	1980-03-03 to 2019-02-18	1971-07-29 to 2017-06-06
	Cohort end date	—	Median [Q25 - Q75]	1990-12-04 [1974-01-30 - 2005-08-09]	1999-11-16 [1988-03-03 - 2008-08-07]	2007-07-06 [1998-09-17 - 2013-04-07]	2004-11-17 [1993-03-05 - 2011-05-22]
			Range	1912-04-30 to 2018-11-08	1941-12-05 to 2019-04-14	1980-03-03 to 2019-02-18	1971-07-29 to 2017-06-06
	Age	—	Median [Q25 - Q75]	32.00 [17.75 - 50.25]	38.00 [36.00 - 41.00]	82.00 [78.00 - 86.25]	41.00 [34.25 - 55.00]
			Mean (SD)	33.49 (22.14)	38.15 (3.27)	80.33 (9.48)	45.22 (15.11)
			Range	0.00 to 81.00	31.00 to 46.00	53.00 to 96.00	24.00 to 77.00
	Age group	0 to 10	N (%)	20 (20.00%)	0 (0.00%)	0 (0.00%)	0 (0.00%)
		11 to 18	N (%)	7 (7.00%)	0 (0.00%)	0 (0.00%)	0 (0.00%)
		19 to 150	N (%)	73 (73.00%)	2,630 (100.00%)	52 (100.00%)	18 (100.00%)
	Sex	Female	N (%)	78 (78.00%)	1,341 (50.99%)	36 (69.23%)	8 (44.44%)
		Male	N (%)	22 (22.00%)	1,289 (49.01%)	16 (30.77%)	10 (55.56%)
	Prior observation	—	Median [Q25 - Q75]	12,045.00 [6,843.00 - 18,706.00]	14,097.00 [13,161.25 - 15,039.00]	30,079.00 [28,538.25 - 31,726.50]	15,330.00 [12,866.25 - 20,440.00]
			Mean (SD)	12,570.42 (8,107.27)	14,105.97 (1,193.14)	29,498.13 (3,467.33)	16,870.94 (5,513.54)
			Range	0.00 to 29,930.00	11,340.00 to 17,044.00	19,485.00 to 35,210.00	9,125.00 to 28,469.00
	Future observation		Median [Q25 - Q75]	9,641.50 [4,596.00 -	6,790.00 [3,568.00 -	12,000.00 [10,000 - 15,000]	6,065.50 [4,058.00 -

Strata example

```
result <- cdm$my_cohort |>
  addAge(ageGroup = list(c(0,10),c(11,18), c(19,150))) |>
  summariseCharacteristics(cohortId = "anemia",
                           strata = list("age_group"))
tableCharacteristics(result = result, header = c("cdm_name", "cohort_name","age_group"))
```

Strata example

			CDM name			
			GiBleed			
			Cohort name			
			anemia			
Variable name	Variable level	Estimate name	Age group			
			overall	0 to 10	11 to 18	19 to 150
Number records	–	N	100	20	7	73
Number subjects	–	N	100	20	7	73
Cohort start date	–	Median [Q25 - Q75]	1990-12-04 [1974-01-30 - 2005-08-09]	1963-03-26 [1956-03-03 - 1972-12-27]	1969-08-19 [1963-03-03 - 1975-04-11]	1999-03-30 [1987-11-03 - 2007-06-08]
		Range	1912-04-30 to 2018-11-08	1912-04-30 to 1987-01-08	1951-04-05 to 1986-04-07	1939-01-29 to 2018-11-08
Cohort end date	–	Median [Q25 - Q75]	1990-12-04 [1974-01-30 - 2005-08-09]	1963-03-26 [1956-03-03 - 1972-12-27]	1969-08-19 [1963-03-03 - 1975-04-11]	1999-03-30 [1987-11-03 - 2007-06-08]
		Range	1912-04-30 to 2018-11-08	1912-04-30 to 1987-01-08	1951-04-05 to 1986-04-07	1939-01-29 to 2018-11-08
Age	–	Median [Q25 - Q75]	32.00 [17.75 - 50.25]	2.50 [0.75 - 5.25]	17.00 [14.50 - 17.50]	43.00 [28.00 - 56.00]
		Mean (SD)	33.49 (22.14)	3.30 (3.15)	15.86 (2.27)	43.45 (16.93)
		Range	0.00 to 81.00	0.00 to 9.00	12.00 to 18.00	19.00 to 81.00
Sex	Female	N (%)	78 (78.00%)	10 (50.00%)	6 (85.71%)	62 (84.93%)
	Male	N (%)	22 (22.00%)	10 (50.00%)	1 (14.29%)	11 (15.07%)
Prior observation	–	Median [Q25 - Q75]	12,045.00 [6,843.00 - 18,706.00]	1,094.50 [365.00 - 2,280.25]	6,570.00 [5,657.00 - 6,752.00]	16,060.00 [10,585.00 - 20,804.00]
		Mean (SD)	12,570.42 (8,107.27)	1,478.00 (1,221.91)	6,152.57 (827.72)	16,224.85 (6,180.75)
		Range	0.00 to 29,930.00	0.00 to 3,650.00	4,745.00 to 6,935.00	7,300.00 to 29,930.00

Intersection with another table

You can get count/flag/date/days summaries with other tables.

```
result <- cdm$my_cohort |>
  summariseCharacteristics(
    cohortId = "anemia",
    tableIntersectCount = list(
      "Number of visits prior year" = list(
        tableName = "visit_occurrence",
        window = c(-365, 0)
      )
    ),
    cohortIntersectFlag = list(
      "Previous rheumatoid arthritis diagnosis" = list(
        targetCohortTable = "my_cohort",
        targetCohortId = "rheumatoid_arthritis",
        window = c(-Inf, 0)
      )
    ),
    conceptIntersectDays = list(
      "Time since last X-ray" = list(
```

Intersection with another table

You can get count/flag/date/days summaries with other tables.

CDM name	Variable name	Variable level	Estimate name	Cohort name			
				anemia	osteoarthritis	pneumonia	rheumatoid_arthritis
GiBleed	Number records	—	N	100	2,630	52	18
	Number subjects	—	N	100	2,630	52	18
	Cohort start date	—	Median [Q25 - Q75]	1990-12-04 [1974-01-30 - 2005-08-09]	1999-11-16 [1988-03-03 - 2008-08-07]	2007-07-06 [1998-09-17 - 2013-04-07]	2000-08-10 [1992-01-18 - 2008-01-01]
			Range	1912-04-30 to 2018-11-08	1941-12-05 to 2019-04-14	1980-03-03 to 2019-02-18	1971-07-29 to 2017-06-06
	Cohort end date	—	Median [Q25 - Q75]	1990-12-04 [1974-01-30 - 2005-08-09]	1999-11-16 [1988-03-03 - 2008-08-07]	2007-07-06 [1998-09-17 - 2013-04-07]	2004-11-17 [1993-03-05 - 2011-05-22]
			Range	1912-04-30 to 2018-11-08	1941-12-05 to 2019-04-14	1980-03-03 to 2019-02-18	1971-07-29 to 2017-06-06
	Age	—	Median [Q25 - Q75]	32.00 [17.75 - 50.25]	38.00 [36.00 - 41.00]	82.00 [78.00 - 86.25]	41.00 [34.25 - 55.00]
			Mean (SD)	33.49 (22.14)	38.15 (3.27)	80.33 (9.48)	45.22 (15.11)
			Range	0.00 to 81.00	31.00 to 46.00	53.00 to 96.00	24.00 to 77.00
	Sex	Female	N (%)	78 (78.00%)	1,341 (50.99%)	36 (69.23%)	8 (44.44%)
		Male	N (%)	22 (22.00%)	1,289 (49.01%)	16 (30.77%)	10 (55.56%)
	Prior observation	—	Median [Q25 - Q75]	12,045.00 [6,843.00 - 18,706.00]	14,097.00 [13,161.25 - 15,039.00]	30,079.00 [28,538.25 - 31,726.50]	15,330.00 [12,866.25 - 20,440.00]
			Mean (SD)	12,570.42 (8,107.27)	14,105.97 (1,193.14)	29,498.13 (3,467.33)	16,870.94 (5,513.54)
			Range	0.00 to 29,930.00	11,340.00 to 17,044.00	19,485.00 to 35,210.00	9,125.00 to 28,469.00
	Future observation	—	Median [Q25 - Q75]	9,641.50 [4,596.00 - 15,796.25]	6,790.00 [3,568.00 - 10,781.25]	12.00 [10.00 - 15.00]	6,065.50 [4,058.00 - 9,716.00]
			Mean (SD)	10,773.47 (7,880.91)	7,686.76 (5,364.39)	12.25 (3.16)	6,978.83 (4,735.79)
			Range	1.00 to 39,143.00	10.00 to 28,328.00	7.00 to 19.00	668.00 to 17,356.00

Intersections

Note

The arguments of the intersection functions match the *PatientProfiles* functions that we have just seen:

- `tableIntersectFlag -> PatientProfiles::addTableIntersectFlag()`
- `tableIntersectCount -> PatientProfiles::addTableIntersectCount()`
- `tableIntersectDays -> PatientProfiles::addTableIntersectDays()`
- `tableIntersectDate -> PatientProfiles::addTableIntersectDate()`
- `conceptIntersectFlag -> PatientProfiles::addConceptIntersectFlag()`
- `conceptIntersectCount -> PatientProfiles::addConceptIntersectCount()`
- `conceptIntersectDays -> PatientProfiles::addConceptIntersectDays()`
- `conceptIntersectDate -> PatientProfiles::addConceptIntersectDate()`
- `cohortIntersectFlag -> PatientProfiles::addCohortIntersectFlag()`
- `cohortIntersectCount -> PatientProfiles::addCohortIntersectCount()`
- `cohortIntersectDays -> PatientProfiles::addCohortIntersectDays()`
- `cohortIntersectDate -> PatientProfiles::addCohortIntersectDate()`



Your turn!

Exercise 1 - characterise an osteoarthritis cohort

```
cdm <- omock::mockCdmFromDataset(datasetName = "GiBleed",  
                                  source = "duckdb")  
  
cdm[["my_cohort"]] <- conceptCohort(cdm = cdm,  
                                   conceptSet = codes,  
                                   name = "my_cohort")
```

Can you characterise the **osteoarthritis** cohort?

- Start with only demographics
- Add if they had any anemia diagnosis during the last 180 days (including index date).
- Add if they had any pneumonia diagnosis last 2 years (excluding index date).

►  [Click to see solution](#)

summariseLargeScaleCharacteristics

Sometimes we might want to summarise all clinical events for cohorts across different time windows. We can do this with `summariseLargeScaleCharacteristics()`.

```
result <- cdm$my_cohort |>
  summariseLargeScaleCharacteristics(
    window = list(c(-Inf, -1), c(1, Inf)),
    eventInWindow = "condition_occurrence",
    minimumFrequency = 0.05
  )

glimpse(result)
```

Rows : 250

Columns: 13

```
$ result_id      <int> 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1...
$ cdm_name      <chr> "GiBleed", "GiBleed", "GiBleed", "GiBleed", "GiBleed", "GiBleed", "GiBleed", "GiBleed", "GiBleed", ...
$ group_name    <chr> "cohort_name", "cohort_name", "cohort_name", "cohort_name", "cohort_name", "cohort_name", "cohort_name"...
$ group_level   <chr> "anemia", "anemia", "anemia", "anemia", "anemia", "anemia", "anemia", "anemia", "anemia", "anemia"...
$ strata_name   <chr> "overall", "overall", "overall", "overall", "overall", "overall", "overall", "overall", "overall", ...
$ strata_level  <chr> "overall", "overall", "overall", "overall", "overall", "overall", "overall", "overall", "overall", ...
$ variable_name <chr> "Acute allergic reaction", "Acute allergic reaction", "Acute bronchitis", "Acute bronchi...
$ variable_level <chr> "-inf to -1", "-inf to -1", "-inf to -1", "-inf to -1", "-inf to -1", "-inf to -1", "-inf to -1", "-in...
$ estimate_name <chr> "count", "percentage", "count", "percentage", "count", "percentage", "count", "percentage"...
$ estimate_type <chr> "integer", "percentage", "integer", "percentage", "integer", "percentage", "integer", "integer", "p...
$ estimate_value <chr> "5", "5.10", "77", "78.57", "75", "76.53", "7", "7.14", "6", "6.12", "6", "6.12", "7", "...
$ additional_name <chr> "concept_id", "concept_id", "concept_id", "concept_id", "concept_id", "concept_id", "concept_id", "con..."
```

tableTopLargeScaleCharacteristics

`tableTopLargeScaleCharacteristics()` creates a tidy table of the most frequent concepts.

```
tableTopLargeScaleCharacteristics(result = result, topConcepts = 5)
```

Top	Cohort name						
	anemia		osteoarthritis		pneumonia	rheumatoid_arthritis	
			Window				
	-inf to -1	1 to inf	-inf to -1	1 to inf	-inf to -1	-inf to -1	1 to inf
1	Viral sinusitis (40481087) 82 (83.7%)	Viral sinusitis (40481087) 82 (82.0%)	Viral sinusitis (40481087) 2594 (98.6%)	Viral sinusitis (40481087) 2066 (78.6%)	Alzheimer's disease (378419) 43 (82.7%)	Acute viral pharyngitis (4112343) 17 (94.4%)	Viral sinusitis (40481087) 13 (72.2%)
2	Acute bronchitis (260139) 77 (78.6%)	Acute viral pharyngitis (4112343) 70 (70.0%)	Acute viral pharyngitis (4112343) 2403 (91.4%)	Acute viral pharyngitis (4112343) 1664 (63.3%)	Streptococcal sore throat (28060) 43 (82.7%)	Viral sinusitis (40481087) 17 (94.4%)	Acute viral pharyngitis (4112343) 12 (66.7%)
3	Acute viral pharyngitis (4112343) 75 (76.5%)	Acute bronchitis (260139) 65 (65.0%)	Acute bronchitis (260139) 2292 (87.2%)	Acute bronchitis (260139) 1515 (57.6%)	Otitis media (372328) 41 (78.8%)	Otitis media (372328) 13 (72.2%)	Acute cholecystitis (198809) 1 (5.6%)
4	Otitis media (372328) 69 (70.4%)	Osteoarthritis (80180) 58 (58.0%)	Otitis media (372328) 1935 (73.6%)	Chronic sinusitis (257012) 251 (9.5%)	Sprain of ankle (81151) 35 (67.3%)	Osteoarthritis (80180) 12 (66.7%)	Alzheimer's disease (378419) 1 (5.6%)
5	Streptococcal sore throat (28060) 59 (60.2%)	Angiodysplasia of stomach (4310024) 9 (9.0%)	Streptococcal sore throat (28060) 1399 (53.2%)	Cerebrovascular accident (381316) 194 (7.4%)	Laceration of foot (4109685) 5 (9.6%)	Streptococcal sore throat (28060) 9 (50.0%)	Anemia (439777) 1 (5.6%)

tidy

You can create a tidy table that you can explore with the `tidy()` function:

```
tidy(result)
```

```
# A tibble: 125 × 10
```

	cdm_name	cohort_name	variable_name	variable_level	concept_id	count	percentage	analysis	table_name	type
	<chr>	<chr>	<chr>	<chr>	<chr>	<int>	<dbl>	<chr>	<chr>	<chr>
1	GiBleed	anemia	Acute allergic reacti...	-inf to -1	4084167	5	5.1	standard	condition...	event
2	GiBleed	anemia	Acute bronchitis	-inf to -1	260139	77	78.6	standard	condition...	event
3	GiBleed	anemia	Acute viral pharyngit...	-inf to -1	4112343	75	76.5	standard	condition...	event
4	GiBleed	anemia	Angiodysplasia of sto...	-inf to -1	4310024	7	7.14	standard	condition...	event
5	GiBleed	anemia	Appendicitis	-inf to -1	440448	6	6.12	standard	condition...	event
6	GiBleed	anemia	Child attention defici...	-inf to -1	440086	6	6.12	standard	condition...	event
7	GiBleed	anemia	Concussion with loss ...	-inf to -1	375671	7	7.14	standard	condition...	event
8	GiBleed	anemia	Cystitis	-inf to -1	195588	7	7.14	standard	condition...	event
9	GiBleed	anemia	Facial laceration	-inf to -1	4156265	9	9.18	standard	condition...	event
10	GiBleed	anemia	First degree burn	-inf to -1	4296204	8	8.16	standard	condition...	event

```
# i 115 more rows
```

tidy

Note

The `tidy()` function works for any `<summarised_result>` object that we create.

summariseCohortOverlap

When creating multiple cohorts, we might be interested in the overlap of individuals between those cohorts. `summariseCohortOverlap()` does this for you.

```
result <- cdm$my_cohort |>
  summariseCohortOverlap()
```

```
glimpse(result)
```

Rows: 72

Columns: 13

```
$ result_id      <int> 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1...
$ cdm_name       <chr> "GiBleed", "GiBleed", "GiBleed", "GiBleed", "GiBleed", "GiBleed", "GiBleed", "GiBleed", "GiBleed", ...
$ group_name     <chr> "cohort_name_reference &&& cohort_name_comparator", "cohort_name_reference &&& cohort_na...
$ group_level    <chr> "anemia &&& osteoarthritis", "anemia &&& osteoarthritis", "anemia &&& osteoarthritis", "...
$ strata_name    <chr> "overall", "overall", "overall", "overall", "overall", "overall", "overall", "overall", "overall", ...
$ strata_level   <chr> "overall", "overall", "overall", "overall", "overall", "overall", "overall", "overall", "overall", ...
$ variable_name  <chr> "Only in reference cohort", "In both cohorts", "Only in comparator cohort", "Only in ref...
$ variable_level <chr> "Subjects", "Subjects", "Subjects", "Subjects", "Subjects", "Subjects", "Subjects", "Subjects", "Sub...
$ estimate_name  <chr> "count", "count", "count", "count", "count", "count", "count", "count", "count", "count", "count"...
$ estimate_type  <chr> "integer", "integer", "integer", "integer", "integer", "integer", "integer", "integer", "integer", ...
$ estimate_value <chr> "3", "97", "2533", "93", "7", "45", "99", "1", "17", "2533", "97", "3", "2578", "52", "0...
$ additional_name <chr> "overall", "overall", "overall", "overall", "overall", "overall", "overall", "overall", "overall", ...
```

tableCohortOverlap

You can also get a table with `tableCohortOverlap()`.

```
tableCohortOverlap(result)
```

Cohort name reference	Cohort name comparator	Estimate name	Variable name		
			Only in reference cohort	In both cohorts	Only in comparator cohort
GiBleed					
anemia	osteoarthritis	N (%)	3 (0.11%)	97 (3.68%)	2,533 (96.20%)
	pneumonia	N (%)	93 (64.14%)	7 (4.83%)	45 (31.03%)
	rheumatoid_arthritis	N (%)	99 (84.62%)	1 (0.85%)	17 (14.53%)
osteoarthritis	pneumonia	N (%)	2,578 (98.02%)	52 (1.98%)	0 (0.00%)
	rheumatoid_arthritis	N (%)	2,612 (99.32%)	18 (0.68%)	0 (0.00%)
pneumonia	rheumatoid_arthritis	N (%)	52 (74.29%)	0 (0.00%)	18 (25.71%)

summariseCohortTiming

When creating multiple cohorts, we might be interested in the timing of entry between cohorts. `summariseCohortTiming()` does this for you.

```
result <- cdm$my_cohort |>
  summariseCohortTiming(restrictToFirstEntry = TRUE)

glimpse(result)
```

Rows: 8,274

Columns: 13

```
$ result_id      <int> 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1...
$ cdm_name       <chr> "GiBleed", "GiBleed", "GiBleed", "GiBleed", "GiBleed", "GiBleed", "GiBleed", "GiBleed", "GiBleed", ...
$ group_name     <chr> "cohort_name_reference &&& cohort_name_comparator", "cohort_name_reference &&& cohort_na...
$ group_level    <chr> "anemia &&& osteoarthritis", "anemia &&& osteoarthritis", "anemia &&& osteoarthritis", "...
$ strata_name    <chr> "overall", "overall", "overall", "overall", "overall", "overall", "overall", "overall", "overall", ...
$ strata_level   <chr> "overall", "overall", "overall", "overall", "overall", "overall", "overall", "overall", "overall", ...
$ variable_name  <chr> "number records", "number subjects", "days_between_cohort_entries", "days_between_cohort...
$ variable_level <chr> NA, NA, NA, NA, NA, NA, NA, "density_001", "density_002", "density_003", "density_004", ...
$ estimate_name  <chr> "count", "count", "min", "q25", "median", "q75", "max", "density_x", "density_x", "densi...
$ estimate_type  <chr> "integer", "integer", "integer", "integer", "integer", "integer", "integer", "integer", "numeric", ...
$ estimate_value <chr> "97", "97", "-17715", "-4915", "2205", "6680", "15530", "-16313.4", "-16251.7980430528",...
$ additional_name <chr> "overall", "overall", "overall", "overall", "overall", "overall", "overall", "overall", "overall", ...
```

tableCohortTiming

You can also get a table with `tableCohortTiming()`.

```
tableCohortTiming(result = result,
  timeScale = "years",
  uniqueCombinations = FALSE)
```

Cohort name reference	Cohort name comparator	Variable name	Estimate name	Estimate value
GiBleed				
anemia	osteoarthritis	number records	N	97
		number subjects	N	97
		years_between_cohort_entries	Median [Q25 - Q75]	6.04 [-13.46 - 18.29]
			Range	-48.50 to 42.52
	pneumonia	number records	N	7
		number subjects	N	7
		years_between_cohort_entries	Median [Q25 - Q75]	29.90 [15.57 - 38.99]
			Range	1.77 to 53.45
	rheumatoid_arthritis	number records	N	1
		number subjects	N	1
		years_between_cohort_entries	Median [Q25 - Q75]	-17.99 [-17.99 - -17.99]
			Range	-17.99 to -17.99
osteoarthritis	anemia	number records	N	97
		number subjects	N	97
		years_between_cohort_entries	Median [Q25 - Q75]	-6.04 [-18.29 - 13.46]
			Range	-42.52 to 48.50
	pneumonia	number records	N	52

Cohort name reference	Cohort name comparator	Variable name	Estimate name	Estimate value
		number subjects	N	52
		years_between_cohort_entries	Median [Q25 - Q75]	43.49 [38.25 - 50.44]
			Range	16.24 to 61.03
	rheumatoid_arthritis	number records	N	18
		number subjects	N	18
		years_between_cohort_entries	Median [Q25 - Q75]	5.61 [-2.01 - 15.46]
			Range	-18.11 to 34.77
pneumonia	anemia	number records	N	7
		number subjects	N	7
		years_between_cohort_entries	Median [Q25 - Q75]	-29.90 [-38.99 - -15.57]
			Range	-53.45 to -1.77
	osteoarthritis	number records	N	52
		number subjects	N	52
		years_between_cohort_entries	Median [Q25 - Q75]	-43.49 [-50.44 - -38.25]
			Range	-61.03 to -16.24
rheumatoid_arthritis	anemia	number records	N	1
		number subjects	N	1
		years_between_cohort_entries	Median [Q25 - Q75]	17.99 [17.99 - 17.99]
			Range	17.99 to 17.99
	osteoarthritis	number records	N	18
		number subjects	N	18
		years_between_cohort_entries	Median [Q25 - Q75]	-5.61 [-15.46 - 2.01]
			Range	-34.77 to 18.11



Your turn!

Exercise 2 - large scale characteristics for anemia cohort

Can you get the large-scale characteristics from the **drug_exposure** table for the **anemia** cohort table using an anytime-prior time window (excluding index date)? Visualise the result showing the top 5 most frequent codes.

►  [Click to see solution](#)

Top	Estimate value
1	Acetaminophen 325 MG Oral Tablet (1127433) 79 (80.6%)
2	Aspirin 81 MG Oral Tablet (19059056) 71 (72.5%)
3	poliovirus vaccine, inactivated (40213160) 64 (65.3%)
4	Penicillin V Potassium 250 MG Oral Tablet (19133873) 54 (55.1%)
5	Ampicillin 100 MG/ML Injectable Solution (19129655) 9 (9.2%)

CohortCharacteristics

- 👉 [Package website](#)
- 👉 [CRAN link](#)
- 👉 [Manual](#)



PhenotypeR

Review codelists and cohorts in OMOP CDM

2026-09-17

Introduction

- PhenotypeR package can help us to assess the research-readiness of a set of cohorts we have defined.
- The code is publicly available in OHDSI's GitHub repository **PhenotypeR**.
- PhenotypeR 0.5 is available in **CRAN**.
- Vignettes with further information can be found in the package **website**.

Set of Functions: Individual Diagnostics Assessment

- **Database diagnostics**
 - `databaseDiagnostics()`
- **Cohort diagnostics**
 - `cohortDiagnostics()`
- **Codelist diagnostics**
 - `codelistDiagnostics()`
- **Population diagnostics**
 - `populationDiagnostics()`

Set of Functions: Phenotype Diagnostics

- Comprises all the diagnostics that are being offered in this package.

```
result <- phenotypeDiagnostics(  
  cohort,  
  databaseDiagnostics = list(),  
 odelistDiagnostics = list(),  
  cohortDiagnostics = list(),  
  populationDiagnostics = list()  
)
```

- Run only some of the diagnostics:

```
result <- phenotypeDiagnostics(  
  cohort,  
  databaseDiagnostics = NULL,  
  oodelistDiagnostics = list(),  
  cohortDiagnostics = NULL,  
  populationDiagnostics = list()  
)
```

Create a shiny app to visualise all the results!

- Create a shiny app to visualize all the results

```
shinyDiagnostics(result = result,  
                 directory = here(),  
                 minCellCount = 5,  
                 removeEmptyTabs = FALSE)
```

See the example app provided with the training materials.

Database Diagnostics

- Summarise the database metadata including:
 1. **Snapshot**
 2. Summary of the **person table**
 3. Summary of the **observation period**
 4. Summary of the **clinical tables** (i.e., *condition_occurrence*) where the concepts of the codelist defining your cohort are found.

Database Diagnostics

```
db_diagnostics <- databaseDiagnostics(  
  cohort,  
  cohortId = NULL,  
  snapshot = TRUE,  
  personTableSummary = TRUE,  
  observationPeriodsSummary = TRUE,  
  clinicalRecordsSummary = FALSE  
)  
  
# Modify databaseDiagnostics in phenotypeDiagnostics:  
result <- phenotypeDiagnostics(  
  cohort,  
  databaseDiagnostics = list(  
    "cohortId" = c(1,2),  
    "snapshot" = FALSE  
  )  
)
```

Codelist Diagnostics

- Summarise the codelist associated with your cohort including:
 1. **Achilles codes use** (only if ACHILLES tables are present)
 2. **Orphan codes use** (only if ACHILLES tables are present)
 3. **Cohort code use**
 4. **Measurement code use** (only if measurement concepts are present in your codelist)
 5. **Drug diagnostics** (only if drug concepts are present in your codelist)

Codelist Diagnostics

```
cl_diagnostics <- codelistDiagnostics(  
  cohort,  
  cohortId = NULL,  
  achillesCodeUse = FALSE,  
  orphanCodeUse = TRUE,  
  cohortCodeUse = TRUE,  
  drugDiagnostics = FALSE,  
  drugDiagnosticsSample = 20000,  
  measurementDiagnostics = FALSE,  
  measurementDiagnosticsSample = 20000  
)  
  
# Modify codelistDiagnostics in phenotypeDiagnostics:  
result <- phenotypeDiagnostics(  
  cohort,  
  codelistDiagnostics = list(  
    "cohortId" = c(1,2),  
    "achillesCodeUse" = FALSE
```

Cohort Diagnostics

- Summarise your cohort characteristics:
 1. **Cohort count & attrition**
 2. **Cohort characteristics**: Baseline characteristics of your cohort
 3. **Large scale characteristics**: Summary of the records from clinical tables within a time window
 4. **Compare cohorts**: Overlap and timing between cohorts (only if more than one cohort are present)
 5. **Cohort survival**: Survival until the event of death (if death table is present)

Cohort Diagnostics

```
c_diagnostics <- cohortDiagnostics(  
  cohort,  
  cohortId = NULL,  
  cohortCount = TRUE,  
  cohortCharacteristics = TRUE,  
  largeScaleCharacteristics = TRUE,  
  compareCohorts = FALSE,  
  cohortSurvival = FALSE,  
  cohortSample = 20000,  
  matchedSample = 1000  
)  
  
# Modify cohortDiagnostics in phenotypeDiagnostics:  
result <- phenotypeDiagnostics(  
  cohort,  
  cohortDiagnostics = list(  
    "cohortSample" = 1000  
  ),  
)
```

Population Diagnostics

- Contextualises the frequency of your study cohorts in the database calculating:
 1. **Incidence**
 2. **Period Prevalence**

Population Diagnostics

```
c_diagnostics <- populationDiagnostics(  
  cohort,  
  cohortId = NULL,  
  incidence = TRUE,  
  periodPrevalence = TRUE,  
  populationSample = 1e+05,  
  populationDateRange = as.Date(c(NA, NA))  
)  
  
# Modify populationDiagnostics in phenotypeDiagnostics:  
result <- phenotypeDiagnostics(  
  cohort,  
  populationDiagnostics = list(  
    "populationSample" = 10000  
  )  
)
```

Summary:

- **Database diagnostics**
 - `databaseDiagnostics()`
- **Codelist diagnostics**
 - `codelistDiagnostics()`
- **Cohort diagnostics**
 - `cohortDiagnostics()`
- **Population diagnostics**
 - `populationDiagnostics()`
- Run all the diagnostics together:

```
result <- phenotypeDiagnostics(  
  cohort,  
  databaseDiagnostics = list(),  
  codelistDiagnostics = list(),  
  cohortDiagnostics = list(),  
  populationDiagnostics = list(),  
  stagingDirectory = NULL  
)  
  
shinyDiagnostics(result = result,  
  directory = here(),  
  minCellCount = 5,  
  removeEmptyTabs = FALSE)
```

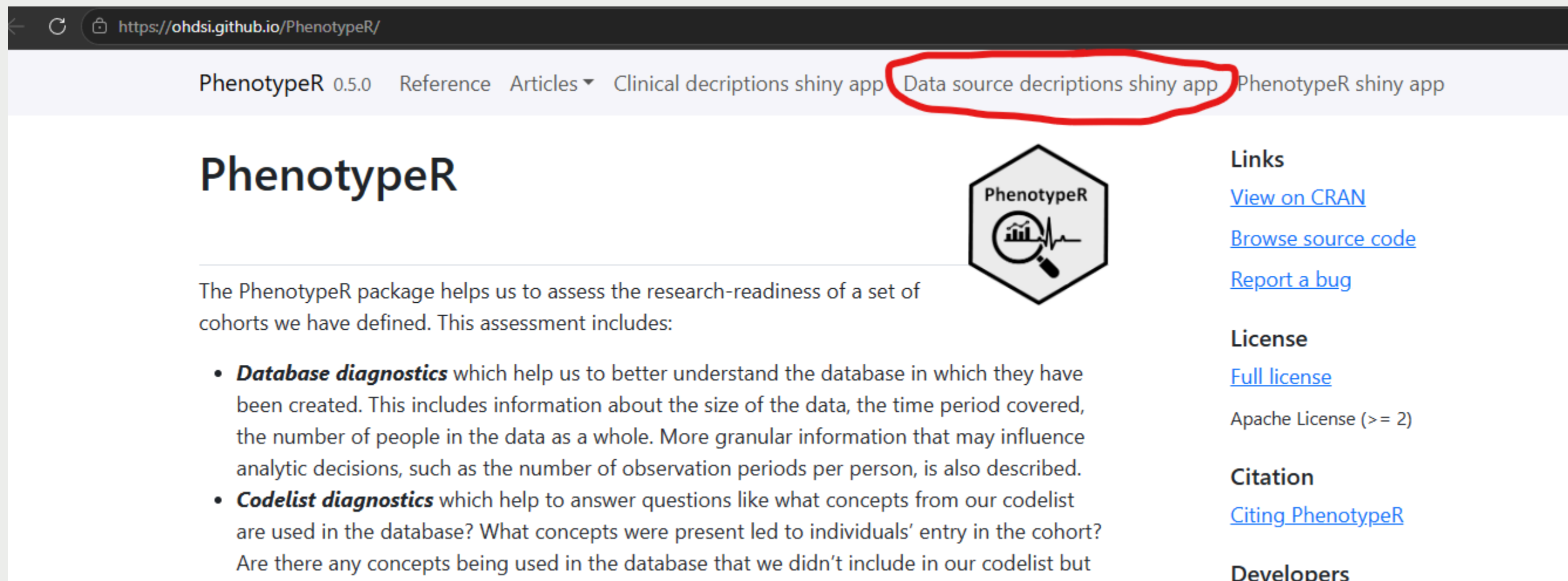
Extra: Changing additional parameters

- Modify extra parameters (*window*, *eventInWindow*, *episodesInWindow* for the LSC) using global variables:

```
options("PhenotypeR_summariseLargeScaleCharacteristics_window" = list(c(0, 0)))  
options("PhenotypeR_summariseLargeScaleCharacteristics_eventInWindow" = c("condition_occurrence"))  
options("PhenotypeR_summariseLargeScaleCharacteristics_episodeInWindow" = c("drug_exposure"))
```

Extra: Adding context to your results via data source description

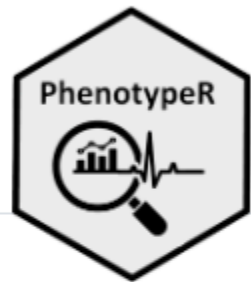
- Visit [Data source description shiny app](#)



The screenshot shows the PhenotypeR website at <https://ohdsi.github.io/PhenotypeR/>. The navigation bar includes links for PhenotypeR 0.5.0, Reference, Articles, Clinical descriptions shiny app, **Data source descriptions shiny app** (highlighted with a red circle), and PhenotypeR shiny app. The main content area features the PhenotypeR logo, a description of the package's purpose, and a list of diagnostic categories. A sidebar on the right contains links for View on CRAN, Browse source code, Report a bug, License (Full license), Apache License (>= 2), Citation (Citing PhenotypeR), and Developers.

PhenotypeR 0.5.0 Reference Articles Clinical descriptions shiny app **Data source descriptions shiny app** PhenotypeR shiny app

PhenotypeR



The PhenotypeR package helps us to assess the research-readiness of a set of cohorts we have defined. This assessment includes:

- **Database diagnostics** which help us to better understand the database in which they have been created. This includes information about the size of the data, the time period covered, the number of people in the data as a whole. More granular information that may influence analytic decisions, such as the number of observation periods per person, is also described.
- **Codelist diagnostics** which help to answer questions like what concepts from our codelist are used in the database? What concepts were present led to individuals' entry in the cohort? Are there any concepts being used in the database that we didn't include in our codelist but

Links
[View on CRAN](#)
[Browse source code](#)
[Report a bug](#)

License
[Full license](#)
Apache License (>= 2)

Citation
[Citing PhenotypeR](#)

Developers

Extra: Adding context to your results via data source description

- Visit [Data source description shiny app](#)

Data source description

A specification for data source descriptions

Administrative details

Name of data source ⓘ

Data source acronym ⓘ

Data source website ⓘ

HMA-EMA catalogue entry ⓘ

Main references ⓘ

Data collected

Database Description

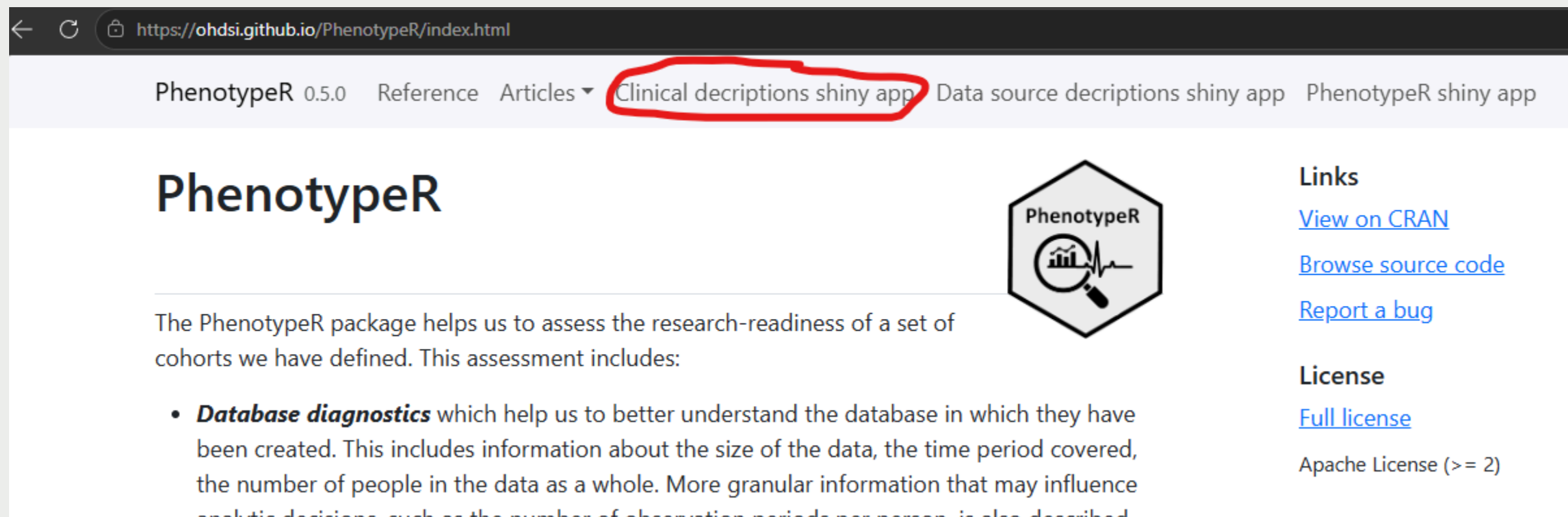
Geography ⓘ

OHDSI-UK 2026 Training

17

Extra: Adding context to your results via clinical descriptions

- Visit [Clinical Descriptions shiny app](#)



The screenshot shows the PhenotypeR website at <https://ohdsi.github.io/PhenotypeR/index.html>. The navigation bar includes links for PhenotypeR 0.5.0, Reference, Articles, Clinical descriptions shiny app (highlighted with a red circle), Data source descriptions shiny app, and PhenotypeR shiny app. The main content area features the PhenotypeR logo, a description of the package's purpose, and a list of features including Database diagnostics. A sidebar on the right contains links to View on CRAN, Browse source code, Report a bug, and a License section with a link to the Full license.

PhenotypeR 0.5.0 Reference Articles **Clinical descriptions shiny app** Data source descriptions shiny app PhenotypeR shiny app

PhenotypeR

The PhenotypeR package helps us to assess the research-readiness of a set of cohorts we have defined. This assessment includes:

- **Database diagnostics** which help us to better understand the database in which they have been created. This includes information about the size of the data, the time period covered, the number of people in the data as a whole. More granular information that may influence analytic decisions, such as the number of observation periods per person, is also described.

Links

- [View on CRAN](#)
- [Browse source code](#)
- [Report a bug](#)

License

[Full license](#)

Apache License (≥ 2)

Extra: Adding context to your results via clinical descriptions

- Visit [Clinical descriptions shiny app](#)

Clinical Description

A clinical description to inform development of computable phenotypes using real world health data.

🔍 Phenotype name

Phenotype Name ⓘ

✍️ DRAFT WITH AI

🔍 Metadata

Version ⓘ

Created By ⓘ

Created Date ⓘ

2026-06-18

Last Edited By ⓘ

Last Edited Date ⓘ

2026-06-18

Source of Information ⓘ

🔍 Clinical Profile

Introduction & Synonyms ⓘ

Import your descriptions in the shiny app:

```
shinyDiagnostics(result = result,  
  directory = here(),  
  minCellCount = 5,  
  clinicalDescriptionsDir = NULL,  
  databaseDescriptionsDir = NULL,  
  removeEmptyTabs = FALSE)
```

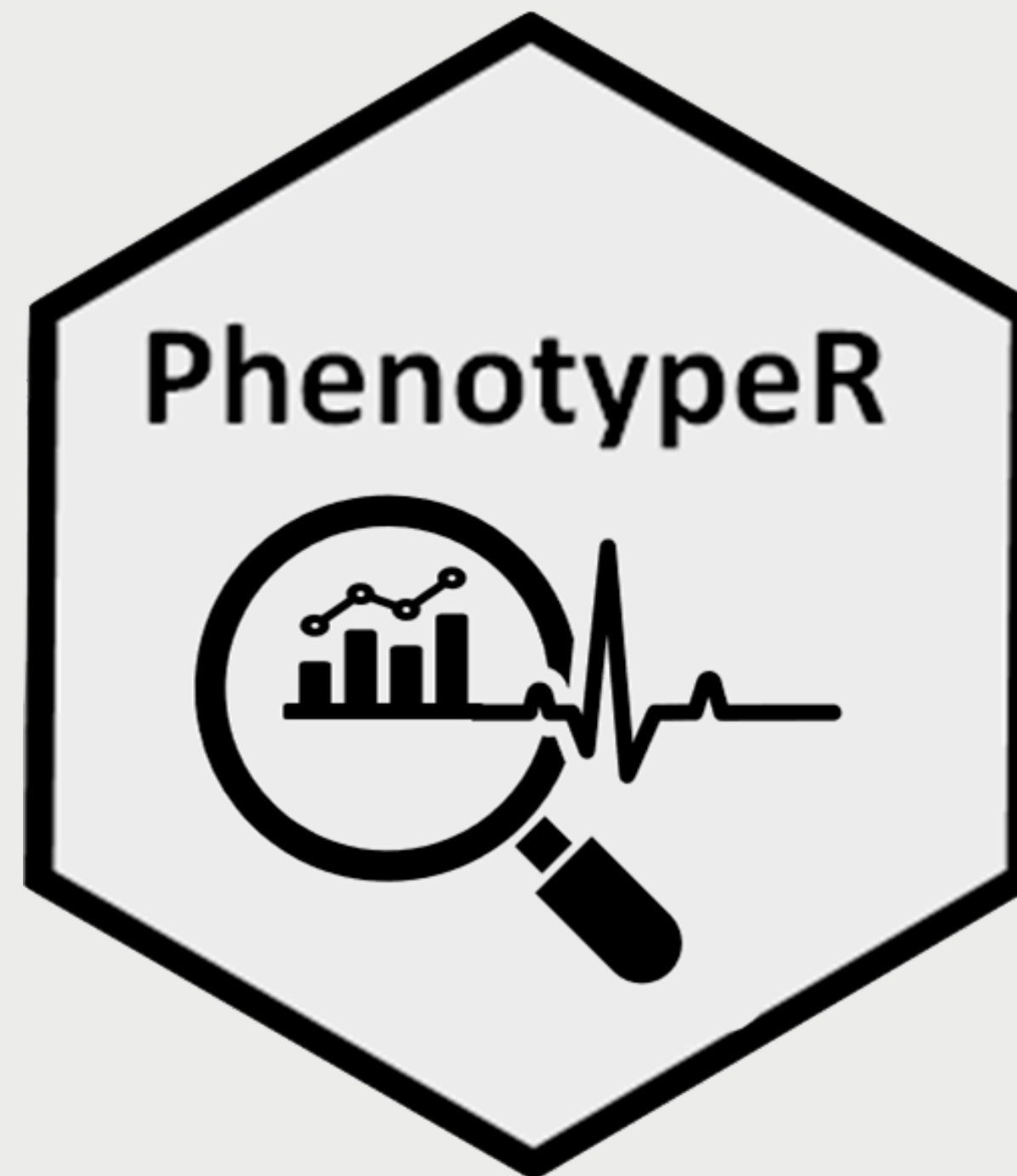
PhenotypeR

👉 [Package website](#)

👉 [CRAN link](#)

👉 [GitHub link](#)

👉 [Manual](#) 

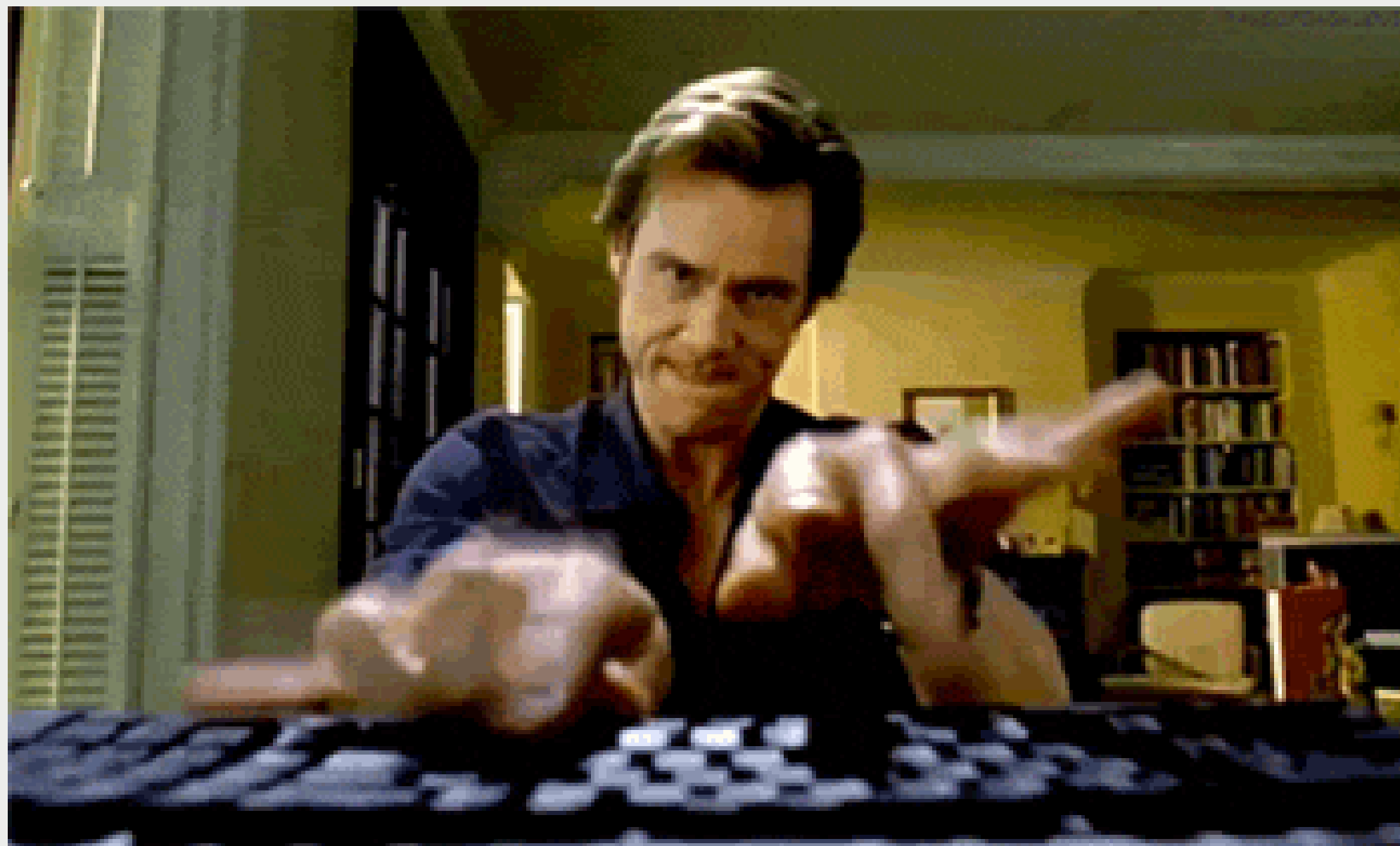


Final remarks

OHDSI-UK 2026 Training

2026-09-17

What We Promised



What We Promised

By the end of the week:

- Be able to conduct descriptive analyses using R with data in the OMOP CDM format
- Understand how to approach writing bespoke code for complex studies
- Know where to look for help

Feedback

We'd love to hear your thoughts!

Please take a moment to fill out the feedback form — especially the questions related to the **Analytics track**.

All feedback is welcome — both positive and critical.

We hope the course met your expectations and helped you grow your skills.

Want more?

Each of the packages we covered this week has its own website with vignettes and documentation to help you go further.

Packages summary

You can explore all the packages we've discussed — and more — on our website.

Stay connected

Want updates on new packages, methods, and publications?

Subscribe to our monthly newsletter — just one brief update on the 1st of each month:

 **Newsletter**

Have questions or suggestions? Please open a GitHub issue or email us. We welcome contributions to all packages hosted in the **OHDSI** organisation.

Bonus resource

We've put together a book that complements this week's course:

 **Tidy R Programming with OMOP**

If you find it helpful, please  the repository on GitHub — and feel free to open an issue if anything's unclear. Your feedback helps us improve.

Youtube channel

We have a YouTube channel with some videos that might be interesting:

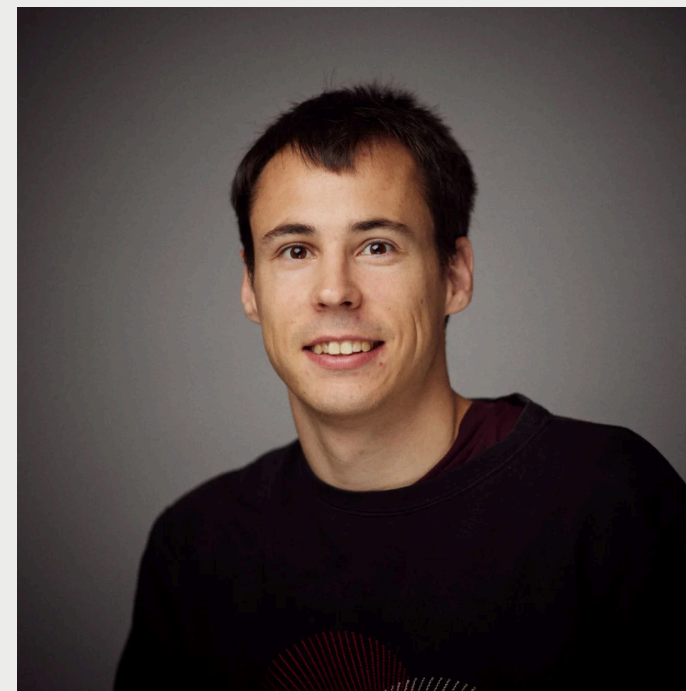
 **YouTube channel**

Course materials

All morning session materials will be shared with you at the end of the course.

The afternoon session materials will remain available online for **two weeks** after the course ends. After that, the website will be taken down and the repository made private. Please download any slides or resources you'd like to keep before then.

Thanks to All Speakers



Thanks!

Thank you all for this amazing week 😊

We hope to see you again soon!